



VRIJE
UNIVERSITEIT
BRUSSEL



Master thesis submitted in partial fulfilment of the requirements for the degree of
Master of Science in Applied Sciences and Engineering: Computer Science

CONVOLUTIONAL TENSOR NETWORKS FOR WEIGHT-BASED MECHANISTIC INTERPRETABILITY

Antoine Dehérand

2025-2026

Promotor(s): Prof. Dr. Dr. Geraint Wiggins, Ward Gauderis
Science and Bio-Engineering Sciences



VRIJE
UNIVERSITEIT
BRUSSEL



Proefschrift ingediend met het oog op het behalen van de graad van Master of
Science in Applied Sciences and Engineering: Computer Science

CONVOLUTIONELE TENSORNETWERKEN VOOR GEWICHTSGEBASEERDE MECHANISTISCHE INTERPRETEERBAARHEID

Antoine Dehérand

2025-2026

Promotor(s): Prof. Dr. Dr. Geraint Wiggins, Ward Gauderis
Wetenschappen en Bio-ingenieurswetenschappen

Abstract

Mechanistic weight-based interpretability aims to reverse-engineer neural networks to make them interpretable based on their weights. The recent works of [Pearce et al., 2025] and [Dooms et al., 2025] have demonstrated how tensor networks can be used to interpret bilinear feed-forward neural networks. In this work, we extend these ideas for convolutional neural networks (CNNs). We investigate how to design a convolutional neural network that achieves competitive performance while remaining convertible to an equivalent tensor network, which is the representation we use to make it interpretable. We further investigate what weight-based insights can be extracted from that tensor network representation by analysing the eigenspectrum of the tensor network.

We propose a multi-stage deep residual CNN architecture built entirely from components that are individually convertible to an equivalent tensor network. The proposed architecture is benchmarked on MNIST, CIFAR-10, CIFAR-100, and SVHN, and compared against state-of-the-art CNNs, which are ResNet, DenseNet, Wide ResNet, and PyramidNet. Despite the architectural constraints imposed by the tensor network conversion requirement, the model achieves competitive performance on MNIST, CIFAR-10, and SVHN.

To enable the weight-based interpretability analysis, a complete tensor network conversion pipeline is developed that covers every component of the proposed architecture. Due to finite floating-point arithmetic, the conversion shows numerical differences although being mathematically equivalent to the model. The largest numerical difference we observed is a mean absolute error of 6.54×10^{-5} between the model output and the tensor network contraction given the same input. A small and shallow case of this conversion is analysed using the weight-based interpretability technique from [Pearce et al., 2025], which we extend from bilinear feed-forward neural networks to bilinear convolutional neural networks. The analysis reveals that the eigenvectors extracted from the convolutional model tensor network are spatially localised and coherent with the classified digit. Models with smaller kernels produce eigenvectors corresponding to individual digit strokes, while larger kernels capture larger patterns. Compared to the feed-forward setting, the convolutional eigenvectors are more compact and coherent, suggesting that the learned patterns are more human-interpretable since they are more robust to spurious correlations in the training data. The technique is also used for instance-based explainability, revealing which patterns support and suppress individual classification decisions. The results within this work show that a shallow instantiation of the proposed architecture yields meaningful weight-based insights from its equivalent tensor network. This provides a promising foundation for extending the approach to deeper and more complex models. The code for all experiments is publicly available on GitHub¹.

¹Link to GitHub repository

Abstract

Mechanistische gewichtsgebaseerde interpreteerbaarheid heeft als doel neurale netwerken te reverse-engineeren om ze interpreteerbaar te maken op basis van hun gewichten. De recente werken van [Pearce et al., 2025] en [Dooms et al., 2025] hebben aangetoond hoe tensornetwerken gebruikt kunnen worden om bilineaire feed-forward neurale netwerken te interpreteren. In dit werk breiden we deze ideeën uit naar convolutionele neurale netwerken (CNN's). We onderzoeken hoe een convolutioneel neuraal netwerk, dat competitieve prestaties behaalt, ontworpen kan worden terwijl het omzetbaar blijft naar een equivalent tensornetwerk. De representatie die we hiervoor gebruiken is een tensornetwerk. We onderzoeken verder welke gewichtsgebaseerde inzichten uit die tensornetwerkrepresentatie geëxtraheerd kunnen worden door het eigenspectrum van het tensornetwerk te analyseren.

We stellen een meerstaps diep residueel CNN-architectuur voor, volledig opgebouwd uit componenten die individueel omzetbaar zijn naar een equivalent tensornetwerk. De voorgestelde architectuur wordt geëvalueerd op MNIST, CIFAR-10, CIFAR-100 en SVHN, en vergeleken met de meest geavanceerde CNN-modellen, namelijk ResNet, DenseNet, Wide ResNet en PyramidNet. Ondanks de architecturale beperkingen opgelegd door de vereiste van tensornetwerkomzetting, behaalt het model competitieve prestaties op MNIST, CIFAR-10 en SVHN.

Om de gewichtsgebaseerde interpreteerbaarheidsanalyse mogelijk te maken, wordt een volledige omzettingspijplijn voor tensornetwerken ontwikkeld die alle componenten van de voorgestelde architectuur omvat. Door eindige floating-point rekenkunde vertoont de omzetting numerieke verschillen, hoewel ze wiskundig equivalent is aan het model. Het grootste numerieke verschil dat we waarnamen is een gemiddelde absolute fout van 6.54×10^{-5} tussen de modeluitvoer en de tensornetwerkcontractie bij dezelfde invoer. Een klein, ondiep en vereenvoudigd geval van deze omzetting wordt geanalyseerd met behulp van de gewichtsgebaseerde interpreteerbaarheidstechniek van [Pearce et al., 2025], die we uitbreiden van bilineaire feed-forward neurale netwerken naar bilineaire convolutionele neurale netwerken. De analyse toont aan dat de eigenvectoren geëxtraheerd uit het convolutionele modeltensornetwerk ruimtelijk gelokaliseerd en coherent zijn met het geclassificeerde cijfer. Modellen met kleinere kernels produceren eigenvectoren die overeenkomen met individuele cijferstrepen, terwijl grotere kernels grotere patronen vastleggen. Vergeleken met de feed-forward setting zijn de convolutionele eigenvectoren compacter en coherenter, wat suggereert dat de geleerde patronen beter menselijk interpreteerbaar zijn omdat ze robuuster zijn tegen spurieuze correlaties in de trainingsdata. De techniek wordt ook gebruikt voor instantiegebaseerde verklaarbaarheid, waarbij wordt onthuld welke patronen individuele classificatiebeslissingen ondersteunen en onderdrukken. De resultaten in dit werk tonen aan dat een ondiepe instantiatie van de voorgestelde architectuur betekenisvolle gewichtsgebaseerde inzichten opleveren vanuit hun equivalent tensornetwerk. Dit biedt een veelbelovende basis voor het uitbreiden van de aanpak naar diepere en complexere modellen. De code van alle experimenten is publiek beschikbaar op GitHub².

²Link to GitHub repository

Acknowledgements

I would like to thank Prof. Dr. Dr. Geraint Wiggins and Ward Gauderis for allowing me to work freely on this project and shape it as I wanted. They both ensured to provide me with great support and get the best out of me. The weekly meetings with Ward Gauderis and the insights he provided were particularly valuable throughout this process. Finally, I would like to thank my family and my girlfriend for their unconditional support throughout this thesis.

Contents

1	Introduction	7
2	Literature review	9
2.0.1	Overview	9
2.1	Tensors	9
2.1.1	Notation	10
2.1.2	Operations	11
2.1.3	Limitations	14
2.2	Tensor networks	14
2.2.1	Origins of tensor networks	14
2.2.2	Notation	15
2.2.3	Special tensor types	18
2.2.4	Multi-stage decompositions	20
2.2.5	Tensor networks in other fields	29
2.2.6	Limitations	30
2.3	Convolutional Neural Networks	30
2.3.1	Machine learning	30
2.3.2	Deep learning	31
2.3.3	Neural networks	31
2.3.4	Model intuition	34
2.3.5	Kernel	35
2.3.6	Architecture	36
2.3.7	Training	38
2.4	Interpretability for neural networks	40
2.4.1	Post-hoc interpretability	41
2.4.2	Mechanistic interpretability	41
2.4.3	Tensor networks as an interpretability tool	42
3	Methods	44
3.0.1	Overview	44
3.1	Convolutional neural network architecture	44
3.1.1	Architecture	44
3.1.2	Residual block	45
3.1.3	Bilinear layer	45
3.1.4	Normalisation	47
3.1.5	Implementation and training	48
3.1.6	Hyperparameters	50

3.2	Tensor network conversion	51
3.2.1	Computational simplifications	51
3.2.2	Conversion outline	54
3.2.3	Convolutional layer	59
3.2.4	Bilinear layer	60
3.2.5	Normalisation	62
3.2.6	Residual block	65
3.2.7	Pooling layers	67
3.2.8	Flatten	70
3.2.9	Linear layer	71
3.2.10	Complete tensor network architecture	72
3.2.11	Implementation	72
3.3	Tensor network analysis	74
3.3.1	Spectral decomposition	74
3.3.2	Eigenspectrum interpretation	75
3.3.3	Convolutional neural network extension	76
4	Results	78
4.0.1	Overview	78
4.1	Model performance	78
4.2	Tensor network conversion accuracy	79
4.3	Interpretability analysis	80
4.3.1	Hyperparameter configuration	82
4.3.2	Eigenspectrum analysis	82
4.3.3	Instance-based explainability	84
5	Discussion	87
5.0.1	Overview	87
5.1	Model performance	87
5.2	Tensor network conversion accuracy	88
5.3	Weight-based interpretability	88
5.3.1	Eigenspectrum analysis	88
5.3.2	Instance-based explainability	90
5.4	Limitations	90
5.5	Future work	90
6	Conclusion	92
A	Auxiliary proofs	99
A.1	Center of orthogonality decomposition	99
A.2	RMS normalisation scaling property	100
B	Hyperparameters	101
B.1	Hyperparameter definitions	101
B.2	Hyperparameter values	102
C	Weight-based interpretability examples	107

Chapter 1

Introduction

Artificial Intelligence (AI) systems are almost everywhere nowadays. People use them to generate images, get advice, automate systems, and more. Our dependency on those intelligent systems continues to grow as they facilitate the lives of many. On the other hand, it also increases the demand for more transparent systems. This need is especially important in situations where AI systems are used in critical domains such as healthcare, the military, and finance. Inaccurate predictions in those fields can have serious consequences on human life. That is why our work focuses on analysing AI systems to make them more transparent. We especially focus on systems built for tasks involving images, such as image classification, object detection, face recognition, and more. Those systems make use of a family of machine learning models called Convolutional Neural Networks (CNNs). Recent works [Dooms et al., 2025, 2026; Gonzalez et al., 2026; Pearce et al., 2025] have demonstrated how tensor networks can be used to make feed-forward neural networks interpretable. We extend the work of [Pearce et al., 2025] to CNNs and aim to make them interpretable. To this end, the CNN must first be converted to an equivalent tensor network, and that tensor network is later analysed using linear algebra techniques. The conversion imposes several restrictions on the CNNs we can design. First, for every building block we use in our CNNs, it must be convertible to an equivalent tensor network. Second, despite being convertible to a tensor network, the resulting CNN should also have comparable performance with state-of-the-art CNNs on standard benchmarking datasets to be used as a credible interpretable substitute. This introduces another challenge since most building blocks in state-of-the-art CNNs are not convertible to tensor networks because of their non-linear properties. The weight-based analysis we conduct on bilinear convolutional networks by extending the work of [Pearce et al., 2025] reveals that more structured deep learning architectures not only achieve better task performance, but also learn more human-interpretable patterns compared to feed-forward neural networks. The learned patterns are more localised, structured, and coherent with the underlying data, suggesting that there is an increase in robustness to spurious correlations in the training data.

Formally, we explore the following research question:

How can a convolutional neural network be designed to achieve competitive performance while remaining convertible to an equivalent tensor network, and what interpretable weight-based insights can be extracted from the tensor network analysis of that CNN?

The contributions of this thesis can be summarised as follows:

1. We provide an overview of tensors, tensor networks, convolutional neural networks, and

existing interpretability work for neural networks. For each concept, we cover the basics and situate its role within this thesis.

2. We introduce a novel CNN architecture and evaluate its performance against state-of-the-art CNNs (ResNet, DenseNet, Wide ResNet, PyramidNet) on standard benchmarking datasets MNIST, CIFAR-10, CIFAR-100, and SVHN. The results show that our model achieves performance comparable to these architectures on MNIST, CIFAR-10, and SVHN.
3. We show how the proposed architecture is converted to an equivalent tensor network by discussing how each of its components is represented as an equivalent tensor network component. The conversion is mathematically exact, but due to finite floating-point arithmetic, some differences may be observed between the model's output and the tensor network output. We empirically demonstrate that the conversion technique achieves a maximum mean absolute error of 6.54×10^{-5} , which is negligibly small in practice.
4. We build, train, and analyse a shallow CNN model for digit classification, and extract interpretable patterns not accessible through existing weight-based techniques. The extracted patterns are visualised in the input space of images and linked to digit classes. They reveal what patterns the model uses to support or suppress an output class. We compare these patterns with those of a feed-forward neural network and discuss structural differences. Finally, we illustrate how the proposed technique can be used for instance-based explainability.
5. The code for all experiments is publicly available on GitHub¹. The results of all conducted experiments can be reproduced using this code.

The remainder of this thesis is structured as follows. Chapter 2 presents an extensive literature review of tensors, tensor networks, CNNs, and existing interpretability work. Chapter 3 presents the proposed convolutional neural network architecture, describes its conversion to an equivalent tensor network, and outlines the techniques used to analyse it. The proposed model is evaluated on standard benchmarking datasets against state-of-the-art CNNs not designed for weight-based interpretability in chapter 4. This chapter also contains the results of the tensor network conversion accuracy and the tensor network analysis conducted on MNIST. A detailed discussion and interpretation of these results is given in chapter 5. Finally, chapter 6 concludes this thesis.

¹Link to GitHub repository

Chapter 2

Literature review

2.0.1 Overview

This chapter presents a literature review covering the four pillars of this thesis. We begin with tensors, the basic building blocks of tensor networks, and then introduce tensor networks. We then turn our attention to convolutional neural networks, the family of models we aim to interpret. Finally, we discuss existing interpretability work for neural networks. In particular, we discuss mechanistic interpretability, whose goal is to reverse-engineer neural networks to understand their working [Bereska and Gavves, 2024]. It forms the bridge between tensor networks and convolutional neural networks [Gauderis et al., 2026]. It outlines why and how tensor networks can help us study the weights of trained neural networks as a new data modality.

2.1 Tensors

The basic building blocks of this work are tensors. They can be seen as a natural generalisation of vectors and matrices [Bridgeman and Chubb, 2017; Levine et al., 2017]. While vectors and matrices are respectively one-dimensional and two-dimensional arrays, tensors are multi-dimensional arrays [Novikov et al., 2015]. Each dimension of such an array is called a mode [Wang et al., 2025]. A vector has one mode, while a matrix has two modes. The number of modes of a tensor is referred to as its order. Each mode has an associated size called the dimension of that mode, which specifies how many entries the tensor contains along that mode. A synonym for a mode of a tensor is an index, and we use both interchangeably throughout this work.

To build an intuition about tensors beyond the vector and matrix case, figure 2.1 illustrates tensors of increasing order. The zeroth-, first-, and second-order tensors correspond to familiar concepts. These are respectively the scalar, the vector, and the matrix. A third-order tensor can then be understood as a natural extension of a second-order tensor (matrix), which is formed by stacking multiple second-order tensors along a new third mode. In our illustration, those matrices were stacked in depth, resulting in a rectangular prism, which represents the third-order tensor. This idea generalises to tensors of arbitrary order; each additional mode introduces a new axis along which tensors of the previous order are organised. In this sense, every new mode simply provides an additional axis for storing and organising tensors.

To clarify, consider a unifying example of a ubiquitous third-order tensor we encounter in our everyday life: an RGB image. This example is illustrated in figure 2.2. The figure proposes three different views of an RGB image. From a tensor perspective, the image is a third-order tensor that consists of three modes, which are the channels, height, and width of the image.

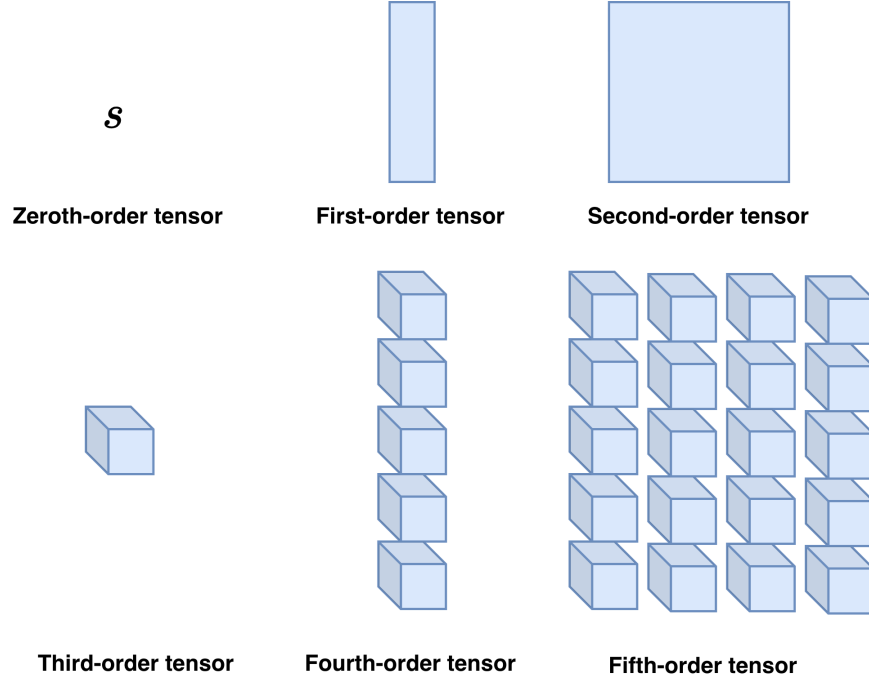


Figure 2.1: Illustration of tensors of increasing order as per [Jain, 2023]. The zeroth-, first-, and second-order cases respectively correspond to a scalar s , a vector, and a matrix. Higher-order tensors are constructed by stacking tensors of the preceding order along their new mode.

The dimension of the channel mode is fixed and is equal to three, representing the red, green, and blue channels. The dimensions of the height and width modes correspond to the spatial resolution of the image. Another perspective involves looking at the values of the tensor entries themselves. Each colour channel consists of 8-bit values for each spatial position set arbitrarily in this example.

2.1.1 Notation

With the intuition in place, we define the tensor notation. Depending on the context in which tensors appear, different notations are used. The two most common notations found in the literature are the mathematical and the diagrammatic notations. The mathematical notation, called the Einstein notation, was developed first. Then followed the diagrammatic notation, called the Penrose notation, that is more visual and intuitive [Coecke and Kissinger, 2017]. In this work, we introduce both notations, but we later work with the diagrammatic notation. The reason why will become clear once we have developed enough intuition about the mathematical notation and tensors to appreciate the expressive power of the diagrammatic notation.

Let $\mathcal{T}$ be a tensor of order N with d_i denoting the dimension of each mode $i \in \{1, \dots, N\}$, then the set of all configurations the tensor $\mathcal{T}$ can take is $\mathbb{R}^{d_1 \times \dots \times d_N}$ [Levine et al., 2017]. This represents the set of all N th-order tensors of shape $d_1 \times \dots \times d_N$ whose entries are real numbers. One element of tensor $\mathcal{T}$ is referred to as $\mathcal{T}_{i_1, \dots, i_N}$ with $i_1, \dots, i_N$ the entries of that element. For example, the order N of an RGB image is equal to 3 and hence $\mathcal{T} \in \mathbb{R}^{d_1 \times d_2 \times d_3}$. The first mode ($d_1 = 3$) corresponds to the colour channels, while the second and third modes (d_2 and

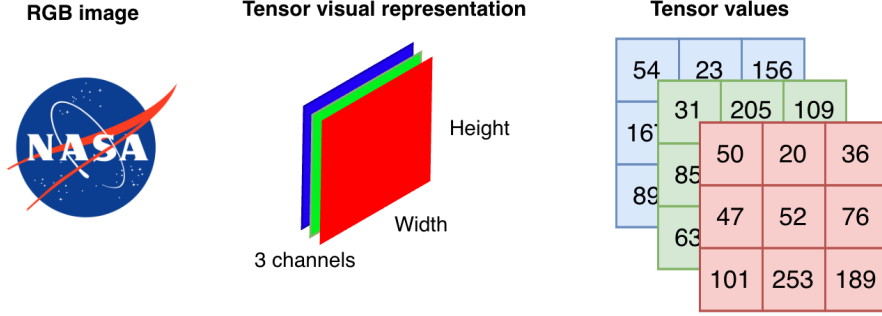


Figure 2.2: Illustration of an RGB image as a third-order tensor. The three modes of the tensor correspond to the channel, height, and width. The channel mode has a dimension of three, representing the red, green, and blue components. Each channel forms a two-dimensional array of pixel values, ranging from $[0, 255]$ for 8-bit images. The numerical values shown are illustrative only and do not correspond to the original image. For the sake of clarity, the spatial dimension of the image was reduced to 3×3 .

d_3) respectively correspond to the height and width of the image. The modes of a tensor $\mathcal{T}$ can be assigned names to reference throughout explanations. With this mathematical notation introduced, we next consider the operations that can be performed on tensors.

2.1.2 Operations

Until now, we have been introducing tensors as a structured storage mechanism. We can store multi-dimensional data along multiple modes, but we have not yet discussed what we can do with those tensors. Their true usefulness emerges when tensors are used as parameters of other operations such as a linear transformation, or when operations are applied directly to tensors to combine and transform them. The latter forms the basis of tensor networks introduced later in section 2.2.

Tensor product. The first operation we discuss is the tensor product, denoted by $\otimes$. Intuitively, this operator combines two tensors into a single tensor by forming all pairwise products of their entries. Formally, the tensor product takes two tensors $\mathcal{A} \in \mathbb{R}^{d_1 \times \dots \times d_P}$ (order P) and $\mathcal{B} \in \mathbb{R}^{d_{P+1} \times \dots \times d_{P+Q}}$ (order Q) and produces a tensor $\mathcal{T}$ of order $P + Q$ denoted as

$$\mathcal{T} = \mathcal{A} \otimes \mathcal{B} \in \mathbb{R}^{d_1 \times \dots \times d_{P+Q}}$$

whose entries are defined by $\mathcal{T}_{i_1, \dots, i_P, j_1, \dots, j_Q} = \mathcal{A}_{i_1, \dots, i_P} \mathcal{B}_{j_1, \dots, j_Q}$ [Levine et al., 2017]. To illustrate this operation, consider two 2×2 matrices, which are second-order tensors, $\mathcal{A}$ and $\mathcal{B}$ defined below

$$\mathcal{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \mathcal{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

The tensor product $\mathcal{A} \otimes \mathcal{B}$ is a fourth-order tensor $\mathbb{R}^{2 \times 2 \times 2 \times 2}$ equal to

$$\mathcal{A} \otimes \mathcal{B} = \begin{bmatrix} 1 \cdot \mathcal{B} & 2 \cdot \mathcal{B} \\ 3 \cdot \mathcal{B} & 4 \cdot \mathcal{B} \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} & \begin{bmatrix} 10 & 12 \\ 14 & 16 \end{bmatrix} \\ \begin{bmatrix} 15 & 18 \\ 21 & 24 \end{bmatrix} & \begin{bmatrix} 20 & 24 \\ 28 & 32 \end{bmatrix} \end{bmatrix}$$

The resulting tensor is obtained by multiplying each entry of $\mathcal{A}$ by the entire tensor $\mathcal{B}$.

Tensor contraction. The next essential operation is the tensor contraction, which generalises the matrix multiplication to tensors of arbitrary order. The tensor contraction combines two tensors along one or more shared indices, producing a single tensor that contains all remaining unshared indices of both tensors [Evenbly, 2022]. This operation is best understood and explained through examples. Consider a third-order tensor $\mathcal{A}$ with its indices named i, j, k and a second-order tensor $\mathcal{B}$ with indices l, m .

$$\mathcal{A} = \left[\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \right] \quad \mathcal{B} = \begin{bmatrix} 9 & 10 \\ 11 & 12 \end{bmatrix}$$

Since the tensor contraction combines two tensors along one or more shared indices, this implies that there are multiple ways to contract tensors with each other. The simplest case involves contracting two tensors along a single shared index. For instance, we can contract tensors $\mathcal{A}$ and $\mathcal{B}$ along index k of $\mathcal{A}$ and index l of $\mathcal{B}$. This is only possible if the contracted indices have the same dimension, i.e., $d_k = d_l$. This contraction results in a third-order tensor $\mathcal{C}$ with the remaining indices of both tensors i, j, m and is obtained by summing over the shared index k or l computed by

$$\mathcal{C}_{ijm} = \sum_{k=1}^{d_k} \mathcal{A}_{ijk} \mathcal{B}_{km} = \sum_{l=1}^{d_l} \mathcal{A}_{ijl} \mathcal{B}_{lm} \quad \text{and hence} \quad \mathcal{C} = \left[\begin{bmatrix} 31 & 34 \\ 71 & 78 \end{bmatrix} \begin{bmatrix} 111 & 122 \\ 151 & 166 \end{bmatrix} \right]$$

The contracted index was removed by the tensor contraction operation. This is analogous to the matrix multiplication operation, which contracts over a single common index, which is forced to be the column of the first matrix and the row of the second matrix. In fact, a matrix multiplication is a tensor contraction, but a restricted one. Both tensors must be of second order, and the shared index is fixed. With tensors of arbitrary order, we can decide what indices are contracted. For instance, we could have also decided to contract on indices $j = m$ or $i = l$. This is possible only if the dimensions of the pairs of contracted indices match. Let $\mathcal{A}$ and $\mathcal{B}$ be contracted over two indices, say $j = l$ and $k = m$. The resulting contraction is expressed as

$$\mathcal{D}_i = \sum_{j=1}^{d_j} \sum_{k=1}^{d_k} \mathcal{A}_{ijk} \mathcal{B}_{jk} \quad \text{and hence} \quad \mathcal{D} = [110 \quad 278]$$

resulting in a tensor $\mathcal{D}$ with the only remaining index i . In summary, a tensor contraction contracts shared indices by summing over them, producing a tensor of order equal to the total number of remaining unshared indices. With this new operation introduced, we can revisit the tensor product from a different perspective. The tensor product can be seen as a special case of a tensor contraction in which no indices are shared between the two tensors. While this may seem counterintuitive since nothing is contracted, the result is equivalent to the tensor product as defined previously.

Tensor trace. The binary tensor contraction operation has a unary variant, known as the tensor trace [Evenbly, 2022]. In the tensor trace, one or more pairs of indices within a single tensor are contracted, producing a tensor of reduced order. Depending on the indices that are chosen, we distinguish between two types of tensor traces: partial and complete tensor traces [Coecke and Kissinger, 2017]. A partial trace occurs when only a subset of the tensor's indices is contracted, and a complete trace occurs when all indices are contracted. Consider a fourth-order

tensor $\mathcal{A}$ with indices i, j, k, l . A contraction performed on the second and third indices (j and k , under the assumption that $d_j = d_k$) results in a second-order tensor $\mathcal{E}$ which is mathematically defined as

$$\mathcal{E}_{il} = \sum_{j=1}^{d_j} \mathcal{A}_{ijjl}$$

A complete tensor trace is achieved when all indices of a tensor are contracted in pairs, resulting in a scalar (zeroth-order tensor). This means that a complete tensor trace can only be applied to a tensor with even order, and the dimensions of the paired indices must be compatible. For the fourth-order tensor $\mathcal{A}$, one possible complete trace involves contracting the first index with the third, and the second index with the fourth

$$\text{Ttr}(\mathcal{A}) = \sum_{i=1}^{d_i} \sum_{j=1}^{d_j} \mathcal{A}_{ijij}$$

So far, the operations we discussed produce new tensors by combining entries of other tensors. We now turn our attention to operations of a different nature; these are the permutation and reshape operations. Those operations reorganise the structure of a tensor without altering its values.

Tensor permutation. The permute operation allows the index ordering of a tensor to be changed [Evenbly, 2022]. Instead of fixing the position of an index, it can be moved to another position with the permute operation. For example, the indices i, j, k of a third-order tensor $\mathcal{A}$ can be reordered according to any valid permutation of the indices. This does not change the content of the tensor, but its representation. If the new permutation is j, i, k then $\mathcal{A}_{ijk}$ is now stored at $\mathcal{A}_{jik}$. Some software libraries, like NumPy and PyTorch, allow the indices to be permuted without moving the underlying data, so the tensor's storage remains unchanged. Reordering the indices of the third-order tensor $\mathcal{A}$ as j, i, k yields the tensor $\tilde{\mathcal{A}}$ with

$$\mathcal{A} = \begin{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} & \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \end{bmatrix} \quad \tilde{\mathcal{A}} = \begin{bmatrix} \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} & \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix} \end{bmatrix}$$

Tensor reshape. The reshape operation changes the order of a tensor by combining or splitting its indices [Evenbly, 2022]. The total number of elements remains the same, but their ordering is adjusted to reflect the new order. For example, the third-order tensor $\mathcal{A}$ can be reshaped by combining indices j and k into a single new index l . This produces a second-order tensor $\mathcal{B}$ with indices i and l , where $d_l = d_j \cdot d_k$. All indices can also be combined into a single index, representing the flatten operation. This results in a first-order tensor $\tilde{\mathcal{B}}$ with

$$\mathcal{B} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{bmatrix} \quad \tilde{\mathcal{B}} = [1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad 7 \quad 8]$$

The first-order tensor $\tilde{\mathcal{B}}$ demonstrates that the underlying data is unchanged, but its representation has been modified. Using this reshaping operation, a tensor of any order can be converted into a matrix. This enables the application of linear algebra concepts and techniques to tensors of any order. It allows us, among other things, to make the bridge between tensor networks and mechanistic interpretability.

All operations discussed so far are linear. This property is fundamental and will become central when we discuss how tensor networks can be used as an interpretability tool in subsection 2.4.2.

These operations will be revisited later in subsection 2.2.2 with the diagrammatic notation. With that notation, many of the operations become more intuitive and easier to represent.

2.1.3 Limitations

Despite their ability to store multi-dimensional data and apply operations on them, tensors have one important limitation. The number of elements in a tensor scales exponentially with its order N . This problem is known as the curse of dimensionality [Levine et al., 2017]. To illustrate this, consider an N th-order tensor where we assume without loss of generality that every mode has dimension d . The total number of elements is the product of the dimensions across all modes. This results in $d \cdot d \cdot \dots \cdot d = d^N$ elements. Adding a mode of dimension d increases the number of elements to $d^N \times d = d^{N+1}$, hence the exponential growth in N . For that reason, tensors quickly become impractical to store and process. For instance, a 6th-order tensor with $d = 10$ already contains one million elements, and adding one additional mode of dimension 10 increases this number to ten million. At first glance, this problem does not appear to be directly solvable since one needs to store all the data after all. In this sense, the curse of dimensionality seems unavoidable. Nevertheless, an effective approach exists that mitigates these issues at a certain cost, which is tensor networks [Bridgeman and Chubb, 2017]. Instead of storing and manipulating a large higher-order single tensor, a tensor network represents the higher-order tensor as a collection of lower-order tensors connected through tensor contractions. There is no need to store the large tensor, but now we must store a collection of lower-order tensors. If those tensors are sufficiently small and the tensor network is constructed appropriately, the total number of elements can be drastically reduced and become polynomial in N . From this perspective, a tensor network acts as a compression strategy by decomposing a single tensor into multiple smaller interconnected tensors.

Replacing a single tensor with a tensor network comes with two main drawbacks. First, since the full high-order tensor is no longer stored explicitly, accessing a single tensor element is no longer instantaneous and requires additional polynomial computations. Second, the resulting tensor network generally provides an approximation of the original tensor. However, the approximation error can be systematically controlled, making tensor networks suitable even for applications requiring high accuracy [Levine et al., 2017; Wang et al., 2025]. Tensor networks form the subject of the next section, in which we discuss their origins, the diagrammatic notation, and how to create such a tensor network from a single tensor.

2.2 Tensor networks

This section is heavily based on the tensor tutorials provided by Glen Evenbly¹. The tutorials build an intuition about tensor networks from the ground up and also provide links to additional resources on the subject.

2.2.1 Origins of tensor networks

Tensor networks were originally developed in quantum physics to represent the wave function of large quantum many-body systems efficiently [Biamonte and Bergholm, 2017; Hamreras et al., 2025; Levine et al., 2017; Stoudenmire and Schwab, 2016]. In such systems, quantum physicists aim to describe the joint state of multiple particles. This joint state can be represented as a tensor, where each mode corresponds to a particle and the dimension of each mode corresponds

¹<https://www.tensors.net/>

to the number of possible states that particle can be in. Each entry of the tensor is associated with a specific configuration of the entire system; it represents a joint state of all particles [Levine et al., 2017]. However, as discussed previously, explicitly working with a single high-order tensor quickly becomes infeasible due to the curse of dimensionality. Adding a new particle to the system increases the number of elements in the tensor exponentially. That is why tensor networks were originally introduced to overcome these limitations by decomposing a high-order tensor into a structured collection of low-order tensors. This decomposition, if done appropriately, reduces the number of elements from exponential to polynomial, enabling efficient storage and computation [Wang et al., 2025]. This way, tensor networks make it feasible to simulate large-scale quantum systems [Nie et al., 2025].

2.2.2 Notation

In the previous section, we introduced the mathematical notation for tensors. However, in this work we use the diagrammatic notation. Since our work involves systems where multiple tensors are involved, which are tensor networks, the presented mathematical notation falls short due to its lack of readability in equations. The more tensors are involved, the more complex and unclear the equation becomes. To make this problem clear, we construct a tensor network to replace a sixth-order tensor $\mathcal{T}$ by four third-order tensors $\mathcal{A}, \mathcal{B}, \mathcal{C}$, and $\mathcal{D}$. Suppose that tensors $\mathcal{A}, \mathcal{B}, \mathcal{C}$, and $\mathcal{D}$ are contracted sequentially according to the given order, each time over a single shared index. Using the mathematical notation, a single entry of the tensor $\mathcal{T}$ is given by

$$\mathcal{T}_{ijklmn} = \sum_{o=1}^{d_o} \sum_{p=1}^{d_p} \sum_{q=1}^{d_q} \mathcal{A}_{ijo} \mathcal{B}_{okp} \mathcal{C}_{plq} \mathcal{D}_{qmn} \quad (2.1)$$

From this equation alone, it is not easy to identify at a glance which tensors are contracted with each other, nor over which indices. This problem illustrates the non-scalability of the mathematical notation; the more tensors and indices involved in the system, the more difficult it becomes to read and work with, as the notation forces a sequential representation. The tensor network consists of four tensors, but most real-world applications are rarely that small, so the need for another notation is evident. That is why we use diagrammatic notation in this work. In this representation, we overcome the sequential representation limitation by using a two-dimensional space. A tensor is depicted as a box with attached wires, where each wire corresponds to a mode of the tensor, so the total number of wires corresponds to the order of the tensor. The box is labelled with the tensor's name. Wires may be labelled with index names when needed to relate the diagram to its mathematical counterpart, but are typically omitted for simplicity and clarity. Figure 2.3 shows how a sixth-order tensor $\mathcal{T}$ can be represented using the diagrammatic notation. Since $\mathcal{T}$ is a sixth-order tensor, it is represented with six wires, each corresponding to one mode. The diagrammatic notation is appreciated for its simplicity and its fast interpretation at a glance. Mathematicians have put a lot of work into showing that the diagrammatic representations are equally formal as the mathematical notation [Coecke and Kissinger, 2017]. However, it is important to be aware that the diagrammatic notation offers more freedom and choices. For instance, the placement of the wires is arbitrary; they may be arranged on any side of the box. Consistency is therefore important because each wire represents a specific mode. One must always know which mode a wire represents. In practice and in our work, wires are typically interpreted counterclockwise, so index i corresponds to the first mode of $\mathcal{T}$, j the second, and so on, as shown in the figure.

With this newly introduced notation, we can revisit some tensor operations introduced previously and study them from another perspective. The **tensor contraction** (subsection 2.1.2),

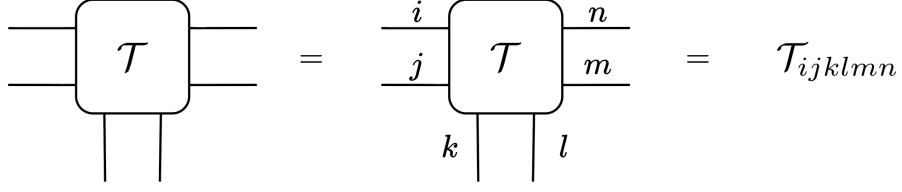


Figure 2.3: Diagrammatic representation of a sixth-order tensor $\mathcal{T}$. A sixth-order tensor $\mathcal{T}$ is depicted as a box with label $\mathcal{T}$ denoting its name and six wires connected to it. The total number of wires refers to the order of the tensor. The names given to each index in the mathematical notation can be omitted in the diagrammatic notation since indices are represented by a wire.

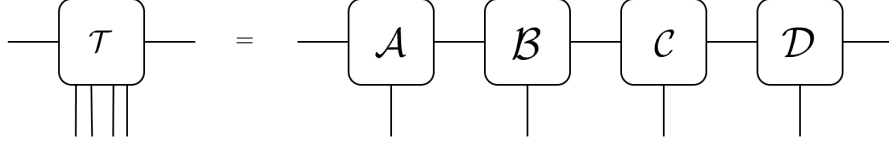


Figure 2.4: Illustration of a diagrammatic representation of equation (2.1). The network represents a sixth-order tensor $\mathcal{T}$ decomposed into four third-order tensors $\mathcal{A}, \mathcal{B}, \mathcal{C}$ and $\mathcal{D}$. Each box denotes a tensor, and each wire denotes an index of that tensor. Connected wires indicate tensor contractions over the corresponding indices, while open wires represent indices of the resulting tensor.

which involves summing over one or more shared indices, is depicted by connecting the corresponding indices (wires) of the tensors. Figure 2.4 diagrammatically represents the tensor network expressed in equation (2.1). Compared to its equivalent equation, the figure provides significantly more information at a glance. It makes the connectivity of the tensors immediately visible, regardless of contraction order. The diagrammatic notation is also scalable: additional tensors can easily be incorporated without sacrificing clarity. It also offers freedom in how to arrange the tensors, and is not limited to a sequential equation. For all the aforementioned reasons, that is why we chose to adopt the diagrammatic notation in this thesis. It facilitates reasoning about tensors and offers flexibility in layout that a sequential equation cannot provide. The diagrammatic notation accepts that tensors have sequential compositions (tensor contractions) that naturally live in two dimensions. The mathematical notation has to flatten this structure, making it less scalable and readable. Since the **tensor product** (subsection 2.1.2) can be reformulated as a tensor contraction with no shared index, its diagrammatic equivalent counterpart is one where two tensors are not connected; they represent a parallel composition.

The **tensor trace** (subsection 2.1.2) operation can also be revisited. The tensor trace operation consists of contracting pairs of indices of a single tensor. This can be visually depicted by connecting the indices of the tensors we wish to pair up for the tensor trace. Figure 2.5 illustrates a partial and a complete tensor trace of a fourth-order tensor $\mathcal{A}$. The partial tensor trace from figure 2.5a pairs up a subset of the indices of $\mathcal{A}$. This results in a new second-order tensor $\mathcal{A}'$. This is only mathematically sound under the assumption that the contracted indices have the same dimension. A complete tensor trace is illustrated in figure 2.5b. In this tensor trace, all indices are paired up. Since all indices are contracted, this results in a tensor with order zero, which is a scalar s .

The last operations we revisit with the diagrammatic notation are the **tensor permutation** (subsection 2.1.2) and the **tensor reshape** (subsection 2.1.2) operation. The tensor permutation

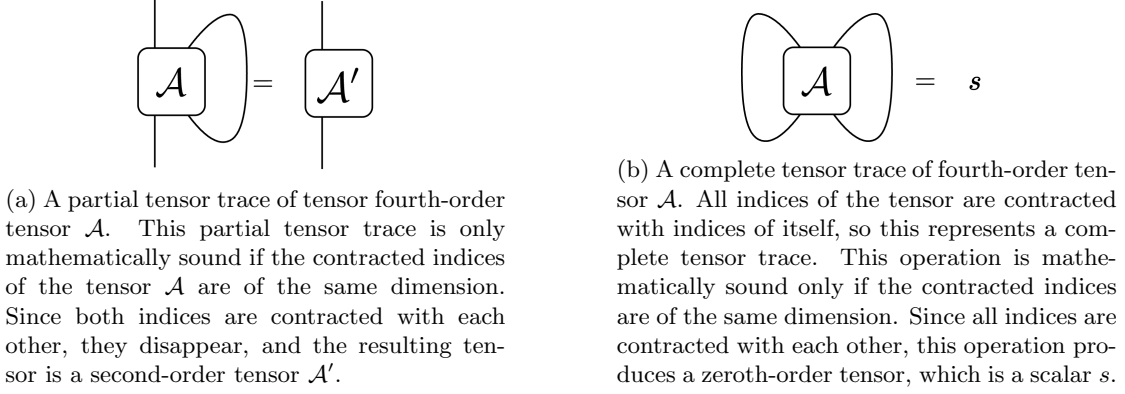
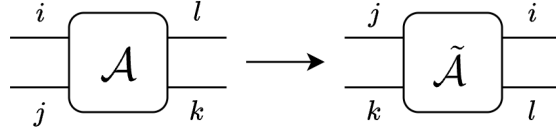
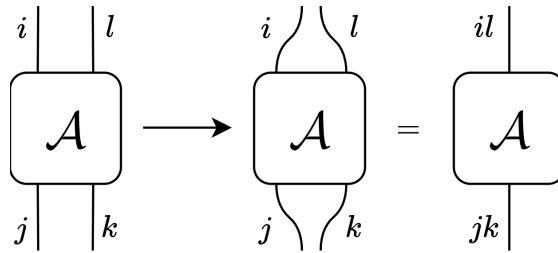


Figure 2.5: Partial and complete tensor traces illustration using the diagrammatic notation.

Figure 2.6: Permutation of the indices of a fourth-order tensor $\mathcal{A}$ illustrated with the diagrammatic notation. Permuting indices with the diagrammatic notation involves swapping wires with each other and producing a new tensor $\tilde{\mathcal{A}}$ of the same order and same data, but stored differently because of the index permutation.

reorders the indices of a tensor according to a permutation of the indices. From a diagrammatic notation perspective, this consists of reordering wires according to the new permutation. This reordering of indices is not observable in the diagrammatic notation, unless we use index names for wires. Figure 2.6 illustrates a permutation using the diagrammatic notation. The indices i, j, k, l of a fourth-order tensor $\mathcal{A}$ are reordered as j, k, l, i and result in a new tensor $\tilde{\mathcal{A}}$.

The tensor reshaping operation combines or splits indices of tensors. We consider an example where indices are combined into one. From a diagrammatic notation perspective, this means that the corresponding wires are merged into a single one as illustrated in figure 2.7. It shows how to depict the tensor reshaping operation in which indices i and l and indices j and k are each combined into a single index. This results in a reshaped second-order tensor.

Figure 2.7: Reshaping of fourth-order tensor $\mathcal{A}$ into a second-order tensor. The reshape operation combines index i with l into a new index il and j with k into index jk . The result is a second-order tensor. The dimension of the resulting indices is the product of the combined indices.

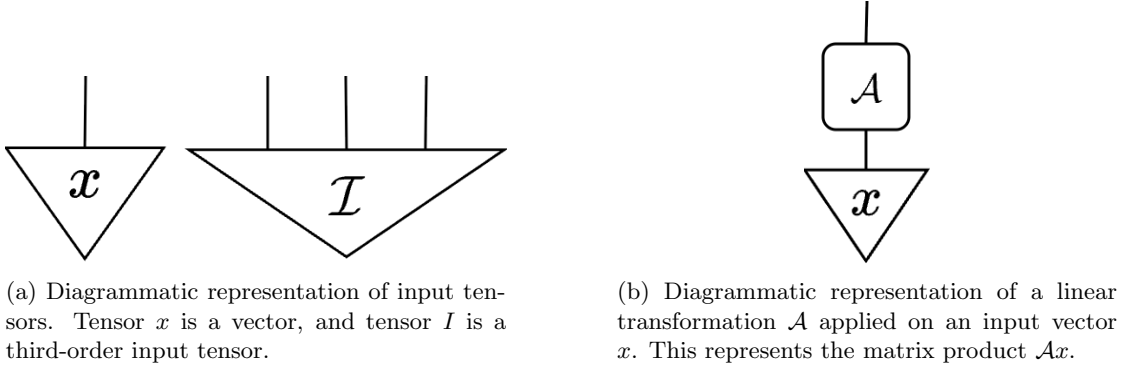


Figure 2.8: Diagrammatic representation and use of input tensors.

2.2.3 Special tensor types

Before diving into how we can decompose a high-order tensor into a tensor network, we first need to introduce the special tensor types that will play an important role in the decomposition procedure. These tensors have useful properties or roles.

When a tensor network is used to represent the parameters of an operation applied to an external tensor, it is useful to visually distinguish between input tensors representing some state (e.g., a vector) and tensors that belong to the network itself. For this purpose, input tensors are typically depicted as triangles on their head. This is illustrated in figure 2.8a, which shows the tensor product of an input vector x and a third-order input tensor I , both represented as inverted triangles. Figure 2.8b demonstrates why differentiating between tensors used as state or as transformation becomes useful. In this example, a linear transformation represented by matrix $\mathcal{A}$ is applied to an input vector x . By depicting x as an inverted triangle, it is immediately clear that x represents input data, while $\mathcal{A}$ corresponds to the parameters of the operation.

So far, tensors have been depicted as boxes with wires, but the notion of a wire has not been formally defined yet. Intuitively, a wire should keep data untouched. From a tensor perspective, this means that a wire corresponds to a second-order identity tensor that preserves values along an index under contraction. This role is fulfilled by the Kronecker delta δ_{ij} , which is defined as

$$\delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

It corresponds to a matrix with ones on the diagonal and zeros elsewhere, the identity matrix. Figure 2.9 shows how the identity matrix is denoted using the diagrammatic notation. It is depicted as a filled black circle with two wires. This notation is also used to depict a diagonal matrix, except that next to the circle is the name of the diagonal matrix to refer to its values, as illustrated in figure 2.10. The Kronecker delta can be generalised to higher-order tensors, leading to the concept of spiders [Coecke and Kissinger, 2017]. While a single line connects two indices, a spider allows multiple indices to be connected simultaneously. In this work, the only spider we use is the third-order spider, which corresponds to a third-order tensor defined as

$$\delta_{ijk} = \begin{cases} 1 & \text{if } i = j = k \\ 0 & \text{otherwise} \end{cases}$$

Diagrammatically, this spider is represented as a filled black circle with three wires attached to it, as illustrated in figure 2.11a. A spider can be used to compute the element-wise multiplication

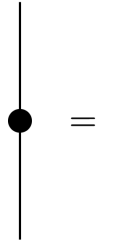


Figure 2.9: Diagrammatic equivalence between the identity matrix and a wire. The identity matrix is represented as a filled black circle with two wires.

$$\mathcal{S} = \begin{bmatrix} s_1 & 0 & \cdots & 0 \\ 0 & s_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & s_n \end{bmatrix} = \text{wire with a filled black circle} \mathcal{S}$$

Figure 2.10: Diagrammatic equivalence of a diagonal matrix. It is depicted the same way as the identity, except that next to the filled black circle is the name of the diagonal matrix.

of two vectors x and y as depicted in figure 2.11b since it is another representation of δ_{ijk} . This means that the tensor to which the spider depicted in figure 2.11b evaluates equals

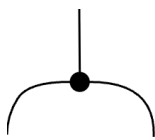
$$\mathcal{O}_i = \sum_{j=1}^{d_i} \sum_{k=1}^{d_j} x_j y_k \delta_{ijk}$$

By definition of δ_{ijk} , the only terms that remain are the ones where $i = j = k$. Therefore,

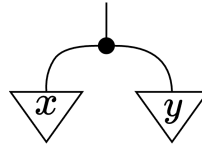
$$\mathcal{O}_i = x_i y_i \quad \forall i$$

which corresponds to the element-wise multiplication of x and y .

We conclude this subsection by introducing two special classes of tensors that will play an important role in the context of tensor decompositions: the unitary and isometric tensors. We first explain what unitary and isometric matrices are in linear algebra, before extending that



(a) Diagrammatic representation of δ_{ijk} , which is a third-order spider.



(b) Diagrammatic representation of element-wise multiplication of two vectors x and y .

Figure 2.11: Spider diagrammatic representation and use case for vector elementwise multiplication.

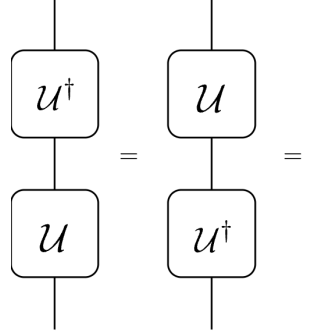


Figure 2.12: Diagrammatic representation of the unitary matrix definition. This representation exactly corresponds to $UU^\dagger = U^\dagger U = I$.

concept towards tensors. A matrix $U \in \mathbb{C}^{m \times m}$ is called unitary if

$$UU^\dagger = U^\dagger U = I$$

with $U^\dagger$ the conjugate transpose of U , and I the identity matrix [Evenbly, 2022]. Using the diagrammatic notation, we can visualise this definition as illustrated in figure 2.12. The concept of unitary matrices can be generalised to tensors, but for this, we need the tensor reshaping operation introduced previously. Using the tensor reshaping operation, we can turn any tensor into a matrix by grouping the indices of the tensor into two disjoint sets. In other words, we need to construct a bipartition of the indices of the tensor. A tensor is called unitary if there exists a bipartition of its indices under which the reshaped tensor is a unitary matrix. Using the diagrammatic notation, this definition can be depicted as in figure 2.13. We start with a fourth-order tensor $\mathcal{U}$, which we want to verify whether it is unitary or not. For this, we must find a bipartition of its indices such that the reshaped matrix is unitary. Let us assume that the bipartition consists of grouping the two top indices and the two bottom indices. We then reshape the tensor according to this bipartition into matrix $\mathcal{A}$. To verify whether this matrix is unitary, and hence $\mathcal{U}$ too, we must verify if $\mathcal{A}\mathcal{A}^\dagger = \mathcal{A}^\dagger\mathcal{A} = \mathcal{I}$. If that is true, then matrix $\mathcal{A}$ is unitary, and hence tensor $\mathcal{U}$ too. If not, we cannot conclude anything yet because it could be that the chosen bipartition of the indices is not one yielding a unitary matrix $\mathcal{A}$. We can only conclude that tensor $\mathcal{U}$ is not unitary if there is no bipartition of its indices that results in a unitary matrix when reshaped Evenbly [2022].

In the same spirit, we can define isometric tensors. A matrix $J \in \mathbb{C}^{m \times m}$ is called isometric if

$$J^\dagger J = I \quad \text{and} \quad J J^\dagger = P$$

with an orthonormal projector P and identity matrix I [Evenbly, 2022]. A projector P is a matrix that squares itself, meaning that $P^2 = P$. Following the same spirit as unitary tensors, we can generalise the isometric matrix definition to tensors. A tensor is isometric if there exists a bipartition of its indices such that the reshaped matrix is isometric. This definition is illustrated in figure 2.14 using the diagrammatic notation.

2.2.4 Multi-stage decompositions

So far, we have introduced tensors, examined special types of tensors, discussed their limitations, and presented tensor networks as a solution for the curse of dimensionality. The question of how

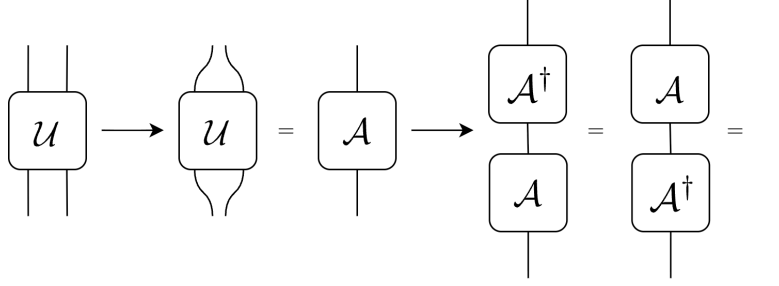


Figure 2.13: Diagrammatic representation of the unitary tensor definition. A fourth-order tensor $\mathcal{U}$ is unitary if there exists a bipartition of its indices, chosen here to be the top and bottom wires, such that the reshaped matrix $\mathcal{A}$ is unitary.

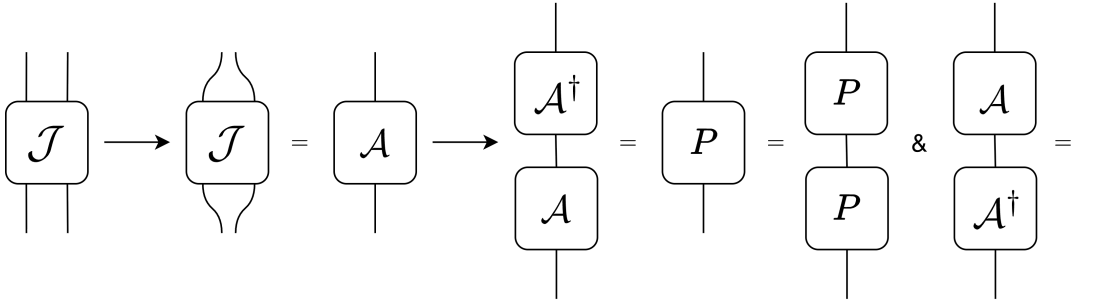


Figure 2.14: Diagrammatic representation of the isometric tensor definition. A fourth-order tensor $\mathcal{J}$ is isometric if there exists a bipartition of its indices, chosen here to be the top and bottom wires, such that the reshaped matrix $\mathcal{A}$ is isometric.

to transform a tensor into such a tensor network has not been addressed yet. This subsection focuses on multi-stage decompositions, which is the procedure that decomposes a tensor into a structured network of lower-order tensors. In this work, we do not apply multi-stage decomposition post-hoc to a known tensor, but we will learn tensors in their decomposed form directly. As we illustrate in section 3.2, we convert a convolutional neural network to an equivalent tensor network by mapping each component separately. This means that the final result will be a tensor network we do not necessarily need to decompose anymore. Nevertheless, analysing the techniques on how to do so yields valuable insights we reuse in that conversion.

Approximation evaluation criterion. Before describing the decomposition procedure, we introduce the criterion used to evaluate the quality of an approximation. Since the decomposition procedure produces a tensor network that approximates the original tensor, it is important to quantify this error. This criterion measures how accurately a tensor network represents a given tensor. Let $\mathcal{A}$ be a tensor of arbitrary order that we aim to decompose using a tensor network. Let $\mathcal{B}$ denote the tensor obtained by fully contracting the tensor network, meaning that all internal indices in the tensor network are contracted until a single tensor remains. The relative approximation error ϵ is defined as the Frobenius norm of the difference between $\mathcal{A}$ and $\mathcal{B}$, normalised by the Frobenius norm of $\mathcal{A}$ [Evenbly, 2022]

$$\epsilon = \frac{\|\mathcal{A} - \mathcal{B}\|}{\|\mathcal{A}\|}$$

The Frobenius norm of a tensor $\mathcal{A}$, denoted by $\|\mathcal{A}\|$, is defined as the square root of the sum of the squares of all its elements. For a tensor of order p , its Frobenius norm is defined as

$$\|\mathcal{A}\| = \sqrt{\sum_{i_1=1}^{d_1} \sum_{i_2=1}^{d_2} \cdots \sum_{i_p=1}^{d_p} \mathcal{A}_{i_1 i_2 \dots i_p}^2}$$

sum over all tensor indices

Problem formulation. With this criterion in place, the multi-stage decomposition problem can be formalised. Given a high-order tensor $\mathcal{A}$, the objective is to construct a tensor network $\mathcal{T}$ consisting of lower-order tensors according to a pre-described structure, such that the relative error

$$\epsilon = \frac{\|\mathcal{A} - \mathcal{T}\|}{\|\mathcal{A}\|}$$

is minimised [Evenbly, 2022]. The Frobenius norm of a tensor $\mathcal{A}$ can also be expressed using the tensor trace operation as

$$\|\mathcal{A}\| = \sqrt{\text{Ttr}(\mathcal{A}^\dagger \mathcal{A})}$$

In the expression $\text{Ttr}(\mathcal{A}^\dagger \mathcal{A})$, the tensor $\mathcal{A}$ is first paired with its conjugate $\mathcal{A}^\dagger$ by contracting each corresponding index (index on the same positions). Since all indices are contracted, this results in a scalar. The equivalence between the tensor trace and the Frobenius norm follows from expanding the contraction and achieving the previously introduced Frobenius definition, obtained by the chain of equivalence below

$$\begin{aligned} \text{Ttr}(\mathcal{A}^\dagger \mathcal{A}) &= \sum_{i_1=1}^{d_1} \sum_{i_2=1}^{d_2} \cdots \sum_{i_p=1}^{d_p} (\mathcal{A}^\dagger \mathcal{A})_{i_1 i_2 \dots i_p i_1 i_2 \dots i_p} \\ &= \sum_{i_1=1}^{d_1} \sum_{i_2=1}^{d_2} \cdots \sum_{i_p=1}^{d_p} \mathcal{A}_{i_1 i_2 \dots i_p}^\dagger \mathcal{A}_{i_1 i_2 \dots i_p} \\ &= \sum_{i_1=1}^{d_1} \sum_{i_2=1}^{d_2} \cdots \sum_{i_p=1}^{d_p} \mathcal{A}_{i_1 i_2 \dots i_p}^2 \\ &= \|\mathcal{A}\|^2 \end{aligned}$$

The second equality follows from expanding the tensor trace contraction. The third equality follows from the assumption that $\mathcal{A} \in \mathbb{R}^{d_1 \times \cdots \times d_p}$, so the conjugate $\mathcal{A}^\dagger$ reduces to the entries of $\mathcal{A}$. Since $\text{Ttr}(\mathcal{A}^\dagger \mathcal{A}) = \|\mathcal{A}\|^2$ follows immediately that $\sqrt{\text{Ttr}(\mathcal{A}^\dagger \mathcal{A})} = \|\mathcal{A}\|$. This result further illustrates the usage of tensor operations and is used in a later proof from appendix A.1.

Multi-stage decomposition outline. The multi-stage decomposition procedure relies on several additional concepts. Before developing them in detail, we briefly outline the overall idea of the procedure. The multi-stage decomposition is iterative: it starts with the original high-order tensor and progressively transforms it into a tensor network with the desired topology. At each step, a specific tensor within the current tensor network is selected. A condition (center of orthogonality) is verified for that tensor. If the tensor satisfies that condition, that tensor is locally decomposed using the singular value decomposition (SVD) for tensors. The resulting

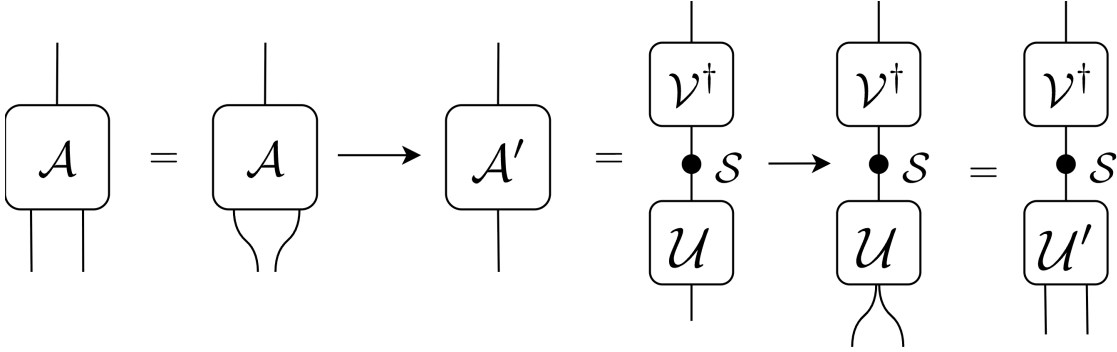


Figure 2.15: Illustration of a singular value decomposition for tensors. The third-order tensor $\mathcal{A}$ is first reshaped into a matrix $\mathcal{A}'$ by reshaping the tensor $\mathcal{A}$. The matrix is then decomposed using the singular value decomposition, yielding the matrices $\mathcal{U}$, $\mathcal{S}$, and $\mathcal{V}^\dagger$. Contracting these matrices reconstructs the matrix $\mathcal{A}'$. If a final third-order tensor representation is desired, the matrix $\mathcal{U}$ can be reshaped into a third-order tensor. This new contraction yields the third-order tensor $\mathcal{A}$.

decomposition is then rearranged through contractions, yielding an intermediate tensor network. This process is repeated until the desired network topology is obtained. The remainder of this subsection develops all relevant concepts in detail, and then concludes with the actual multi-stage decomposition procedure.

Singular value decomposition. The singular value decomposition (SVD)² plays a central role in multi-stage decompositions, as it provides a systematic way to decompose a single tensor into tensors of lower order and smaller dimensions. The SVD is originally defined for matrices, but can be extended for higher-order tensors via the tensor reshaping operation. Similarly to our definitions of unitary and isometric tensors, we can extend the definition of SVD for matrices to tensors. The SVD of a matrix $A \in \mathbb{C}^{m \times n}$ decomposes the matrix in three matrices U, S, V such that

$$A = USV^\dagger$$

with isometric matrix $U \in \mathbb{C}^{m \times n}$, diagonal matrix $S \in \mathbb{C}^{n \times n}$ and unitary matrix $V \in \mathbb{C}^{n \times n}$, assuming that $n \leq m$ and that A has rank n [Lay et al., 2016]. The diagonal matrix S consists of positive, real elements ordered in descending magnitude. The real elements on the diagonal are called the singular values of the decomposition. The SVD can be applied to tensors by first reshaping the tensor as a matrix according to a bipartition of its indices. After performing the SVD on the matrix according to our definition, we obtain three matrices, which can be shaped back into tensors of the appropriate order. With this approach, the SVD provides a systematic way to decompose a tensor into three interconnected tensors. Figure 2.15 illustrates how the SVD can be applied to a third-order tensor $\mathcal{A}$. The tensor must first be reshaped into a matrix according to a bipartition of its indices, which is chosen arbitrarily here. This results in a second-order tensor $\mathcal{A}'$ to which the SVD for matrices can be applied. This step produces the three matrices $\mathcal{U}, \mathcal{S}, \mathcal{V}^\dagger$ defined previously. Since our original goal was to apply a singular value decomposition to the third-order tensor $\mathcal{A}$, we can reshape the tensor $\mathcal{U}$ back to a third-order tensor $\mathcal{U}'$.

²Other decomposition techniques such as the eigendecomposition can also be used.

Rank-restricted approximation. Building on this concept, the singular value decomposition provides a systematic way for constructing rank-restricted tensor approximations. A rank-restricted approximation of a tensor approximates that tensor by decomposing it while explicitly limiting the number of elements in that decomposition. This is achieved by restricting the decomposition rank of the tensor. The decomposition rank r of a tensor $\mathcal{A}$, with respect to a chosen bipartition of its indices, is defined as the minimal internal dimension such that $\mathcal{A}$ can be expressed as the contraction of two tensors $\mathcal{B}$ and $\mathcal{C}$ along a single shared index [Evenbly, 2022]. This notion is particularly important because it characterises the smallest internal dimension required to represent $\mathcal{A}$ exactly under that bipartition. When the SVD is applied to a tensor, this rank r coincides exactly with the number of non-zero singular values Evenbly [2022]. Consequently, SVDs enable controlled approximations. Given a tensor $\mathcal{A}$ with decomposition rank r , a lower-rank approximation $\mathcal{B}$ of rank $\chi \leq r$ can be constructed such that it best approximates $\mathcal{A}$ with respect to the Frobenius norm. This is accomplished by truncating the $r - \chi$ smallest singular values from the diagonal matrix S and correspondingly reducing the dimensions of U and $V^\dagger$ from the SVD of $\mathcal{A}$. The resulting truncated decomposition yields a tensor of rank χ whose approximation is optimal in the Frobenius norm. Figure 2.16 illustrates this procedure for a third-order tensor $\mathcal{A}$. The third-order tensor $\mathcal{A}$ is first reshaped into a matrix $\mathcal{A}'$ according to a chosen bipartition of its indices. Applying the SVD to $\mathcal{A}'$ produces matrices $\mathcal{U}$, $\mathcal{S}$ and $\mathcal{V}^\dagger$, where $\mathcal{S}$ contains the singular values arranged in decreasing order along its diagonal. In a rank-restricted approximation, only the χ largest singular values are retained, forming a reduced matrix $\mathcal{S}'$. The dimensions of $\mathcal{U}$ and $\mathcal{V}^\dagger$ are adjusted accordingly. The resulting truncated decomposition defines a tensor $\mathcal{B}$, which is the optimal rank- χ approximation of the original tensor $\mathcal{A}$. Restricting the decomposition rank r to χ with $\chi \leq r$ is particularly useful because it allows us to reduce the number of elements needed to approximate the original tensor in the decomposition. It gives some flexibility to determine how many elements we wish to keep to approximate $\mathcal{A}$ by ensuring that the approximation error is kept minimal.

This optimal rank-restricted tensor approximation is a key component of the multi-stage decomposition procedure. At each iteration, it enables a local decomposition of a selected tensor while ensuring that the approximation error introduced at that step is minimal in the Frobenius norm [Evenbly, 2022]. However, this procedure cannot be applied to an arbitrary tensor. The optimal rank-restricted tensor approximation only guarantees that the local approximation error is minimal (the single tensor). Ideally, we would also like it to guarantee that the global approximation error is minimised, too. To guarantee this, the tensor being decomposed must be a center of orthogonality. We consider this in the context of loop-free networks, called tree tensor networks. This concept is also defined for other types of networks, but this goes beyond the scope of this thesis [Evenbly, 2018]. In a tree tensor network, any tensor in the network can conceptually be chosen as a root of the tree. This defines a hierarchical structure with distinct branches, where connections between branches are not allowed. Figure 2.17 illustrates such a tree tensor network with root $\mathcal{A}$ and three branches.

Center of orthogonality. With this tree tensor network definition, we can define the concept of a center of orthogonality. The root of a tree tensor network is a center of orthogonality if, for every branch of the network attached to the root, the branch forms an isometry between its open indices and the index connected to the root [Evenbly, 2022]. Concretely, this means that every branch contracted with its adjoint results in the identity. For instance, the tensor $\mathcal{A}$ of the tree tensor network depicted in figure 2.17 is a center of orthogonality if its three branches are isometries. This condition is visually depicted in figure 2.18 for the non-identity branches. This figure depicts each branch contracted with its adjoint version. If that construction contracts to the identity, the tensor $\mathcal{A}$ is a center of orthogonality. A root of a tree tensor network can always

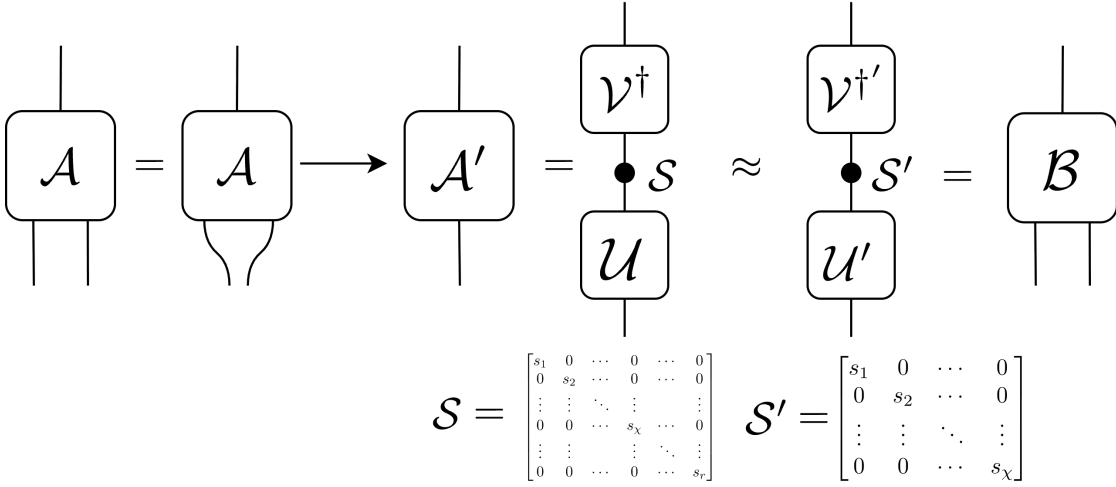


Figure 2.16: Rank-restricted tensor approximation example. A third-order tensor is approximated by a third-order tensor $\mathcal{B}$ constructed using a rank-restricted approximation. Tensor $\mathcal{A}$ is first reshaped into a matrix $\mathcal{A}'$. A singular value decomposition is applied on matrix $\mathcal{A}'$ producing matrices $\mathcal{U}$, $\mathcal{S}$, and $\mathcal{V}^\dagger$. The singular values of the diagonal matrix $\mathcal{S}$ are truncated to only retain the χ largest ones, producing an approximation. Matrices $\mathcal{U}$ and $\mathcal{V}^\dagger$ are changed accordingly by modifying their internal dimensions from r to χ . This contraction results in tensor $\mathcal{B}$.

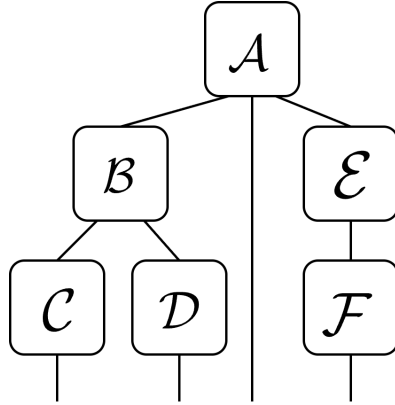


Figure 2.17: Tree tensor network illustration. Since the tensor network contains no loops, it is a tree tensor network. Tensor $\mathcal{A}$ acts as the root within that tree with three branches.

be transformed into a center of orthogonality. This process does not change anything about the tensor to which the entire tensor network contracts, but it changes the values of some component tensors in the tensor network to guarantee that condition Evenbly [2022]. We briefly explain the gist of this process. The process relies on the gauge freedom of tensor networks Evenbly [2018]. The gauge freedom concept allows us to introduce arbitrary tensors locally in our tensor network, without changing its global behaviour, but changing properties about the tensor network itself (for example, creating a center of orthogonality). In the previous section, we discussed that a wire in a tensor network represents the identity I . This means that the identity can be replaced by a series of tensors whose contraction yields the identity. If we analyse this concept assuming

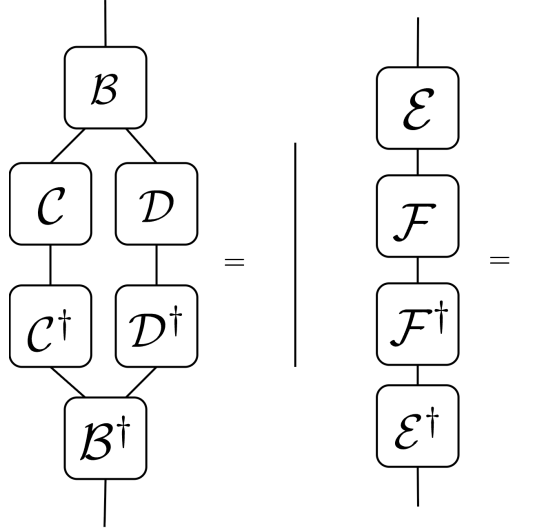


Figure 2.18: Isometric branches diagrammatic definition for two arbitrary branches. If each branch results in the identity when contracted with its adjoint counterpart, the branch is said to be isometric.

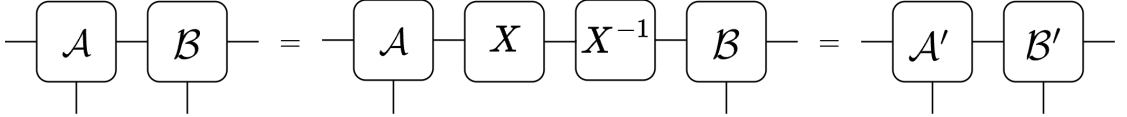


Figure 2.19: Gauge freedom illustration. The internal index connecting tensors $\mathcal{A}$ and $\mathcal{B}$ is replaced by XX^{-1} since a wire represents the identity I . Matrix X is then contracted with tensor $\mathcal{A}$ yielding $\mathcal{A}'$, and matrix X^{-1} is contracted with tensor $\mathcal{B}$ yielding $\mathcal{B}'$. Since the internal wire was replaced by an expression equal to the identity, the overall contraction of tensors $\mathcal{A}$ and $\mathcal{B}$ remains identical to that of $\mathcal{A}'$ and $\mathcal{B}'$.

we only introduce matrices, it means that we can replace any wire in a tensor network by an expression satisfying $XX^{-1} = I$ Evenbly [2018]. By carefully choosing matrix X , one can turn a tensor into a center of orthogonality without changing the tensor to which the entire network evaluates, since $XX^{-1} = I$. Figure 2.19 illustrates this concept. The internal wire connecting tensors $\mathcal{A}$ and $\mathcal{B}$ is replaced by XX^{-1} . The matrices X and X^{-1} can be absorbed into the adjacent tensors by, for instance, contracting $\mathcal{A}$ with X and $\mathcal{B}$ with X^{-1} . This results in two new tensors $\mathcal{A}'$ and $\mathcal{B}'$. Although the tensors $\mathcal{A}$ and $\mathcal{B}$ change, the overall contraction of the network remains the same. Contracting $\mathcal{A}$ and $\mathcal{B}$ yields the same result as contracting $\mathcal{A}'$ with $\mathcal{B}'$. This freedom to insert tensors on internal indices of tensor networks can be used to transform a tensor into a center of orthogonality. Appropriate tensors can be chosen such that all branches connected to a selected tensor become isometric. Using these properties, there exist multiple techniques, discussed in [Evenbly, 2022], to convert a root into a center of orthogonality.

Before finally presenting the multi-stage decomposition algorithm, we briefly motivate the importance of the center of orthogonality and its role in tensor decompositions. Making a tensor the center of orthogonality ensures that the global approximation error introduced by replacing that tensor with its approximation is minimised. This property is important because it allows

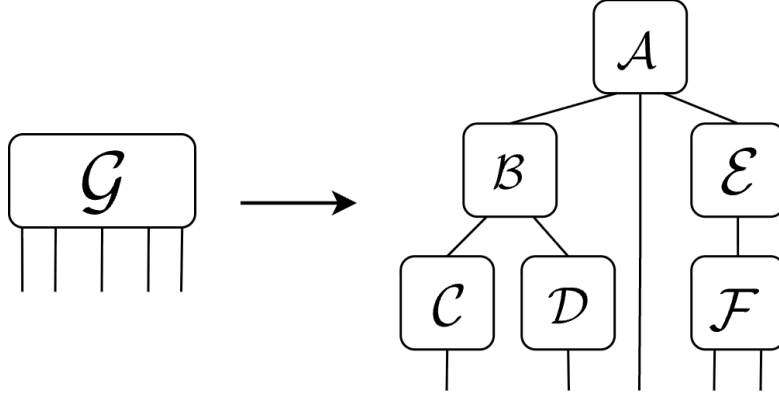


Figure 2.20: Multi-stage decomposition goal. Using the multi-stage decomposition procedure, we show how a fifth-order tensor $\mathcal{G}$ is decomposed into the network structure on the right of the figure.

a tensor to be decomposed locally while ensuring that the resulting global error is the same as the local error of the approximation. This property is formally described as follows: let $\mathcal{A}$ be a center of orthogonality of a tree tensor network $\mathcal{T}$, and let $\mathcal{A}'$ be the local approximation of $\mathcal{A}$ resulting in tree tensor network $\mathcal{T}'$. Then the difference between the original tensor and its approximation, measured by the Frobenius norm $\|\mathcal{A} - \mathcal{A}'\|$, is equal to the global difference of the two tree tensor networks. This means that $\|\mathcal{A} - \mathcal{A}'\| = \|\mathcal{T} - \mathcal{T}'\|$ [Evenbly, 2022]. We constructed a proof for this property given in appendix A.1.

Decomposition procedure. With all relevant concepts introduced, we can now present the multi-stage decomposition procedure. We have already outlined the process. We now explain the procedure through an illustrative example. Consider the decomposition of a fifth-order tensor $\mathcal{G}$ following the tensor network structure illustrated in figure 2.20. We show and explain this decomposition step by step. In this illustrative example, we decided to restrict all internal dimensions to χ . This means that in all SVDs we perform, we keep only the χ largest singular values.

The first step of the multi-stage decomposition procedure is illustrated in figure 2.21. It starts from the fifth-order tensor $\mathcal{G} = \mathcal{G}_0$. Since the current tree tensor network consists of a single tensor, $\mathcal{G}_0$ is by definition a center of orthogonality. Next, a singular value decomposition for tensors is performed on $\mathcal{G}_0$ with respect to a chosen bipartition of its indices. The bipartition is selected such that one of the resulting tensors corresponds to a tensor in the final result. In this step, we decided, without loss of generality, that this would be the tensor $\mathcal{C}$. The SVD yields three tensors: $\mathcal{C}_0$, $\mathcal{S}_0$ and $\mathcal{V}_0^\dagger$. Although $\mathcal{C}_0$ will eventually become the final tensor $\mathcal{C}$, it does not yet equal it. This is because in future steps, the tensor to be decomposed at each step must be a center of orthogonality. To achieve this, the branches possibly need to be changed, and so will $\mathcal{C}_0$ probably too. Therefore, additional modifications of $\mathcal{C}_0$ will very likely be required in later steps to ensure that all connected branches satisfy the necessary isometric conditions. After performing the SVD, the χ largest singular values of $\mathcal{S}_0$ are kept, and the dimensions of $\mathcal{C}_0$ and $\mathcal{V}_0^\dagger$ are changed accordingly. The diagonal tensor $\mathcal{S}_0$ on the internal connection is then absorbed into one of the adjacent tensors. This can be done either by incorporating $\mathcal{S}_0$ into $\mathcal{C}_0$ or into $\mathcal{V}_0^\dagger$. In this example, we choose to absorb $\mathcal{S}_0$ into $\mathcal{V}_0^\dagger$ into the new tensor $\mathcal{G}_1$.

The second stage of the decomposition is shown in figure 2.22, whose goal is to create a tensor

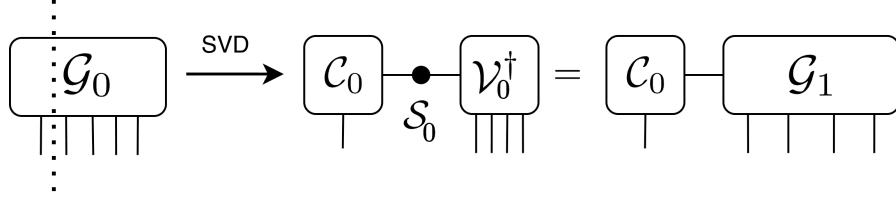


Figure 2.21: Multi-stage decomposition step 1. A bipartition of the indices of the tensor $\mathcal{G}_0$ is chosen to perform the SVD for tensors. This yields three tensors $\mathcal{C}_0$, $\mathcal{S}_0$, and $\mathcal{V}_0^\dagger$. These three tensors are slightly modified to ensure we lower the internal dimension to χ . Finally, tensor $\mathcal{S}_0$ is then absorbed in tensor $\mathcal{V}_0^\dagger$ by applying the tensor contraction resulting in tensor $\mathcal{G}_1$.

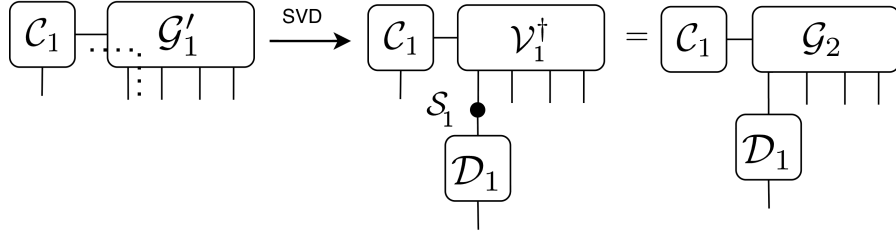


Figure 2.22: Multi-stage decomposition step 2.

that will later become $\mathcal{D}$ in the resulting tensor network. Since tensor $\mathcal{C}_0$ from the previous step will later become tensor $\mathcal{C}$, we do not decompose it, but we decompose $\mathcal{G}_1$ instead. Before applying this decomposition, $\mathcal{G}_1$ must first be a center of orthogonality. We do not discuss the process of turning it into a center of orthogonality here, but rather assume that this process yields tensors $\mathcal{C}_1$ and $\mathcal{G}'_1$ to ensure the center of orthogonality condition. Once $\mathcal{G}'_1$ is the center of orthogonality, we choose a bipartition of its indices. As in the first stage, the bipartition is selected deliberately so that one of the tensors produced by the SVD corresponds to a tensor of the final tree tensor network. In this case, the decomposition is arranged such that the SVD yields the tensor $\mathcal{D}_1$, which will ultimately become the final tensor $\mathcal{D}$. Performing the SVD of $\mathcal{G}'_1$ with respect to this bipartition gives tensors $\mathcal{D}_1$, $\mathcal{S}_1$ and $\mathcal{V}_1^\dagger$. The χ largest singular values of $\mathcal{S}_1$ are kept, and the dimensions of $\mathcal{D}_1$ and $\mathcal{V}_1^\dagger$ are changed accordingly. The diagonal tensor $\mathcal{S}_1$ is then absorbed into the neighbouring tensor $\mathcal{V}_1^\dagger$, resulting in the tensor $\mathcal{G}_2$.

This procedure is repeated in the remaining steps illustrated in figure 2.23, figure 2.24, and figure 2.25. In each step, a tensor is converted into a center of orthogonality. An SVD is then performed on that tensor with respect to a chosen bipartition of its indices, selected so that one of the resulting tensors corresponds to a tensor of the target tree tensor network. The singular values of the diagonal matrix $\mathcal{S}_i$ are then truncated to keep the χ largest ones. Finally, this diagonal matrix $\mathcal{S}_i$ is then absorbed in a neighbouring tensor before proceeding to the next step. Repeating this procedure constructs the desired tree tensor network illustrated in figure 2.20.

The decomposition procedure is not unique. At each step, several choices must be made, such as which tensor to target for the final network and where to absorb the diagonal matrix. Consequently, even though every step guarantees the local approximation to be minimised and controlled, the overall global error may not be globally optimal. There may exist an alternative sequence of choices that leads to a smaller global deviation. However, in practice, this deviation is usually small [Evenbly, 2022]

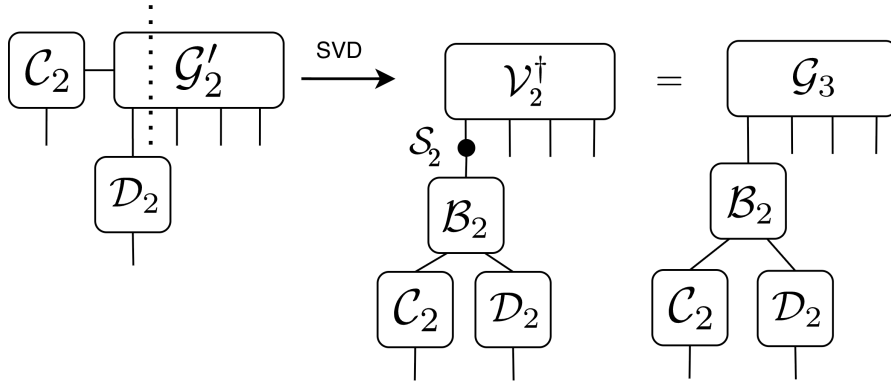


Figure 2.23: Multi-stage decomposition step 3.

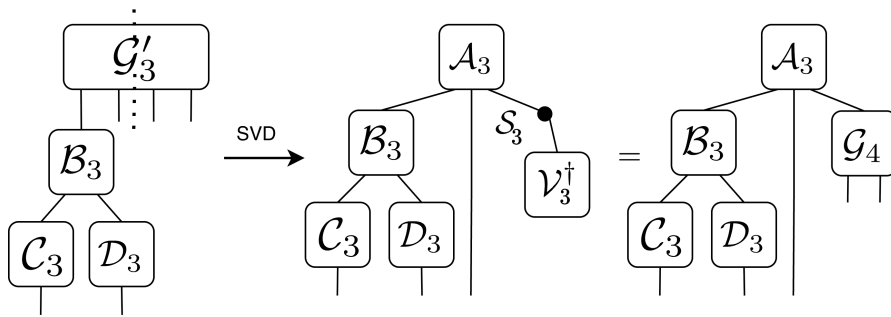


Figure 2.24: Multi-stage decomposition step 4.

2.2.5 Tensor networks in other fields

Despite their origin in quantum physics, tensor networks are broadly used as a compression strategy in other fields. Those fields range from quantum chemistry [Nakatani and Chan, 2013] to big data [Cichocki, 2014] to signal processing [Cichocki et al., 2015] and more. The central principle remains consistent across these fields; high-dimensional data is approximated using tensor networks. In the section about mechanistic interpretability (subsection 2.4.2), we explain how tensor networks can be used to interpret a special class of machine learning models.

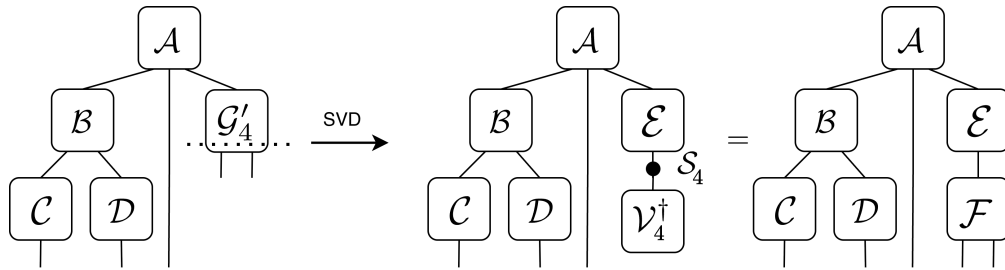


Figure 2.25: Multi-stage decomposition step 5.

2.2.6 Limitations

Despite their potential, tensor networks remain underused in practice [Hamreras et al., 2025]. A key limitation is the insufficient hardware and software support. Efficient tools for tensor network evaluation remain scarce, and tensor contraction operations are not yet fully optimised on GPUs [Hamreras et al., 2025]. Furthermore, decomposing a tensor into a tensor network introduces a large and complex hyperparameter space, including choices related to network topology and the number of tensors. A deeper theoretical understanding is required to clarify how these hyperparameters influence the accuracy of the approximation [Wang et al., 2025].

2.3 Convolutional Neural Networks

With tensor networks introduced, we now turn our attention to another important pillar of this work, which is Convolutional Neural Networks (CNNs). Before exploring CNNs in detail, we first review the context in which CNNs were developed, which is machine learning and specifically deep learning. After that, we explain CNNs in greater detail.

2.3.1 Machine learning

Readers familiar with the fundamentals of machine learning may skip this subsection. Consider the problem of building a program that recognises whether a picture contains a dog or a cat. In theory, it would be possible to define a set of rules that determine whether an image contains a dog or a cat. However, such an approach is extremely complex and impractical due to the high diversity in images. Instead, machine learning algorithms automatically discover these rules from data. Machine learning, a subfield of artificial intelligence, focuses on developing systems that learn from data rather than being explicitly programmed with rules [Goodfellow et al., 2016]. Learning from data is a broad and abstract concept. Tom Mitchell’s Machine Learning book [Mitchell, 1997] defines this concept of learning formally. It states that a computer program is said to learn from experience E with respect to some task T and performance measure P , if the performance measure P at task T improves with the experience E . To enable the learning of task T , a (machine) learning algorithm is used. This algorithm is guided by a performance measure P and learns from experience E to perform a particular task T . Finally, the learning algorithm returns a model that can be used to perform the given task T . The goal of this model is to perform well on task T according to the performance measure P .

Machine learning algorithms are classified into three paradigms [Sutton et al., 2018]: supervised learning, unsupervised learning, and reinforcement learning. In supervised and unsupervised learning, the experience consists of pre-defined data, such as tabular data, images, text, graphs, or videos. In supervised learning, the data is labelled. For example, an image of a dog is labelled as "dog". The goal is to learn to predict either the class of an input (classification) or a continuous value (regression). In unsupervised learning, no labels are provided. Both paradigms aim to discover patterns within data to generalise to unseen data. Supervised learning would, for example, discover patterns in data to recognise dogs, while unsupervised learning would learn patterns to cluster images into distinct groups without labels. Reinforcement learning is fundamentally different. In this paradigm, there is no pre-defined data. Instead, data is generated through interaction with an environment. The tasks in reinforcement learning are primarily concerned with learning a decision-making behaviour.

In this work, we focus on supervised learning, especially on classification tasks related to computer vision. Therefore, our work focuses on machine learning algorithms that are suited for computer vision tasks. One example of these models is a convolutional neural network, which

was developed to solve computer vision tasks in [Lecun et al., 1998] and has since been proven empirically to be successful in achieving strong performance in later work such as [Krizhevsky et al., 2012]. Before diving deeper into its technical workings, we first introduce deep learning, as this is the idea on which CNNs are built. This will help build the intuition behind CNNs.

2.3.2 Deep learning

Until now, we have been vague about the nature of the input data. Data can take many forms, such as tabular data, images, text, videos, or graphs. Conventional machine learning algorithms, however, do not process such data directly. Instead, engineering and domain expertise are needed to design a feature extractor that transforms raw data into a suitable representation, also called features, for the machine learning algorithm [LeCun et al., 2015]. For example, from a cat image, we might manually extract that the cat has green eyes, pointy ears, and a flat face. These manually constructed features or representations are provided to the learning algorithm to enable it to solve a task. However, while this approach can offer interpretability since we know what features are used by the machine learning algorithm to make its decision, it often leads to inferior performance compared to methods that extract features automatically from the image itself without human intervention. This has been shown empirically in [Krizhevsky et al., 2012]. This is not surprising, as we forced the machine learning algorithm to make use of features we think are necessary. Those algorithms may observe patterns we do not see as humans. Manually extracting features does not let the model extract what it thinks is necessary to perform the task. A more powerful alternative would be to let the model learn these features directly from the raw data [LeCun et al., 2015]. This is precisely the idea of deep learning, which eliminates the need for handcrafted features. Deep learning algorithms learn feature representations directly from the data during training, in a way that is tailored to a specific task. Figure 2.26 illustrates the difference between these two approaches on a concrete example. Both approaches aim to predict that the image contains a cat.

The shift from conventional machine learning to deep learning introduces a trade-off [Goodfellow et al., 2016]. Deep learning enables models to learn task-specific representations directly from data while often improving performance. However, since we do not have control over the features used, it makes the model difficult to interpret. This means that we have models with higher performance, but at the cost of interpretability. Depending on the context and the task at hand, this may or may not be a problem. In applications such as our cat and dog image recognition, this trade-off is acceptable. However, in domains such as healthcare, where such a model is used for cancer detection, understanding why a model made a decision is important. Developing interpretability methods that can explain model decisions is therefore an essential step before deep learning can be applied in such domains. Furthermore, using deep learning requires significantly more data than conventional machine learning algorithms to achieve good performance [Goodfellow et al., 2016]. In domains where data is scarce, this can be an important limitation.

CNNs fall under the deep learning method. Given an image as input, CNNs extract meaningful and rich features that are used to solve a given computer vision task. In the next subsection, we discuss how this feature extraction process works, using neural networks.

2.3.3 Neural networks

A neural network is an approximator of a general function f . In our cat and dog image classification task, this function f would, given an image x , output $f(x) = \text{cat}$ or $f(x) = \text{dog}$. It has been shown in [Hornik et al., 1989] that neural networks are universal function approximators. They

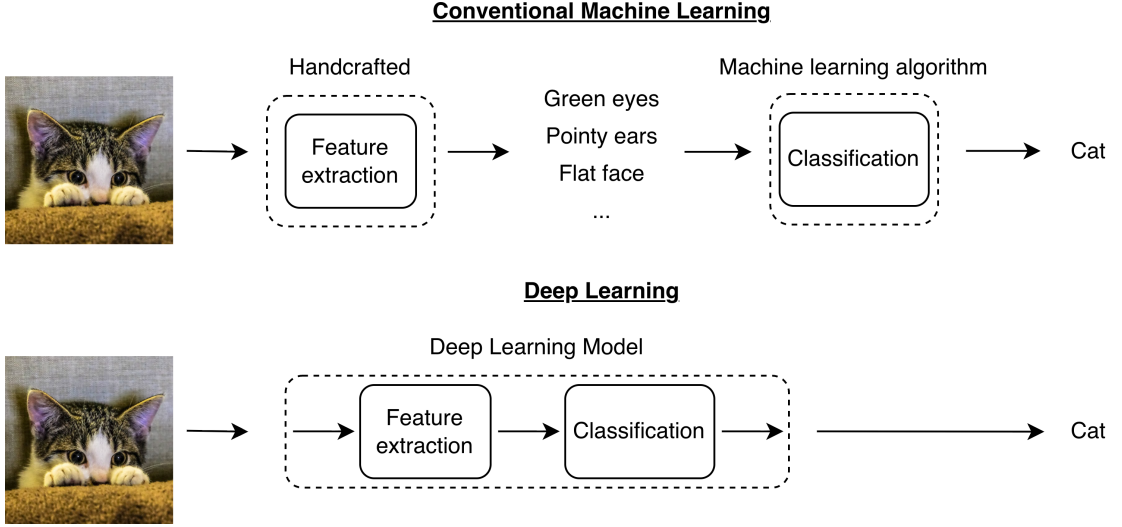


Figure 2.26: Comparison between conventional machine learning and deep learning. The conventional approach relies on handcrafted features, while the deep learning approach relies on feature representations directly learned from the data without human intervention.

consist of layers, where each layer represents a parametrised affine (or linear) transformation followed by a non-linear transformation to its input. One layer l computes

$$l(x) = \sigma(Ax + b) \quad (2.2)$$

with input vector x , bias term b , matrix A representing the parameters of the linear transformation and σ representing some non-linear transformation [Goodfellow et al., 2016]. This non-linear transformation, referred to as the activation function, is essential for the expressive power of the layer. Without it, a layer would represent a linear transformation and would not be able to represent non-linear relations [Abu-Mostafa et al., 2012]. A neural network consists of a chain of these layers. That is how it connects to deep learning: every time a layer is applied, a new representation is constructed from the previous one. By chaining these layers, the network progressively converts raw input to increasingly abstract representations. Each layer enriches the representation by building on what the previous layer produced. The final representation produced by the chain of layers is, if constructed properly, rich enough to support the learning task at hand. This representation can then be used to make a final decision, for instance, whether the original image contains a dog or a cat. This architecture, where information flows from input to output via functions described by equation (2.2), is called a feed-forward neural network [Goodfellow et al., 2016].

Connecting back to deep learning, we did not have to specify how and which features to extract from the raw input. This is all contained and parametrised by the affine transformations. The goal of the learning algorithm is to learn parameters for all affine transformations that will effectively produce a final rich representation that supports the original learning task. Those parameters are learned through an algorithm called backpropagation. Suppose our dataset, the experience E as we called it before, consists of labelled images of dogs and cats. One single input is fed forward through the neural network. This means that we give the image as input to the neural network, which produces an internal representation, and then apply the classification on that to predict whether the image contains a dog or a cat. We then compare the prediction of

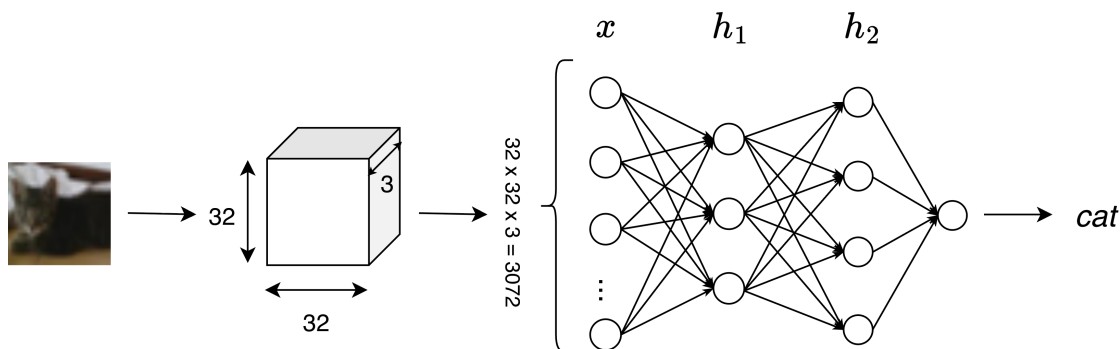


Figure 2.27: Feed-forward neural network with two hidden layers used for image classification. The RGB-image with dimensions $3 \times 32 \times 32$ is first reshaped into a vector x of size 3072. This vector x is fed as input to the feed-forward neural network. The first layer, consisting of an affine transformation followed by a non-linear transformation, produces vector h_1 with three values. This vector h_1 is then mapped onto another vector h_2 by a second layer. This final vector represents a final rich compressed representation of the raw input x and can be used for image classification to produce a single value indicating whether it represents a dog or a cat.

the neural network with the ground truth. Based on this comparison, we compute the error, also called loss, which is then backpropagated through the network. This means that for each parameter that has contributed to the final prediction, we compute a gradient to know how we have to change that parameter to minimise the classification error. By repeating this process multiple times for different images, we get parameters for each affine transformation that will produce a final representation that can be used for the classification task. More mathematical information about backpropagation and the setup can be found in [Goodfellow et al., 2016].

To make it more concrete how a neural network processes an image in practice, consider the following example. This example is purely illustrative and may not produce good results, but the intuition is the most important here. Figure 2.27 presents a simple feed-forward neural network consisting of three layers, one input layer, and two hidden layers. First, our third-order cat image is reshaped into a first-order image (a vector) x . A first affine transformation is applied to this vector, followed by a non-linear activation function. This yields a hidden representation h_1 . This representation consists of three values that capture low-level patterns in the image, such as edges. These values are then used in the following transformation, resulting in a new representation h_2 , which consists of four values. These features combine low-level patterns and can, for example, represent simple shapes such as circles by combining edges. This idea can be further extended to deeper representations, which capture increasingly abstract and task-specific features (e.g., eyes or ears). In this example, we have assumed that two hidden layers with small dimensions (3 and 4 respectively) were enough, but in practice, a feed-forward neural network consists of more layers and wider layers (bigger dimensions). There is no recipe on how many layers we need to use and what their dimensions must be; it depends on the task and the available data at hand.

With all necessary concepts introduced, we can finally dive deeper into CNNs in the next subsection.

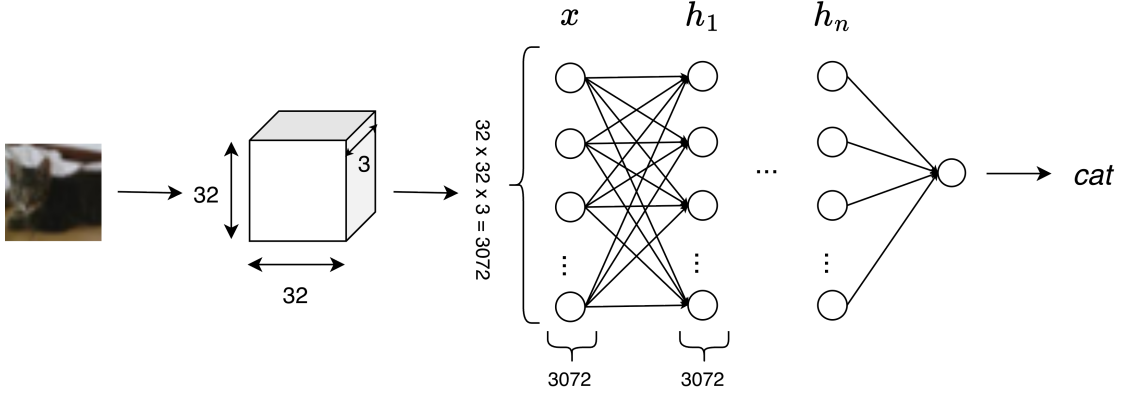


Figure 2.28: Feed-forward neural network with n hidden layers. The input image, which is a third-order tensor, is first flattened to a vector x . This input, along with the first hidden layer h_1 , has size 3072. This creates $3072 \cdot 3072 = 9\,437\,184$ parameters for this layer only. If we repeat this for n layers, the parameter count becomes large.

2.3.4 Model intuition

The existence of CNNs for computer vision tasks is motivated by analysing the shortcomings of feed-forward neural networks on computer vision tasks. The goal of a feed-forward neural network would be to generate a rich representation for a single image based on a chain of layers presented in equation (2.2). This goal remains the same in CNNs, but the way the representation is generated is different this time. To understand why generating representations for images is impractical with feed-forward neural networks, consider the following example illustrated in figure 2.28. This figure shows a feed-forward neural network with n hidden layers. The size of the input vector x consists of 3072 values, and so does the size of the first hidden vector h_1 . Given those sizes, this means that the affine transformation that maps x to h_1 is a matrix A consisting of $3072 \cdot 3072 = 9\,437\,184$ parameters for a single layer. If we repeat this for additional layers, the number of parameters rapidly grows. The large parameter count is not a problem of feasibility; given enough data, a network can still be trained [Goodfellow et al., 2016]. The problem is one of efficiency; the full connectivity requires far more parameters than necessary because it ignores the spatial structure within images. If we consider the very first transformation mapping x to h_1 , the inefficiency becomes clear. The input vector x is constructed by flattening the RGB input image into a first-order vector. By doing so, all spatial information is destroyed. The input vector x , and hence the network, has no knowledge that some pixels are neighbours. The $9\,437\,184$ parameters connect the input values to h_1 by assuming that the input values are not linked to each other, treating them as an unordered set of values. Furthermore, if some parameters detect a pattern somewhere within an image, the parameters are fixed to capture that pattern in that place of the image since they are tied to particular input pixels. If we want to capture the same pattern elsewhere, we are effectively wasting parameters within our network [Goodfellow et al., 2016].

Convolutional neural networks solve this problem in an elegant way. Instead of fully connected layers, it uses reusable kernels, also called filters [Goodfellow et al., 2016]. A kernel is a set of parameters that can be used to detect a particular pattern in a region of an image. Figure 2.29 illustrates such a kernel used to detect a vertical edge in a handwritten digit. The left image shows a handwritten 4 written in grayscale. The kernel we use is meant to capture vertical edges. For the middle image, when the kernel is placed on the green region, we want it to indicate that



Figure 2.29: Illustration of a kernel sliding over an image of a handwritten digit 4 from the MNIST dataset. The green rectangle indicates the current position of the kernel. In the middle image, the kernel is positioned over a region containing a vertical edge. In the right image, the same kernel is repositioned over a region where no vertical edge is present.

there is a vertical edge present. If we reuse the same kernel on the right image, we want it to say that there is no vertical edge there. By solely using this kernel here, we can already capture where in the image a vertical edge resides. We capture this with a scalar feature; a high scalar indicates the presence of the pattern, and a low scalar indicates its absence. With the proposed kernel, we can map an image to a representation telling what region of the image contains a vertical edge. This can be done by convolving the kernel over multiple regions of the image, i.e., we move the kernel over the regions of the image to verify whether the pattern is present or absent. This is where the name convolutional neural network comes from. If we repeat this process for multiple layers and with different kernels, we can enrich the representation of the raw image to make a final decision at the end. The representation is again built hierarchically since each layer makes use of previously generated representations [LeCun et al., 2015]. In our example, we have assumed that the kernel was fixed to capture a vertical edge, but in the spirit of deep learning, we can let the model learn whatever pattern with the backpropagation algorithm. With some trade-off, the model will learn kernels to recognise patterns useful for solving the task that might not be interpretable for a human.

With this idea, we can generate representations from a raw input image, using reusable parameters that exploit the spatial structure of the image. This yields more parameter-efficient models that better exploit the structure of image data, as demonstrated empirically in [Krizhevsky et al., 2012].

2.3.5 Kernel

In this subsection, we describe kernels, the building blocks of CNNs. Like feed-forward neural networks, CNNs consist of layers. However, this time each layer consists of kernels [Goodfellow et al., 2016]. We have already discussed the role of kernels intuitively, but we have not yet specified how they are formally represented. A kernel is a third-order tensor of scalars. When a kernel detects a pattern in a region, we want the output of some operation involving the kernel and that region to be high. In the absence of that pattern, we want it to be low. The value that is computed is the dot product between the kernel and the region of pixels [O’Shea and Nash, 2015]. This means that a kernel used to detect a vertical edge must be constructed in such a way that when the dot product is computed with a region containing that vertical edge, it must return a high value, and a low value otherwise. Figure 2.30 illustrates an example of a vertical edge detector for a single channel. It shows on the left a 9×9 image of the digit 1, along with the corresponding pixel values ranging between 0 and 255, where each value indicates the pixel density. The blue grid shows a 3×3 kernel that detects a vertical edge. When this kernel convolves over the region indicated in blue in the original image, it produces a high value of 765, which is the sum of the element-wise products between the kernel and that region. In

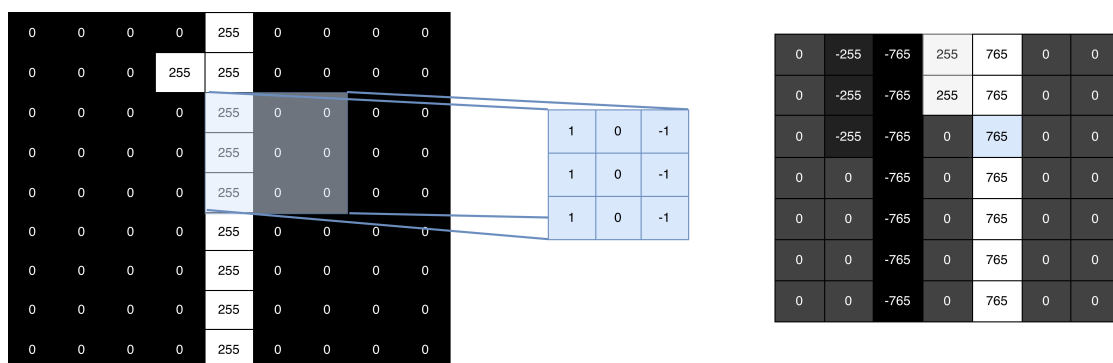


Figure 2.30: Illustration of a vertical edge detector kernel convolving over a 9×9 grayscale image of the digit 1, inspired by [Dhiman et al., 2021]. Pixel values range between 0 and 255, with high numbers indicating high intensity of the pixel. The blue region on the left shows the current position of the kernel, whose weights are shown in the middle. The output of the convolution is shown on the right, where the highlighted cell corresponds to the dot product computed at that position, producing a high value of 765 where the vertical edge is present and low values elsewhere.

regions where the pattern is absent, it produces a low value.³ The grid on the right of the figure shows the output of the kernel convolved over each position of the image. This matrix is also often referred to as a feature map. For clarity, we keep this example to a single channel, but the extension to multiple channels is straightforward. The image and the kernel both have additional channels. However, the output remains a single channel.

In our previous example, we made a couple of assumptions about the setup, which we generalise now. A kernel has a size, which determines the neighbourhood it examines to produce a value. When convolving over an image, the way the kernel moves can be controlled through a parameter called the stride. In figure 2.30, the kernel size was 3×3 , and the stride was 1. A stride of 1 means that the kernel moves one pixel at a time. Additionally, padding can be added to the original image. This means that the image will be surrounded by additional values, which are typically 0. This can serve two different purposes. First, it allows the kernel to better scan the borders of the image. Without padding, a kernel can only convolve over each border pixel a limited number of times. Second, the padding can also be used to preserve the spatial dimension of the input. Since the convolution operation typically reduces the spatial size of its input, adding an appropriate amount of padding can ensure the output keeps the same spatial dimensions as the input [Goodfellow et al., 2016].

2.3.6 Architecture

A kernel in a single layer allows us to capture a single pattern in an image. However, in practice, we want to capture multiple different patterns simultaneously on the same image. This is achieved by using multiple kernels within a single layer. Each kernel convolves over the input independently and produces its own feature map [LeCun et al., 2015]. These feature maps are then stacked together to form the output of the layer, which is a third-order tensor where each channel corresponds to a different detected pattern. The weights of a convolutional layer are

³This kernel is known as the Prewitt kernel in the literature and is primarily used for edge detection [Bhardwaj and Mittal, 2012].

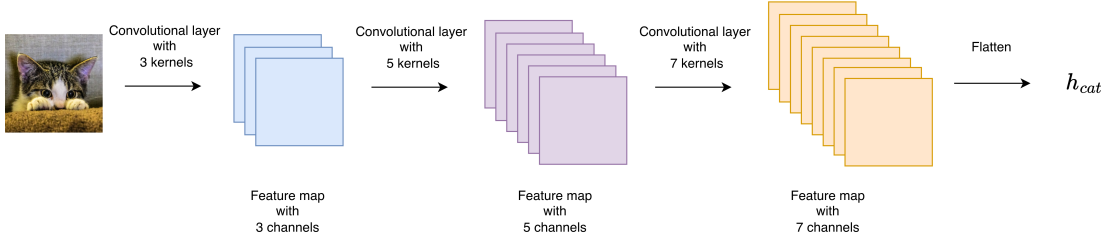


Figure 2.31: Illustration of a CNN architecture with three convolutional layers. The first layer consists of three kernels, each producing a second-order feature map, which, when combined with each other, form a third-order tensor with three channels. The second layer has five kernels, which are used to generate a new feature map based on the output of the previous feature map. Lastly, the last layer repeats this process with seven different kernels. This last feature map is flattened to produce a vector h_{cat} representing the cat within the image.

represented as a fourth-order tensor with shape $C_{out} \times C_{in} \times H \times W$ since there is one kernel of shape $C_{in} \times H \times W$ for every output channel C_{out} , which corresponds to the number of kernels in that layer.

Figure 2.31 shows an example of a CNN with 3 layers. The first layer consists of 3 different kernels, each producing a single feature map. These individual feature maps are then combined into a single feature map consisting of three channels. In this layer, low-level patterns within the image itself are extracted. In the next layer, 5 kernels are used, resulting in a feature map with 5 channels. However, this time the kernels operate on the feature map produced by the previous layer, not on the original image. This means that this layer now uses extracted low-level patterns from the previous layer instead of raw pixels from the image. Lastly, the third layer consists of 7 kernels, effectively capturing 7 different patterns simultaneously on the same input to generate a feature map consisting of 7 channels. This last feature map can be flattened (reshaped) into a first-order vector h_{cat} and represents the embedding of the cat image. The embedding of the cat image can be used, for instance, to solve an image classification task. In that case, it should provide enough information such that one can extract whether the image contains a cat or not based on h_{cat} . In this example, we made the output of each layer explicit by depicting a third-order tensor as a stack of rectangles (matrices). However, in the literature, to formally represent an architecture, this is not usually done. Instead, the output of each layer is omitted and depicted as an arrow indicating where that output goes. The same architecture is illustrated in figure 2.32. A layer is depicted as a box with a label indicating what that layer represents. This can be used for any type of layer and is not restricted to the use of convolutional layers.

The last component we discuss that is specific to CNNs is the pooling layer. The goal of a pooling layer is to reduce the spatial dimension of a feature map while merging and preserving the most important information [LeCun et al., 2015]. Pooling layers have no learnable parameters, act purely as spatial dimension reducers, and introduce local translation invariance since summarising a small neighbourhood into a single value makes the output insensitive to small shifts in the input. Reducing the spatial dimension of the input means that the width and the height of the spatial map are reduced, while the number of channels remains the same. This is important for two reasons. First, since it reduces the size of a feature map, it results in fewer computations. This is particularly important for deep CNNs. As CNNs grow deeper, the number of channels in a feature map typically increases with depth. Without a reduction in spatial dimension, this would lead to a significant growth in computational cost. Second, at the end of a CNN, the final

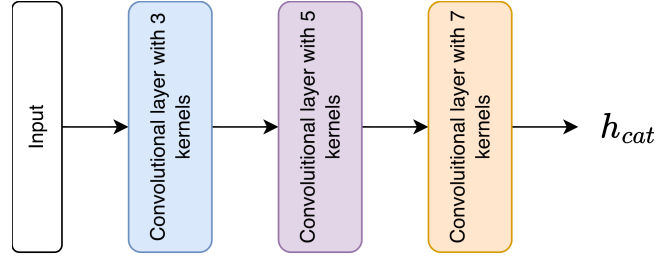


Figure 2.32: Formal illustration of a CNN architecture with three convolutional layers. Each box represents a layer, with the label indicating what that layer represents. The incoming arrow within a box represents the input that the layer receives, and its outgoing arrow represents its output. The colour of the boxes is chosen arbitrarily. However, we use them to indicate layers that contain learnable parameters.

feature map can be large. If this feature map is then passed to a feed-forward neural network for the final classification step, for example, the number of parameters can quickly become large. Pooling can therefore help keep the total number of parameters low. In the literature, there are two types of pooling layers often used. These are the max pooling layers and the average pooling layers [LeCun et al., 2015; O’Shea and Nash, 2015]. Both pooling layers operate on the same mechanism. The feature map is divided into non-overlapping regions of a fixed size. For each region, a single scalar is computed based on an aggregate function. The choice of the aggregator function determines the type of pooling. If the aggregator function takes the maximum of a region, it gives the max pooling, if it takes the average, it gives the average pooling.

2.3.7 Training

We now explain how CNNs are trained on supervised learning tasks. We cover this part as it yields important insights into understanding how current state-of-the-art CNNs are built. The same backpropagation algorithm is used as introduced in subsection 2.3.3, except that it now operates on the kernel parameters. Algorithm 1 illustrates at a high level how a CNN is trained. Given a labelled dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ containing N samples, the goal of the training algorithm is to find kernel parameters $\{\mathcal{K}_i\}_{i=1}^n$ that will help solve the learning task at hand. Each sample is passed through the CNN to compute the output of the model at the given learning task with the current parameters $\{\mathcal{K}_i\}_{i=1}^n$. The predicted outcome $\hat{y}$ is then compared with the ground truth y via a loss metric $\mathcal{L}$. The loss metric $\mathcal{L}$ should reflect the performance of the model at the learning task for the given sample. In our formulation, we assume that we want to minimise it and consider it as an error measure. Formally speaking, the goal of the algorithm, in this formulation, is to find the parameters $\{\mathcal{K}_i\}_{i=1}^n$ that will minimise the loss $\mathcal{L}$ on dataset $\mathcal{D}$ in the hope that it will generalise to other unseen data not present in $\mathcal{D}$. Based on this loss, all gradients with respect to the kernel parameters of the CNN are computed to determine how those kernel parameters should change. The gradient $\frac{\partial \mathcal{L}}{\partial \mathcal{K}_i}$ indicates in what direction the parameters of the kernels must be changed. To determine how strongly they need to change, the learning rate $\alpha \in [0, 1]$ is used for that. This process is repeated for a certain number of iterations, which are called epochs. [Goodfellow et al., 2016] explains the optimisation algorithm in greater mathematical detail. Nevertheless, we highlight how the gradients $\frac{\partial \mathcal{L}}{\partial \mathcal{K}_i}$ are computed because the equation gives us insights into the working and limitations of CNNs. It also motivates why state-of-the-art CNNs are structured the way they are.

Computing these gradients is done through the chain rule from calculus. Let us assume that

Algorithm 1: Training a CNN [Goodfellow et al., 2016]

Input: Dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, learning rate α , number of epochs T
Output: Trained kernel parameters $\{\mathcal{K}_i\}_{i=1}^n$

for $t = 1$ **to** T **do**
 for *each batch* $(x, y) \sim \mathcal{D}$ **do**
 $\hat{y} \leftarrow$ forward pass of x through all n layers
 $\mathcal{L} \leftarrow$ compute loss between $\hat{y}$ and y
 Compute $\frac{\partial \mathcal{L}}{\partial \mathcal{K}_i}$ for all i via backpropagation
 $\mathcal{K}_i \leftarrow \mathcal{K}_i - \alpha \frac{\partial \mathcal{L}}{\partial \mathcal{K}_i}$ for all i
 end
end
return $\{\mathcal{K}_i\}_{i=1}^n$

the loss is denoted by $\mathcal{L}$ and that our CNN consists of n layers. The gradient of $\mathcal{L}$ with respect to the kernel parameters $\mathcal{K}_i$ of layer i is computed by

$$\frac{\partial \mathcal{L}}{\partial \mathcal{K}_i} = \frac{\partial \mathcal{L}}{\partial h_n} \cdot \frac{\partial h_n}{\partial h_{n-1}} \cdot \frac{\partial h_{n-1}}{\partial h_{n-2}} \cdots \frac{\partial h_{i+1}}{\partial h_i} \cdot \frac{\partial h_i}{\partial \mathcal{K}_i}$$

with h_l the hidden representation of layer l [Goodfellow et al., 2016; Lecun et al., 1998; Rumelhart et al., 1986]. For deep layers, this equation results in a long chain of multiplications. This poses one major problem that arises whenever deep neural networks are trained: the vanishing gradient problem [Bengio et al., 1994]. This problem follows directly from the equation. Since the equation consists of a product of many terms, if (some) individual terms are small, the product vanishes with depth. This makes the network not trainable since the gradient update will be very small. There are multiple solutions for this problem. One solution is to only work with shallow neural networks. This, however, imposes a representational limit; since the network is shallow, we cannot learn as abstract features as we could with deep layers. Two solutions found in the literature are the use of normalisation and residual connections as popularised by [He et al., 2016].

Normalisation helps control the distribution of a layer's output h_i [Goodfellow et al., 2016]. By normalising the output of a layer, it ensures that the $\frac{\partial h_i}{\partial h_{i-1}}$ components do not grow or vanish, but remain controlled. It keeps those terms in a stable range and hence prevents the product from collapsing or exploding regardless of the depth. There are multiple types of normalisation, each with its own advantages and disadvantages. These include batch normalisation, root mean square normalisation, layer normalisation, and more. The normalisation phase can or cannot be parametrised. Figure 2.33 shows what modifications are needed to include normalisation in the CNN architecture. The output of every layer is normalised before feeding it to the next layer.

Another successful technique to avoid the vanishing gradient problem is the use of residual connections in the architecture as popularised by [He et al., 2016]. Instead of controlling the scale of the outputs h_i like normalisation does, residual connections restructure the computations. Instead of computing $h_i = f(h_{i-1})$, a layer now computes $h_i = h_{i-1} + f(h_{i-1})$. This additional component has a significant effect on the gradient. Computing $\frac{\partial h_i}{\partial h_{i-1}}$ now yields an additional component

$$\frac{\partial h_i}{\partial h_{i-1}} = \frac{\partial f(h_{i-1})}{\partial h_{i-1}} + 1$$

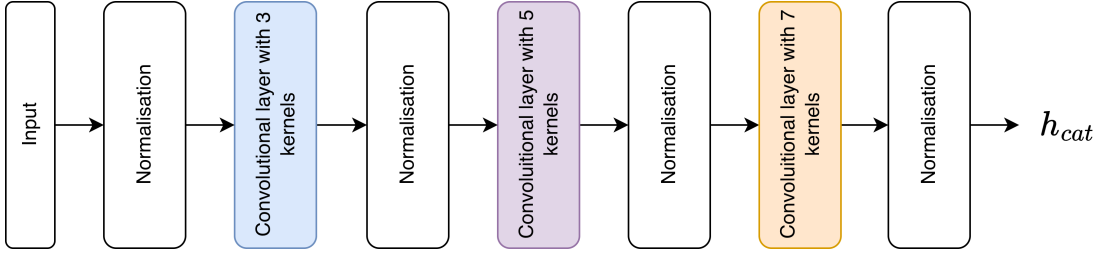


Figure 2.33: Illustration of a CNN architecture with normalisation. The output of every layer, including the input of the CNN architecture, is normalised. Each normalisation step rescales the output of a layer to a stable distribution before it is passed to the next layer.

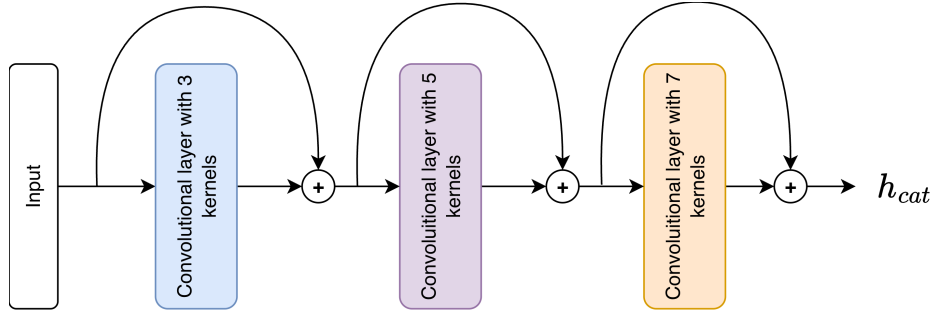


Figure 2.34: Illustration of a CNN architecture with residual connections. The output of a layer l , with input x , is now $x + f(x)$, where $f(x)$ denotes the transformation applied to x by layer l . This is depicted by curved arrows that bypass each convolutional layer and are added to the output of every layer.

The $+1$ term makes a drastic difference in the gradient computation. Even if $\frac{\partial f(h_{i-1})}{\partial h_{i-1}}$ is very small, causing the vanishing gradients problem, the term is unlikely to vanish because of that additional term. Figure 2.34 shows what modifications are needed to include the residual connections in the CNN architecture. The output of every layer is now added to the input of the layer. Even though both techniques solve the problem differently, modern deep neural networks use both together.

With this, all necessary concepts have been introduced about CNNs. In the next section, we cover existing interpretability work for neural networks. It discusses, among other things, how tensor networks can be used to analyse neural networks. More information about CNNs can be found in this introductory survey [O'Shea and Nash, 2015]. More general information about CNNs and deep learning is to be found in [Goodfellow et al., 2016; LeCun et al., 2015].

2.4 Interpretability for neural networks

Despite their strengths and good performance, convolutional neural networks remain black-box models. They consist of millions or billions of parameters, which we do not understand as a whole. This is a problem in domains where understanding the model's behaviour is critical, such as healthcare. A mistake in a model's prediction, without understanding how the model reached that conclusion, can be fatal for a patient. Without this interpretability, we can never be sure that a model makes a correct prediction, and hence we can never safely replace critical decisions

with such models.

This section bridges the two pillars of this thesis, the tensor networks and the convolutional neural networks. We discuss how existing interpretability techniques make a neural network interpretable, and how tensor networks fit in these. We start by making a distinction between two paradigms for interpretability: post-hoc interpretability and mechanistic interpretability [Pearce et al., 2025].

2.4.1 Post-hoc interpretability

Most of today’s work focuses on post-hoc interpretability. In this paradigm, the model is treated as a black-box model. It consists of feeding input data into a trained model, observing its prediction, and then using this observation to draw conclusions about the model’s behaviour. For example, to understand how a model classifies the digit 4 illustrated in figure 2.29, one might remove a region of pixels within the image and observe when the classification breaks down. This region of pixels can be used as evidence that the model relies on it to make its prediction. The work of [Zeiler and Fergus, 2014] demonstrates and discusses this technique. While this paradigm offers interpretability by directly relating input to output, it requires diverse additional data for every output class. This data might not always be available, and the construction of such datasets can introduce bias. Furthermore, the interpretation is solely based on this additional dataset. With that dataset, one can never span the entire input space. So one can understand why a model made a decision, but never with complete certainty, since the analysis is tied to a particular dataset. On top of that, the model is treated entirely as a black box, and the internal parameters or representations defining the behaviour of the model are not analysed at all. A better technique would be to reverse-engineer the mechanism of a model by analysing its internal mechanisms Doods et al. [2025]. This paradigm, known as mechanistic interpretability, is introduced next. Other examples of well-known post-hoc interpretability research include [Petsiuk et al., 2018; Ribeiro et al., 2016].

2.4.2 Mechanistic interpretability

Interpretability techniques following this paradigm seek to understand the inner mechanism of a neural network. They do not treat the neural network as a black-box model, but rather analyse it in detail. Intuitively, this paradigm is the most trustworthy, since we analyse the model’s internal mechanism. This paradigm can be further divided into two approaches: activation-based approaches and weight-based approaches Pearce et al. [2025].

Activation-based approach. The goal of the activation-based approaches is to open the box and analyse the intermediate activations or properties produced by each layer given some input data. For example, the work of [Selvaraju et al., 2017] discusses how the gradients of a model can be used to construct a heatmap highlighting the importance of regions within an image to make a prediction. It requires backpropagation through the network and access to internal values of the network, hence making it an activation-based approach. The techniques within this approach suffer from the same problem as post-hoc interpretability; they depend on some additional data. They try to understand the model based on additional samples, and not the samples used during training. However, this time the interpretability is constructed based on the model’s internal mechanism, rather than treating it as a black-box model. A better alternative are weight-based approaches discussed next. They remove the need to use additional data by analysing the trained weights of the model, and make the model interpretable without linking it to additional data.

Weight-based approach. Intuitively, the most trustworthy interpretation we could have is based on techniques that analyse the learned parameters of a model directly. Since the model parameters define the behaviour of the model, and hence its output, their analysis yields insights into the model’s internal workings. Furthermore, analysing the trained parameters of a model does not require any additional input data; it only requires trained parameters using training data.

In the case of image digit classification, the work of [Pearce et al., 2025] has demonstrated how the parameters of a feed-forward neural network can be analysed to understand a model’s behaviour using tensor networks. The analysis reveals for each output class (a digit), how a prototypical digit looks like, and how anti-prototypical digits look like. Based on these visualisations, humans can verify whether these prototypes are coherent with the underlying digit. This yields more informative and trustworthy insights than the other two approaches discussed since the conclusions are based on parameters and tied to a class, rather than tied to some input data.

Despite the weight-based approaches yielding more informative and trustworthy insights into a model’s behaviour grounded in its parameters, these approaches are typically constrained in their usage. The models on which they can be applied are architecturally restricted. Especially models with non-linearities form a primary obstacle. The analytic tools used in weight-based approaches typically rely on linear algebra, but non-linearities do not fit with those. This makes it impossible to use linear algebra techniques without severe approximation. Since almost all state-of-the-art CNN architectures make use of non-linearities, we cannot directly apply weight-based techniques to them [Pearce et al., 2025].

In this work, we follow this approach and aim to make CNNs interpretable on image classification tasks by analysing their weights without relying on any additional input data. This requires that the CNNs we design avoid non-linearities that would obstruct the analysis. To analyse the weights, we use tensor networks. Since they express linear transformations, they are well suited to our architectural constraint. The analysis we do is based on our extension of the work of [Pearce et al., 2025]. We convert our CNN model into an equivalent tensor network, building on the concepts introduced in section 2.2 and the works of [Dooms et al., 2025; Nie et al., 2025; Pearce et al., 2025; Stoudenmire and Schwab, 2016], and then later analyse that tensor network for a shallow case using our extension introduced in subsection 3.3.3.

2.4.3 Tensor networks as an interpretability tool

In this subsection, we explain how tensor networks can be used to make models interpretable via weight-based approaches. All necessary concepts have been introduced in section 2.2, but we now explain how they can be used to analyse a tensor network and reveal information about the model.

Before the analysis can be performed, the model must first be converted into an equivalent tensor network. Here is where the connection between tensor networks and CNNs is created. We say that a tensor network $\mathcal{T}$ is equivalent to a model f if for all x holds that the contraction of $\mathcal{T}$ and x is equal to $f(x)$. Concretely, this means that the output of the model f should be equal to the contraction of its tensor network counterpart with input x incorporated in that tensor network. Constructing this tensor network is not straightforward, as every operation of the CNN should be converted to its tensor network counterpart. Furthermore, every operation (linear or non-linear) of the model should be expressed as a linear operation in the tensor network. Not every non-linear operation is expressible as a linear operation in a tensor network. For example,

the Rectified Linear Unit (ReLU) is popularly used in neural networks as a non-linear operation

$$f(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

is simply not expressible as a linear operation. This imposes several constraints on the original model. There are two solutions to this problem. The first one is to avoid the use of non-linear functions, but that means that the model is limited and cannot represent non-linear relations, which are ubiquitous in data [Abu-Mostafa et al., 2012]. The second one is to use non-linear operations that are expressible as multilinear operations. We choose this last approach and illustrate in chapter 3 how we implemented it. Assuming that this conversion is done, we can use the resulting tensor network $\mathcal{T}$ and analyse it using linear algebra techniques for tensors.

Since a tensor network expresses only multilinear operations, linear algebra techniques become directly applicable and can be used to yield insights into the tensor network, and hence into its model equivalent counterpart. The linear algebra techniques, such as the singular value decomposition or the eigendecomposition, can be used in this case. These can be applied by first contracting the tensor network into a single tensor, which represents the behaviour of the model, and then reshaping it into a matrix to enable its analysis. This is illustrated in the work of [Pearce et al., 2025]. This only works for small and shallow models since, as discussed in section 2.2, contracting the entire tensor network into a single tensor is not a good idea due to its exponential growth in order N . The work of [Dooms et al., 2025; Wang et al., 2025] proposes a solution for this and discusses how to analyse a (tree) tensor network without the need to contract it to a single tensor.

Chapter 3

Methods

3.0.1 Overview

This chapter is structured in three parts. It first presents the CNN architecture we built with the aim of achieving performance comparable to state-of-the-art CNNs on standard benchmarking datasets. It then describes how this architecture is converted to an equivalent tensor network. Finally, we analyse a shallow CNN for interpretability. This analysis is done by first training the CNN on a digit classification task, and then converting the resulting CNN to an equivalent tensor network. This tensor network is then analysed using our weight-based technique extension of the work of [Pearce et al., 2025].

3.1 Convolutional neural network architecture

The architectural design space of our CNN is constrained by two key requirements. First, the resulting CNN must be convertible to an equivalent tensor network. This means that every component of the architecture must be individually convertible to an equivalent tensor network, which rules out some building blocks used in state-of-the-art CNN models. Second, the final architecture must deliver performance comparable to state-of-the-art CNNs since our goal is to provide a credible interpretable substitute for CNNs. Our goal is not to achieve marginally better accuracies. These two requirements make the task not trivial because the first one reduces the design space drastically, and the second one requires achieving performance comparable to state-of-the-art CNNs within that restricted design space. With this restricted design space, we cannot simply reuse architectures from existing work, because almost all state-of-the-art CNNs use components that are not convertible to an equivalent tensor network. For example, all state-of-the-art CNNs we compare our model with in section 4.1 use non-linear components that are not mappable to a tensor network equivalent (e.g., ReLU). Consequently, we must explore different architectures and components that are each convertible to a tensor network, and try to achieve comparable performance. In what follows, we discuss the final architecture and then discuss each component in greater detail.

3.1.1 Architecture

The architecture is presented in figure 3.1. It is a multi-stage deep residual convolutional neural network designed for image classification tasks. The network consists of N stages, each composed of a sequence of residual blocks. Each stage doubles the number of channels of its input and

progressively builds richer representations. Every stage is followed by an average pooling layer, which reduces the spatial dimensions between stages. This is the only component that changes the spatial dimensions of the feature maps, the other components preserve the same spatial dimensions. At the end of the network, a global average pooling is applied. This pooling reduces every channel to a single value by averaging all values channel-wise. This significantly reduces the size of the final representation compared to flattening the full feature map directly, which would otherwise result in a very large input to the following linear layer. The resulting tensor is then flattened to a vector, which represents a task-specific representation of the input image. This flatten operation stacks all values of the first channel, then the second channel and so on. This representation is then passed through a linear layer that acts as a classifier, which produces an output for each output class.

3.1.2 Residual block

The residual block structure is depicted in figure 3.2. It implements the mapping

$$x \rightarrow f(x) + \text{skip}(x) \quad (3.1)$$

with third-order input tensor x , function f representing the chain of alternating root mean square (RMS) normalisation and bilinear layer (presented in subsection 3.1.3) components, and skip representing the residual connection. The chain represented by $f(x)$ preserves the spatial dimensions of its input x , but may increase the number of channels depending on the placement of the residual block within a stage. Each stage doubles the number of channels of its input, so there must be at least one residual block in a stage increasing the number of channels. That role is taken by the first residual block within a stage. When the number of channels doubles in the residual block, the skip function is a parametrised convolutional layer that doubles the channels of x to match the number of channels of $f(x)$. When the number of channels remains constant, the skip function acts as the identity and hence $\text{skip}(x) = x$. The benefit of using a residual connection for the vanishing gradient problem is therefore reduced compared to a pure residual connection, but this approach is standard in the literature for deep convolutional neural networks that make use of residual connections [He et al., 2016].

3.1.3 Bilinear layer

As illustrated previously, bilinear layers are part of residual blocks, and their structure is depicted in figure 3.3. It implements the mapping

$$x \rightarrow P(L(x) \odot R(x)) \quad (3.2)$$

with third-order input tensor x , convolutional layers P , L , and R and $\odot$ the element-wise multiplication operator. Functions L and R are both convolutional layers that operate independently on the same input x , each with their own learnable parameters. They project the input x to a higher-dimensional space by doubling the number of channels. In this higher-dimensional space, the two outputs $L(x)$ and $R(x)$ are combined through an element-wise multiplication before being downprojected to the original number of channels of x by the convolutional layer P . The choice of using bilinear layers is inspired by [Dooms et al., 2025; Pearce et al., 2025] and grounded in the tensor network conversion. We have already mentioned that our architecture cannot contain components that are not convertible to tensor networks. Most of the incompatible components are non-linearities since tensor networks can only express linear operations. However, the non-linear components of CNNs are important, since these are the ones that allow the model to learn

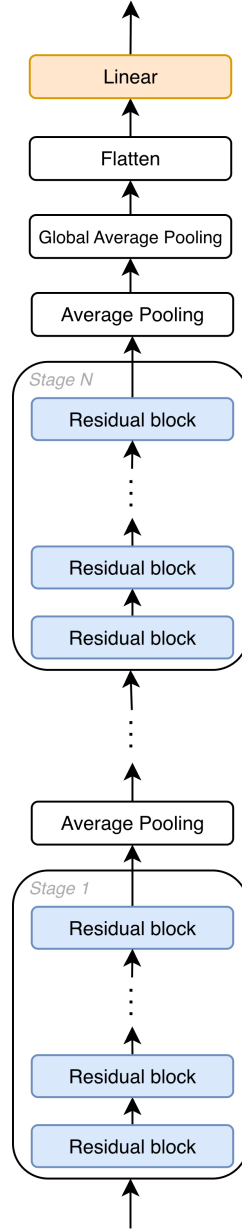


Figure 3.1: Overview of the proposed CNN architecture for image classification convertible to a tensor network. The network is organised in N sequential stages. Each stage consists of a sequence of residual blocks. The stages are interleaved with average pooling layers, which reduce the spatial dimensions of the intermediate feature maps. After the final stage, a global average pooling reduces the output to a single value per channel, followed by a flatten operation and a linear layer.

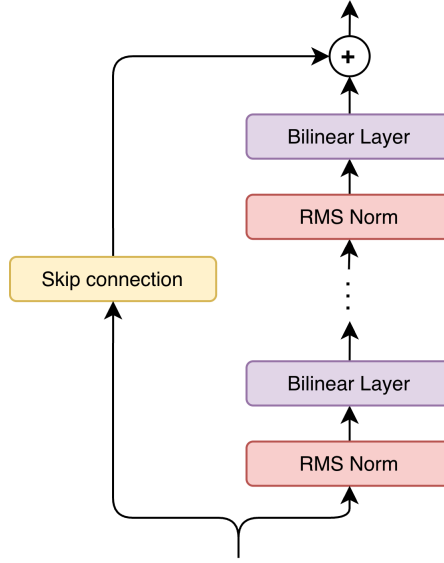


Figure 3.2: Overview of a residual block, part of the proposed CNN architecture. The residual block implements the mapping $x \rightarrow f(x) + \text{skip}(x)$ with third-order tensor input x . The function f represents the chain of alternating RMS normalisation and bilinear layer components. This chain produces $f(x)$, which has the same width and height as x , but may not have the same number of channels. If x and $f(x)$ have the same number of channels, then the skip function acts as the identity, i.e., $\text{skip}(x) = x$. Otherwise, the skip function changes the number of channels of x by a parameterised convolutional layer.

non-linear relationships, which are ubiquitous in real-world data Abu-Mostafa et al. [2012]. This means that we need non-linear components to express non-linear relationships, but they must be convertible to tensor networks. That is why we use bilinear layers. Although the element-wise multiplication is non-linear, it remains expressible as a multilinear operation and is convertible to an equivalent tensor network component. The work of [Pearce et al., 2025] uses bilinear layers for vectors; we extend this concept to third-order tensors. Despite allowing us to model non-linear relationships, element-wise multiplications introduce a practical challenge. It can cause the output to grow or vanish during training because of the quadratic interaction. This makes normalisation and residual connections even more necessary, both of which are present in the proposed architecture.

3.1.4 Normalisation

So far, we have introduced one mechanism to stabilise training in our architecture, and that was the residual connection. However, we also use normalisation after each bilinear layer to stabilise training. There exist various types of normalisations, but our choice is grounded in the tensor network conversion again. We need to pick one that is convertible to an equivalent tensor network component. That is why we use the RMS normalisation. It may be worth investigating how alternative normalisation techniques affect the performance of this architecture, but the RMS norm is one option that can be naturally incorporated into the tensor network framework. Other commonly used normalisations such as batch normalisation [Ioffe and Szegedy, 2015] and

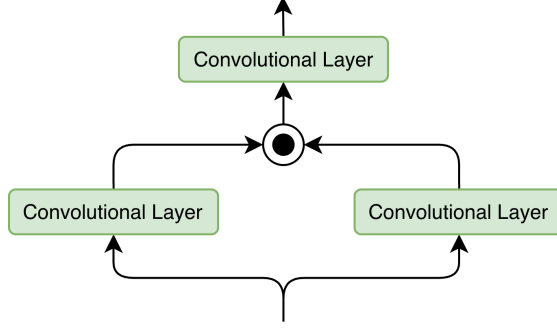


Figure 3.3: Overview of a bilinear layer, part of the proposed CNN architecture. A bilinear layer implements the mapping $x \rightarrow P(L(x) \odot R(x))$ with $\odot$ the element-wise multiplication operator. The functions L and R are both independent convolutional layers applied to the same input x . Their role is to project the input x to a higher-dimensional space by doubling the number of channels of input x . Both outputs are combined with each other through an element-wise multiplication that introduces non-linear pairwise interactions. This output is then fed to the function P , which is a convolutional layer whose role is to downscale the number of channels back to the number of input channels.

layer normalisation [Ba et al., 2016] are also convertible to tensor network equivalents, but we chose the RMS normalisation because of its efficiency [Zhang and Sennrich, 2019]. The RMS normalisation implements the following mapping [Zhang and Sennrich, 2019] for a third-order input tensor x

$$x \rightarrow \frac{x \cdot \gamma_i}{\text{RMS}(x)} \quad (3.3)$$

where γ_i is a learnable scale parameter for channel i of x . The $\text{RMS}(x)$ is defined as

$$\text{RMS}(x) = \sqrt{\epsilon + \frac{1}{n} \sum_{i=1}^n x_i^2}$$

with n the total number of elements over which the normalisation is computed, and $\epsilon > 0$ a small constant added for numerical stability, preventing a division by zero when the input values are very small. The RMS norm has two variants depending on how n is defined. These are the channel-wise and global variants. In the channel-wise variant, n corresponds to the number of elements within a single channel, meaning each channel receives its own denominator computed from its own values exclusively. In the global variant, n corresponds to the total number of elements across all channels, yielding a single shared denominator for the entire input. We experimented with both variants, but only the global variant is compatible with tensor network conversion, as we will argue in subsection 3.2.1.

3.1.5 Implementation and training

The architecture is benchmarked on four datasets to give a picture of how it compares with other state-of-the-art CNNs. These datasets are MNIST [Lecun et al., 1998], CIFAR-10 [Krizhevsky, 2009], CIFAR-100 [Krizhevsky, 2009], and SVHN [Netzer et al., 2011]. These are standard

Table 3.1: Overview of benchmark image classification datasets.

Dataset	Train size	Test size	Image size	Classes	Description
MNIST	60,000	10,000	$1 \times 28 \times 28$	10	Handwritten digit recognition (0-9)
CIFAR-10	50,000	10,000	$3 \times 32 \times 32$	10	Object recognition
CIFAR-100	50,000	10,000	$3 \times 32 \times 32$	100	Object recognition
SVHN	604,388	26,032	$3 \times 32 \times 32$	10	House number recognition



Figure 3.4: Samples from each benchmarking dataset. Each row shows five samples of each dataset. The rows are ordered in this order: MNIST, CIFAR-10, CIFAR-100, and SVHN.

datasets used for image classification benchmarking. Their characteristics are listed in table 3.1, and samples from each dataset are illustrated in figure 3.4.

The loss function $\mathcal{L}$ we use is the cross-entropy loss, which is the standard for classification tasks. To prevent overfitting, three regularisation techniques were used. These are the L2 regularisation (weight decay), label smoothing [Szegedy et al., 2016], and data augmentation techniques. The data augmentation strategy differs per dataset, and an overview of the used techniques for each dataset is provided in table 3.2. All datasets are first normalised using dataset-specific parameters to stabilise training. MNIST has no data augmentation as it consists of small grayscale images with only 10 classes. SVHN has no augmentation too because of the large amount of training samples. CIFAR-10 and CIFAR-100 require more aggressive augmentation due to their greater complexity. The chosen data augmentation techniques follow the standard methods found in the literature [Wightman et al., 2021]. The weights of the model

Table 3.2: Data augmentation techniques applied per dataset during training. The full hyperparameter details (role and values) for each augmentation technique are provided in appendix B.

Augmentation	MNIST	CIFAR-10	CIFAR-100	SVHN
Random crop ¹	×	✓	✓	×
Random horizontal flip ²	×	✓	✓	×
Random augment [Cubuk et al., 2020]	×	×	✓	×
Random erasing [Zhong et al., 2020]	×	✓	✓	×
Mixup [Zhang et al., 2018]	×	✓	✓	×
CutMix [Yun et al., 2019]	×	✓	✓	×

are initialised using PyTorch’s default initialisation for convolutional layer weights, which is the Kaiming uniform initialisation [He et al., 2015].

The model and the benchmarking experiments were implemented using PyTorch on an NVIDIA system with L40S GPUs with 64 cores. Further training details, including all hyperparameters, are discussed in section 4.1 and appendix B.

3.1.6 Hyperparameters

The proposed architecture is flexible within its construction as it has several architectural hyperparameters. These hyperparameters control the depth, width, and capacity of the network. This allows the architecture to be scaled to different datasets and tasks. The architectural hyperparameters are listed below

- Number of stages: the number of stages within the proposed architecture.
- Number of residual blocks: the number of residual blocks within each stage, which is the same across all stages.
- Repeat factor: the number of RMS normalisations and bilinear layer alternations within a single residual block.
- Upscale factor: the factor by which the number of channels is increased in the hidden dimension of a bilinear layer, i.e., the number of output channels of L and R in $L(x) \odot R(x)$ from equation (3.2).
- Channels factor: the factor by which the number of channels is multiplied at each stage.
- Starting output channels: the number of output channels produced by the first residual block. The first block does not follow the channels factor, but immediately changes the number of input channels to this value.
- Kernel size: the fixed kernel size used in every convolutional layer.
- RMS normalisation mode: determines how the RMS normalisation layer computes the RMS denominator term from equation (3.3).
- Bias: determines whether the convolutional layer and linear layer use a bias term or not.

The remaining hyperparameters related to training are not further discussed here since they are standard and not specific to the proposed architecture. Nevertheless, we provide more details about their role and values in appendix B for each dataset.

3.2 Tensor network conversion

We now address the conversion of the proposed architecture into an equivalent tensor network. We first analyse the model's computations in greater detail and highlight simplifications that facilitate the conversion. The intuition behind the conversion is then presented, followed by a detailed description of how each component is mapped to an equivalent tensor network. This section is concluded by presenting the overall architecture and showing how all components fit together to represent the proposed architecture as a tensor network.

3.2.1 Computational simplifications

Given our current architecture, some of its computations can be simplified. It is important to note that these simplifications facilitate the tensor network conversion and result in a different tensor network than one would obtain without them. This is fine as long as both final tensor networks are equivalent to the model.

The first simplification we apply concerns the computations of a single residual block. The function f of equation (3.1) is defined as a chain of alternating RMS normalisations and bilinear layers. A chain consisting of n alternations with input x is defined as

$$f(x) = g_n \circ g_{n-1} \circ \dots \circ g_1(x) \quad \text{with} \quad g_i(x) = \text{bilinear}_i(\text{RMS}_i(x))$$

representing the i th pair of an RMS normalisation layer followed by a bilinear layer. It turns out that all intermediate normalisation factors of the RMS normalisation cancel out, and only a single normalisation factor from the last but one output survives. Let us formally derive this.

Proof. Let x_n be defined as

$$x_n = k_n \circ k_{n-1} \circ \dots \circ k_1(x_0) \quad \text{with} \quad k_i(x) = \text{bilinear}_i(x \cdot \gamma_i) \quad (3.4)$$

with γ_i the scalar of the i th RMS layer of the residual block. It represents the output of the chain of the first n bilinear layers to output x_0 of the residual block, where the RMS normalisation does not divide by its normalisation term but scales the input with γ_i . We show by induction on n that

$$f(x_0) = \frac{x_n}{\text{RMS}(x_{n-1})^2}$$

Base case ($n = 1$). Using equation (3.2) and equation (3.3) the first pair represented by

$g_1(x_0)$ simplifies to

$$\begin{aligned}
g_1(x_0) &= \text{bilinear}_1(\text{RMS}_1(x_0)) \\
&= P\left(L\left(\frac{x_0 \gamma_1}{\text{RMS}(x_0)}\right) \odot R\left(\frac{x_0 \gamma_1}{\text{RMS}(x_0)}\right)\right) \\
&= P\left(\frac{L(x_0 \gamma_1)}{\text{RMS}(x_0)} \odot \frac{R(x_0 \gamma_1)}{\text{RMS}(x_0)}\right) \\
&= P\left(\frac{L(x_0 \gamma_1) \odot R(x_0 \gamma_1)}{\text{RMS}(x_0)^2}\right) \\
&= \frac{P(L(x_0 \gamma_1) \odot R(x_0 \gamma_1))}{\text{RMS}(x_0)^2} \\
&= \frac{\text{bilinear}_1(x_0 \gamma_1)}{\text{RMS}(x_0)^2} \\
&= \frac{x_1}{\text{RMS}(x_0)^2}
\end{aligned}$$

In this simplification, we used the fact that P , L , and R are linear, meaning that constants can be put upfront. Convolutional layers are linear in their inputs since they consist of a dot product with a kernel. This derivation only holds if the RMS normalisation uses the global variant since it produces one scalar for the entire input, allowing it to be put upfront. With the channel-wise normalisation variant, this simplification would not be possible. That is why our architecture uses the global variant throughout this work.

Inductive step. Assume the output after $n - 1$ pairs is

$$\frac{x_{n-1}}{\text{RMS}(x_{n-2})^2}$$

We need to show that the output after n pairs is

$$f(x_0) = g_n\left(\frac{x_{n-1}}{\text{RMS}(x_{n-2})^2}\right) = \frac{x_n}{\text{RMS}(x_{n-1})^2}$$

First the RMS normalisation is applied to $\frac{x_{n-1}}{\text{RMS}(x_{n-2})^2}$. Using the property that $\text{RMS}(\frac{x}{c}) = \frac{\text{RMS}(x)}{c}$ for scalar $c > 0$ (proved in appendix A.2), the RMS normalisation step simplifies to

$$\begin{aligned}
\text{RMS}_n\left(\frac{x_{n-1}}{\text{RMS}(x_{n-2})^2}\right) &= \frac{\frac{x_{n-1} \gamma_n}{\text{RMS}(x_{n-2})^2}}{\text{RMS}\left(\frac{x_{n-1}}{\text{RMS}(x_{n-2})^2}\right)} \\
&= \frac{\frac{x_{n-1} \gamma_n}{\text{RMS}(x_{n-2})^2}}{\frac{\text{RMS}(x_{n-1})}{\text{RMS}(x_{n-2})^2}} \\
&= \frac{x_{n-1} \gamma_n}{\text{RMS}(x_{n-1})}
\end{aligned}$$

This simplification allows us to cancel the $\text{RMS}(x_{n-2})^2$ term entirely. Applying the bilinear layer to $\frac{x_{n-1}\gamma_n}{\text{RMS}(x_{n-1})}$ yields

$$\text{bilinear}_n \left(\frac{x_{n-1}\gamma_n}{\text{RMS}(x_{n-1})} \right) = \frac{x_n}{\text{RMS}(x_{n-1})^2}$$

This shows that after n pairs of RMS normalisation and bilinear layers, the output of f with input x_0 is

$$f(x_0) = \frac{x_n}{\text{RMS}(x_{n-1})^2}$$

□

This proof shows that all intermediate normalisations are not necessary for strict equivalence, and only the RMS of the last but one output x_{n-1} of the residual block survives. However, we can not discard an RMS normalisation layer completely, since the computation of x_n still requires the intermediate γ 's of these layers. Substituting this simplification of f into the residual block equation (3.1) gives

$$x_0 \rightarrow \frac{x_n}{\text{RMS}(x_{n-1})^2} + \text{skip}(x_0)$$

Factoring the RMS term upfront allows us to rewrite the equation as

$$x_0 \rightarrow \frac{1}{\text{RMS}(x_{n-1})^2} \left(x_n + \text{RMS}(x_{n-1})^2 \text{skip}(x_0) \right) \quad (3.5)$$

With this proof, we have illustrated that the output of a residual block can be computed with equation (3.5) and does not necessarily need to apply the RMS normalisation between every bilinear layer. From a tensor network conversion perspective, this simplification removes the need to map every RMS normalisation layer to an equivalent tensor network component. This, however, introduces one practical challenge worth noting. Since the intermediate normalisations are dropped, the outputs of x_i can vanish or explode for deep networks, since their roles were to stabilise the input. If we use the tensor network to analyse a model, this is not a problem. However, it means that if we use the tensor network as a direct substitute for the model, a direct numerical comparison between the output of the model and its equivalent tensor network, given the same input, may result in some large differences. We explain in subsection 3.2.11 how this problem is solved.

This simplification propagates to the stage level, where a chain of residual blocks can be simplified by using the new residual block equation (3.5). Before diving deeper into the new formulation, we introduce some notation to keep equations compact and readable. For residual block k , we define

$$\lambda_k = \text{RMS}(x_{n_{k-1}}^{(k)})^2$$

as the surviving normalisation scalar, n_k the number of pairs of RMS normalisation and bilinear layers of block k , $x_n^{(k)}$ as equation (3.4), but tied to residual block k , and u_k as the unnormalised output of block k , i.e., the numerator of equation (3.5). We build the simplification for a stage by considering an example with three residual blocks starting with input x_0 . Following equation (3.5), the first residual block maps x_0 to

$$\frac{1}{\lambda_1} u_1 \quad \text{with} \quad u_1 = x_{n_1}^{(1)} + \lambda_1 \text{skip}(x_0)$$

The second residual block takes $\frac{u_1}{\lambda_1}$ as input and following equation (3.5) it produces

$$\frac{1}{\lambda_2} \left(x_{n_2}^{(2)} + \lambda_2 \text{skip} \left(\frac{u_1}{\lambda_1} \right) \right)$$

Since the skip function is either the identity or a convolutional layer, both of which are linear, we can factor out the scalar $\frac{1}{\lambda_1}$ because

$$\text{skip}\left(\frac{u_1}{\lambda_1}\right) = \frac{1}{\lambda_1} \text{skip}(u_1)$$

The output of the second residual block then becomes

$$\begin{aligned} \frac{1}{\lambda_2} \left(x_{n_2}^{(2)} + \lambda_2 \text{skip}\left(\frac{u_1}{\lambda_1}\right) \right) &= \frac{1}{\lambda_2} \left(x_{n_2}^{(2)} + \lambda_2 \frac{1}{\lambda_1} \text{skip}(u_1) \right) \\ &= \frac{1}{\lambda_1 \lambda_2} (\lambda_1 x_{n_2}^{(2)} + \lambda_2 \text{skip}(u_1)) \\ &= \frac{1}{\lambda_1 \lambda_2} u_2 \end{aligned}$$

with

$$u_2 = \lambda_1 x_{n_2}^{(2)} + \lambda_2 \text{skip}(u_1)$$

Applying the same argument to the third residual block yields

$$\frac{1}{\lambda_1 \lambda_2 \lambda_3} (\lambda_1 \lambda_2 x_{n_3}^{(3)} + \lambda_3 \text{skip}(u_2)), \quad u_3 = \lambda_1 \lambda_2 x_{n_3}^{(3)} + \lambda_3 \text{skip}(u_2)$$

There is a clear pattern here. Each additional residual block contributes one normalisation term λ_i to the global denominator and weights the skip connection. The other normalisation terms are used altogether to weigh $x_{n_k}^{(k)}$, which represents the output of applying equation (3.4) for residual block k starting from input u_{k-1} . The equation can be summarised as follows for one stage containing K residual blocks

$$\frac{1}{\prod_{i=1}^K \lambda_i} \left(\prod_{j=1}^{K-1} \lambda_j x_{n_k}^{(j)} + \lambda_K \text{skip}(u_{K-1}) \right) = \frac{1}{\prod_{i=1}^K \lambda_i} u_K \quad (3.6)$$

with

$$\lambda_k = \text{RMS}(x_{n_{k-1}}^{(k)})^2, \quad u_0 = x_0, \quad u_k = \prod_{i=1}^{k-1} \lambda_i x_{n_k}^{(i)} + \lambda_k \text{skip}(u_{k-1})$$

Despite facilitating the tensor network conversion, these simplifications have two critical implications for the tensor network conversion. First, the ability to factor the division term $\frac{1}{\prod_{i=1}^K \lambda_i}$ allows us to delay all non-constant division operations until the end of the tensor network. This is important because a division is a non-linear operation and therefore cannot be expressed as a tensor network. With this simplification, we can focus on a tensor network computing u_K and $\prod_{i=1}^K \lambda_i$, and after the tensor network, apply the division manually. Furthermore, all required RMS terms are squared because of the quadratic interactions in bilinear layers. This is crucial for the conversion since the RMS consists of a square root operation, which has no tensor network equivalent. However, by squaring it, the square root disappears, and the remaining square term can be computed using tensor networks, as will be illustrated in subsection 3.2.5.

3.2.2 Conversion outline

With these architecture simplifications, we can now build up the intuition of the tensor network conversion. The goal of this conversion is illustrated in figure 3.5. Given our convolutional neural

network architecture, we want to construct an equivalent tensor network. This means that for every third-order input x_0 , the output of the model with input x_0 must be equal to the output of the contraction of the equivalent tensor network with input x_0 . The resulting tensor network can then be used for an interpretability analysis of the corresponding model using techniques discussed later in section 3.3.

The conversion is based on the idea of converting every layer or component separately to an equivalent tensor network, and then chaining all these tensor networks together. This is intuitive since every layer applies a mapping to its input, and so does every tensor network component when contracted with its input. It also makes the conversion modular and extensible since any layer can be replaced by another as long as an equivalent network component exists for it. However, this extensibility is reduced to some extent due to the model specific simplifications we introduced earlier, but the underlying conversion concepts remain applicable to other architectures. Each tensor network component must be thought of as simulating a corresponding layer. However, a mapping exactly simulating every single layer of the model is not possible since tensor networks are fundamentally limited compared to the convolutional neural network layers. For example, the RMS normalisation layer of a convolutional neural network computes the RMS term and then uses that term to divide the scaled input. These are two operations that cannot be simulated in a tensor network, namely computing the RMS (involves a square root) and division. That is why we have introduced the simplifications in the previous subsection. With the current simplifications in place, we have shown that it is not necessary to divide at every single normalisation step; we only need to divide at the very end because we are always capable of putting the RMS term upfront. Furthermore, the RMS never needs to be computed since it is squared. This means that the square root can be avoided, and its computation is expressible using tensor networks. However, this imposes one restriction on the tensor network. Since we need to divide by $\prod_{i=0}^N \lambda_i$ at the very end, we need to compute it and keep track of it, but tensor networks have no memory. That is why, besides computing u_K , the final tensor network also needs to keep track of $\prod_{i=0}^N \lambda_i$. There is only one way to store variable data in a tensor network, and that is in the input and output of a tensor network component. That is why the input and output of any tensor network component is not simply a third-order tensor x , but rather a third-order tensor of the form

$$(1, \Lambda, x, y) \in \mathbb{R}^{(2+C_x+C_y) \times H \times W}$$

This form represents a third-order tensor with one channel consisting of ones, one channel consisting of the scalar $\Lambda = \prod_{i=1}^N \lambda_i$ representing the accumulated RMS scalar so far, some channels dedicated to data x , and some channels dedicated to data y . The data channels x and y are both tensors of shape $C_x \times H \times W$ and $C_y \times H \times W$ respectively. The newly introduced form has shape $(2 + C_x + C_y) \times H \times W$. The width and height are fixed for each channel and set to H and W , respectively. Concretely, the tensor has the following structure

$$\left[\begin{array}{c} \begin{bmatrix} 1 & \cdots & 1 \end{bmatrix} \\ \vdots \\ \begin{bmatrix} 1 & \cdots & 1 \end{bmatrix} \end{array}, \begin{array}{c} \begin{bmatrix} \Lambda & \cdots & \Lambda \end{bmatrix} \\ \vdots \\ \begin{bmatrix} \Lambda & \cdots & \Lambda \end{bmatrix} \end{array}, \underbrace{\begin{array}{c} \begin{bmatrix} x_{1,1} & \cdots & x_{1,W} \end{bmatrix} \\ \vdots \\ \begin{bmatrix} x_{H,1} & \cdots & x_{H,W} \end{bmatrix} \end{array}}_{C_x \text{ channels}}, \dots, \underbrace{\begin{array}{c} \begin{bmatrix} y_{1,1} & \cdots & y_{1,W} \end{bmatrix} \\ \vdots \\ \begin{bmatrix} y_{H,1} & \cdots & y_{H,W} \end{bmatrix} \end{array}}_{C_y \text{ channels}}, \dots \end{array} \right]$$

By working with this format, we introduce a storage mechanism in our tensor network. The channel consisting of the Λ scalar value can store the RMS scalar we need to accumulate until the end of the tensor network. Working with a storage mechanism imposes one restriction on

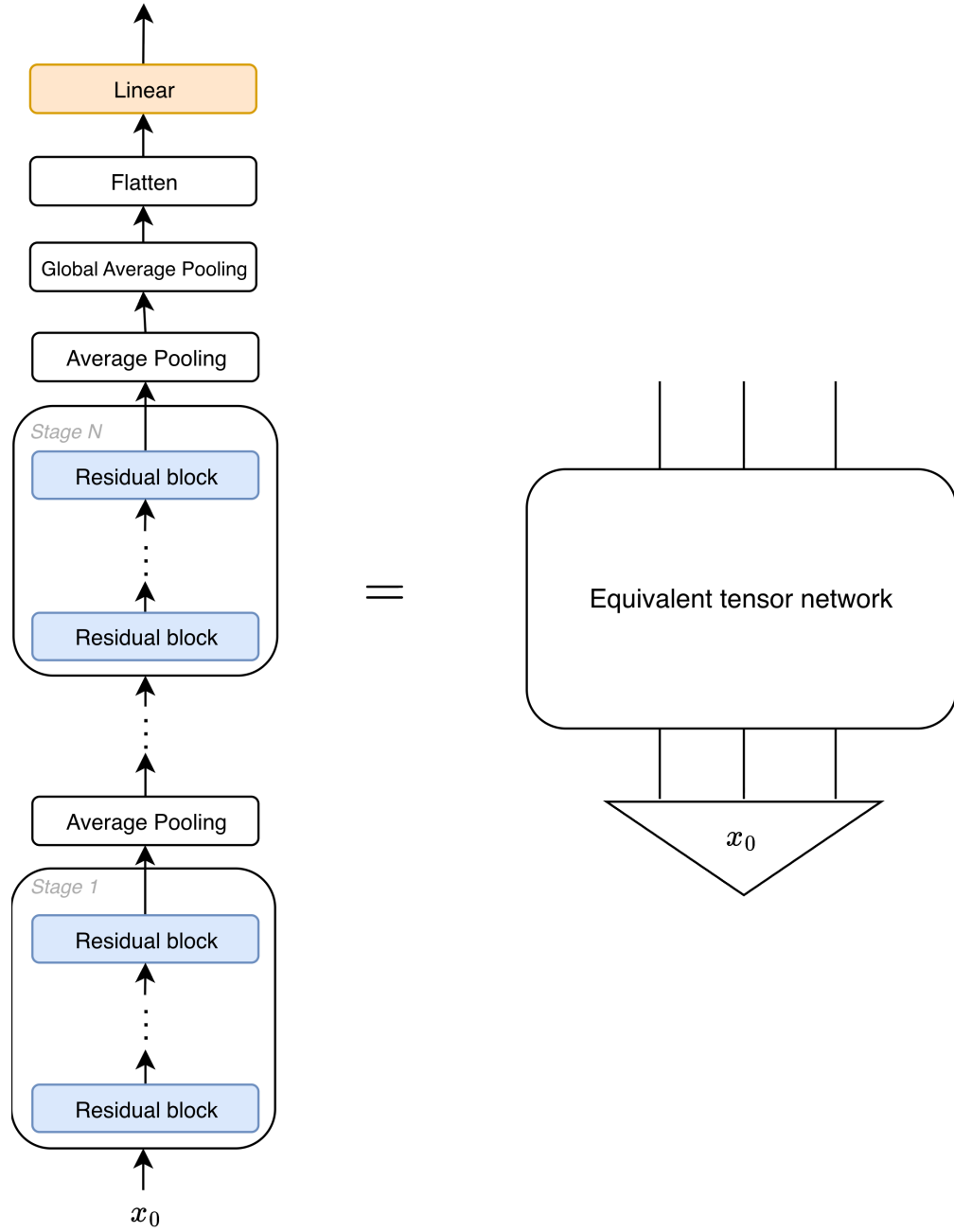


Figure 3.5: Tensor network conversion goal illustration. The goal of the conversion is to convert the convolutional neural network (left) to an equivalent tensor network (right). The tensor network is equivalent to the model if and only if for every third-order input x_0 , the output of the model equals the contraction of the tensor network with x_0 . Note here that the equivalent tensor network is depicted as a 6th-order tensor, but in fact that tensor network is of higher order. It is only depicted as a sixth-order tensor for illustration purposes.

the tensor network components. If a tensor network component does not need to modify data meant to store information, but applies its operation on data channels of x , for example, then it must leave those parts untouched. Those parts must pass through the tensor network untouched. Concretely, it means that if a tensor network applies an operation f on x , and only x , it must implement the mapping

$$(1, \Lambda, x, y) \rightarrow (1, \Lambda, f(x), y)$$

The resulting tensor network component now has two roles: it must apply the operation f on x , but it must also keep all other data untouched. This introduces some overhead in each conversion. That is why the format also has a channel consisting of ones; it allows us to use this data to simulate a pass through mechanism, as will be illustrated in subsection 3.2.3. The slot containing Λ is used to store the RMS scalar we need to accumulate, the slot containing x is used for the data channels used to store intermediate data, and y is used to store the skip connection data within a single residual block.

The complete conversion outline is presented in figure 3.6. It shows the CNN architecture on one side and the state of the corresponding tensor network output at most boundary layers of the architecture on the other side. This gives a complete picture of what each block is expected to compute in the tensor network equivalent. The input of the tensor network is initialised with $(1, 1, x_0, x_0)$. The RMS scalar Λ is initialised to 1 since no residual blocks have been processed yet, and both data slots carry the original input x_0 . In this illustration, we have assumed that λ^P is equal to the product of all λ 's generated by the residual block within stage P , and u^P is used to denote the unnormalised output of that stage. The functions *AVG* and *GAVG* represent the average pooling and global average pooling. The flatten operation reshapes the third-order tensor into a vector, dropping the constant channel and skip data channels since they are no longer needed. The linear layer operates on this vector, and the final output is computed by dividing that result by the accumulated RMS scalar. With this outline, the problem of constructing an equivalent tensor network is reduced to ensuring that each component implements its required mapping correctly.

The remainder of this section explains how each component of the CNN architecture is mapped to an equivalent tensor network component. We explain the complete conversion bottom-up by starting with the building blocks of the architecture. For each component, we first state the mapping it implements and then explain how the tensor network implements it. We use the following notation to keep equations and definitions to the point and compact

- $x_m^{P,N}$ denotes the output of bilinear layer m in the P th residual block of stage N .
- u^N denotes the unnormalised output at stage N .
- u_P^N denotes the unnormalised output of the P th residual block in stage N .
- λ^N denotes the RMS term of stage N .
- λ_P^N denotes the RMS term generated by the P th residual block of stage N .
- $\prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^P \lambda_p$ denotes the accumulated RMS scalar at residual block P of stage N .
- $T[i, j, k]$ denotes the tensor entry $T_{i,j,k}$.
- $T[i, :, k]$ denotes all values of the second index of tensor T , where the first and third indices are fixed to i and k respectively.
- $T[:, j, k]$ denotes all values of the second index of tensor T up until dimension j , where the first and third indices are fixed to i and k respectively.

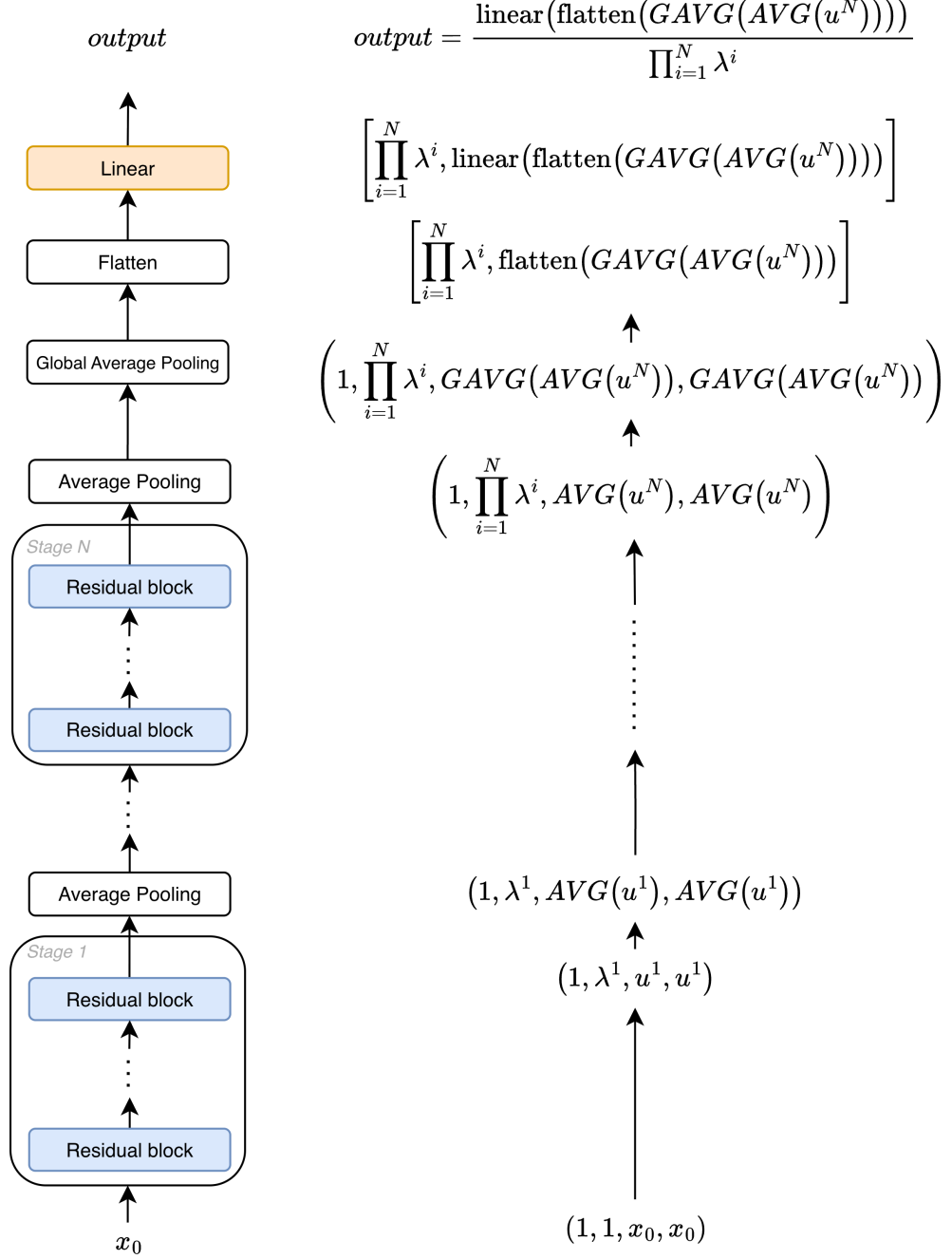


Figure 3.6: Tensor network conversion outline. The left part shows the CNN architecture, and the right part shows the state of the tensor network output at most boundary layers of the architecture. The brackets notation $(1, 1, x_0, x_0)$ represents a third-order tensor, while the square brackets notation represents a first-order tensor (a vector). The first entry of each tuple represents a matrix consisting of ones, needed for some intermediate operations. The second entry represents the accumulated RMS scalar in matrix form. This scalar needs to be computed and accumulated until the end to divide the result with. The two other entries are used to represent data channels.

The input of all mappings has the form $(1, \Lambda, x, y)$, but for each conversion we make it more concrete by specifying what Λ , x , and y represent concretely at each layer. All tensor network diagrams should be interpreted bottom-up starting with the input. The input consists of three wires, representing from left to right the height, channels, and width of the input. Throughout the remaining subsections, we assume zero-indexing of tensors in our definitions.

3.2.3 Convolutional layer

A convolutional layer can be elegantly represented as a tensor network. Its role is to simulate the selection of input patches and compute their dot product with the learned kernel parameters, all within a single tensor contraction. We first present the conversion for a third-order input tensor x of shape $C_{in} \times H_{in} \times W_{in}$, and later extend it to the augmented input format required by our architecture. Figure 3.7 shows a convolutional layer with its equivalent tensor network. The equivalence means that feeding an input x to the convolutional layer produces the same result as contracting the equivalent tensor network with that same input x . This conversion works for any stride, padding, kernel size, and number of output channels. The tensor network consists of three tensors, which are C^H , C^W , and K , each playing a different role. Tensor K has shape $C_{out} \times C_{in} \times K_h \times K_w$, with K_h and K_w the kernel size of the height and width, and contains the learned kernel parameters of the convolutional layer directly. It is the only tensor in the tensor network that contains parameters of the convolutional layer. The other two tensors, C^H and C^W , encode the convolution operation for the height and width, respectively, and contain no learnable parameters. Both are determined by the kernel size, stride, and padding of the convolutional layer. To understand how these tensors simulate the convolution, it is useful to write out the result of the tensor network contraction explicitly

$$o_{c'h'w'} = \sum_c \sum_h \sum_w \sum_{k_h} \sum_{k_w} x_{chw} \cdot C_{h'hk_h}^H \cdot C_{w'wk_w}^W \cdot K_{c'ck_hk_w}$$

This tensor contraction over all shared indices should represent the dot product of an input region and a kernel. To achieve this dot product, which is a weighted sum with the kernel parameters as weights, we need C^H and C^W to act as selectors by setting their entries to 1 at the right spots and 0 elsewhere. By doing so, the output $o_{c'h'w'}$ becomes a weighted sum of an input region with the necessary kernel parameters as weights. All other components that should not contribute to the dot product are zero because of the selectors C^H and C^W . These two tensors, with shapes respectively $H_{out} \times H_{in} \times K_h$ and $W_{out} \times W_{in} \times K_w$, are hence responsible for encoding the convolutional operation. The entries of C^H are defined as

$$C^H[h_{out}, h_{in}, k_h] = \begin{cases} 1 & \text{if } h_{in} = h_{out} \cdot \text{stride} + k_h - \text{padding} \text{ and } 0 \leq h_{in} < H_{in} \\ 0 & \text{otherwise} \end{cases}$$

The entry $C^H[h_{out}, h_{in}, k_h]$ equals 1 when input row h_{in} is the one that the kernel element at position k_h should be multiplied with, and this accounting for the stride and padding. The boundary condition $0 \leq h_{in} < H_{in}$ handles the padding. All input positions that fall outside the input boundary are 0 and should not contribute. The tensor C^W is constructed identically along the width dimensions. By contracting C^W , C^H , and K together with input x , the full convolutional operation is reproduced, and this as a single tensor contraction.

This tensor network presents the base case for a simple third-order input x of shape $C_{in} \times H_{in} \times W_{in}$. However, as explained previously, our tensor network carries an input of the form $(1, \Lambda, x, y)$. When a convolutional layer needs to be applied to this input, it is applied to a data

slice of it, which is either x or y . The mapping the tensor network implements is

$$\begin{aligned}
 & \left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p, u_{P-1}^N, u_{P-1}^N\right) \\
 & \quad \downarrow \\
 & \left\{ \begin{aligned} & \left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p, \text{conv}(u_{P-1}^N), u_{P-1}^N\right) & (1) \\ & \left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p, u_{P-1}^N, \text{conv}(u_{P-1}^N)\right) & (2) \end{aligned} \right. \quad (3.7)
 \end{aligned}$$

Depending on what the convolutional layer is used for, either a convolutional layer in a bilinear layer or one used as a skip connection, it should be either applied to the first data slot of data channels or the second one used to represent the skip connection. It should apply the convolution to the target slice while leaving all other channels unchanged. To achieve this, the three tensors C^H , C^W , and K are extended to handle the pass through of the untouched channels. The key idea is to increase the kernel dimension (K_h and K_w) by one to both C^H and C^W , which acts as a pass through slot. This slot is then used to implement an identity for the channels that should remain unchanged. Concretely, these tensors are constructed as follows. Let C_{meta} denote the number of channels that must pass through unchanged. In our case, this includes the constant 1 channel, the RMS scalar channel, and any data channels that are not part of the convolution operation. Tensors C^H and C^W now become respectively tensors $\tilde{C}^H$ and $\tilde{C}^W$ and have shapes $H_{out} \times H_{in} \times (K_h + 1)$ and $W_{out} \times W_{in} \times (K_w + 1)$. The first K_h slots of $\tilde{C}^H$ are identical to the base case of C^H and implement the convolution operation as before. The last slot is used as a pass through slot, it is set to the identity matrix

$$\tilde{C}^H[:, :, K_h] = I_{H_{in}}$$

The same principle is used to construct $\tilde{C}^W$. This slot allows a channel to pass through unchanged, since contracting with the identity simply copies the input to the output. The extended parameter tensor $\tilde{K}$ has shape $(C_{meta} + C_{out}) \times (C_{meta} + C_{in}) \times (K_h + 1) \times (K_w + 1)$ and consists of two (virtual) parts. One part is filled with the original kernel parameters exactly like in the base case

$$\tilde{K}[C_{meta} :, C_{meta} :, : K_h, : K_w] = K$$

The other part is used for the pass through channels, to ensure that they remain untouched, and it is set to 1

$$\tilde{K}[m, m, K_h, K_w] = 1 \quad \text{for each pass through channel } m$$

The result is that the tensor contraction of $\tilde{C}^W$, $\tilde{C}^H$, and $\tilde{K}$ with the augmented input applies the convolution to the target data slice and passes all other channels through unchanged. The structure of the tensor network remains the same as in the base case, but the extended tensors now encode both the convolution and the pass through mechanism. This idea is reused across other layers further in this work. The full resulting tensor network is depicted in figure 3.8. It is the same as the one depicted before, except that it now has the new tensors that, besides the convolutional layer mechanism, also have the pass through mechanism.

3.2.4 Bilinear layer

With the extended convolutional tensor network conversion, we can address the next building block of the model, the bilinear layer. The m th bilinear layer of the P th residual block of stage N implements the mapping

$$\left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^P \lambda_p, x_{m-1}^{P,N}, \text{skip}(x_0^{P,N})\right) \rightarrow \left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^P \lambda_p, x_m^{P,N}, \text{skip}(x_0^{P,N})\right) \quad (3.8)$$

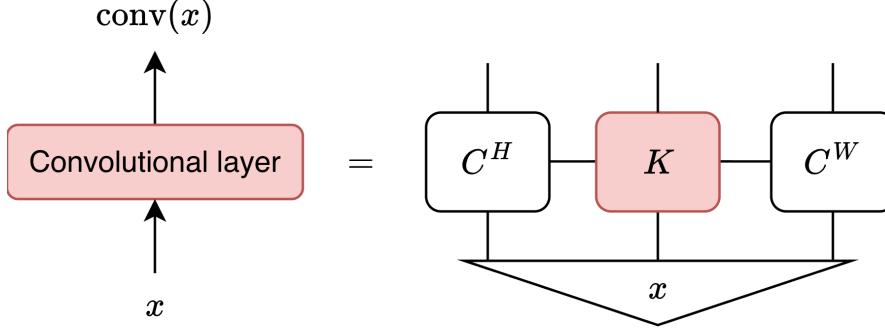


Figure 3.7: Convolutional layer tensor network equivalent. The tensor network consists of three tensors, which are K , C^H , and C^W . The fourth-order tensor K contains the parameters of the convolutional layer; hence, it is highlighted in red. The two other tensors C^H and C^W are used to encode the convolutional operation, and this is for any stride, padding, and kernel size.

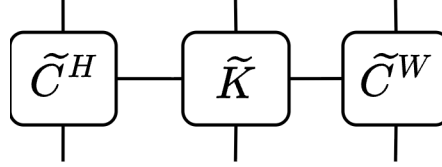


Figure 3.8: Extended convolutional layer tensor network equivalent. This tensor network simulates a convolutional layer on input of the form $(1, \Lambda, x, y)$ by simulating the convolutional layer on data x or y , and by leaving all other channels untouched. It consists of three tensors $\tilde{K}$, $\tilde{C}^H$ and $\tilde{C}^W$. Tensor $\tilde{K}$ contains the parameters of the convolutional layer, while $\tilde{C}^H$ and $\tilde{C}^W$ encode the convolutional operation. This works for any stride, padding, and kernel size.

The role of the bilinear layer conversion is to transform $x_{m-1}^{P,N}$ into $x_m^{P,N}$ while keeping all other channels unchanged. Therefore, the resulting tensor network component must act as a pass through for the constant channel, the accumulated RMS scalar, and the skip connection data, while applying the bilinear operation only on the $x_{m-1}^{P,N}$ data slice. Following equation (3.2), the bilinear operation is defined as $x_m^{P,N} = P(L(x_{m-1}^{P,N}) \odot R(x_{m-1}^{P,N}))$ with convolutional layers L , R , and P . These convolutional layers can already be mapped to three distinct tensor networks using the technique introduced previously in subsection 3.2.3. However, we still need a mechanism to combine these three tensor networks and represent the element-wise multiplication. That role is taken by the tensors S_C , S_H , and S_W . After applying L and R to the input, we have two tensors we need to multiply with each other. For the data channels $L(x_{m-1}^{P,N})$ and $R(x_{m-1}^{P,N})$ we need element-wise multiplication. For all other channels, we want to keep them unchanged. The tensors S_H and S_W are spiders enforcing that corresponding spatial positions from both branches are combined with each other. This is the mechanism that makes the multiplication element-wise. The tensor S^C is responsible for the channels and has two roles. For the data channels, it must implement the element-wise multiplication, and for all other channels, it must act as a pass through, so they do not need to be multiplied with each other. This trick is achieved by multiplying the input of a left branch by the first channel of the right branch, which is always guaranteed to be 1. The result is therefore the same value unchanged, and that is why the

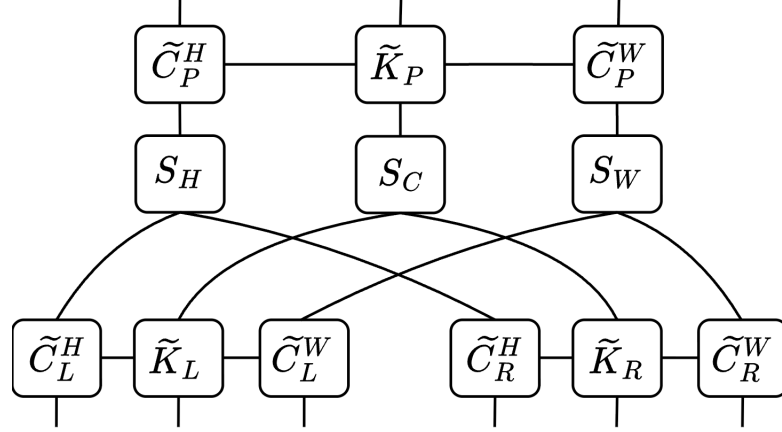


Figure 3.9: Tensor network equivalent of the bilinear layer. The tensors $\tilde{C}_L^H, \tilde{K}_L$ and $\tilde{C}_L^W$ represent the convolutional layer L of the bilinear layer, the tensors $\tilde{C}_R^H, \tilde{K}_R$ and $\tilde{C}_R^W$ represent the convolutional layer R of the bilinear layer, and $\tilde{C}_P^H, \tilde{K}_P$ and $\tilde{C}_P^W$ the convolutional layer P of the bilinear layer. The tensors S_C, S_H , and S_W are used to combine these and implement the element-wise multiplication along with a pass through mechanism.

constant one channel is kept along almost the entire conversion. The tensor S_C is defined as

$$S_C[c_{\text{out}}, c_L, c_R] = \begin{cases} 1 & \text{if } c_L = c_{\text{out}} \text{ and } c_R = c_{\text{out}} \text{ and } c_{\text{out}} \in \text{data channels} \\ 1 & \text{if } c_L = c_{\text{out}} \text{ and } c_R = 0 \text{ and } c_{\text{out}} \notin \text{data channels} \\ 0 & \text{otherwise} \end{cases}$$

In this definition, we assume that the constant channel is taken from the right branch to pass the left branch through unchanged. The reverse would have been equally correct.

The resulting tensor network is illustrated in figure 3.9. It consists of twelve tensors grouped in three different rows. The bottom row represents, respectively, from left to right, the L and R convolutional layers. They both receive the same input, this will become clear when a bilinear layer tensor network is composed with the preceding tensor network component in subsection 3.2.6. The middle row contains the three tensors S_C, S_H , and S_W , which combine the outputs of L and R through element-wise multiplication for the data channels representing $x_{m-1}^{P,N}$ and a pass through for the remaining channels. The top row represents the convolutional layer P , which projects the hidden representation back to the original number of channels. The contraction of all shared indices within this tensor network yields the mapping from equation (3.8).

3.2.5 Normalisation

The RMS normalisation conversion benefits strongly from the architectural simplifications since we have shown that we only need the normalisation term of the last but one output of a residual block. However, an RMS normalisation layer cannot be skipped entirely since its role is also to scale the input with the learnable factor γ stored in the layer. Depending on the position of the RMS layer within a residual block, the conversion is different and is represented by the following mapping for RMS layer m in residual block P or stage N containing M pairs of alternations of

RMS and bilinear layers

$$\begin{aligned}
 & \left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p, x_{m-1}^{P,N}, \text{skip} \left(x_0^{P,N}\right)\right) \\
 & \quad \downarrow \\
 & \begin{cases} \left(1, \lambda_P^N, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p, \gamma_m \cdot x_{m-1}^{P,N}, \text{skip} \left(x_0^{P,N}\right)\right) & \text{if } m = M \quad (1) \\ \left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p, \gamma_m \cdot x_{m-1}^{P,N}, \text{skip} \left(x_0^{P,N}\right)\right) & \text{otherwise} \quad (2) \end{cases} \quad (3.9)
 \end{aligned}$$

In case the normalisation layer is the last one within a residual block ($m = M$), the RMS term $\lambda_{m-1}^{P,N}$ needs to be computed. Otherwise, we have illustrated that it is not worth computing due to the architectural simplifications. However, in both cases, the current bilinear data $x_{m-1}^{P,N}$ must be scaled with the learned γ_m of the corresponding RMS normalisation layer m . This is done channel-wise as γ_m consists of a learned value for each channel. Since this mapping consists of two different cases, the conversion consists of two different tensor network components.

We start with the simple case indicated by (2) in equation (3.9) since it is also needed for the complex case. The equivalent tensor network must multiply the data channels $x_{m-1}^{P,N}$ by γ_m while leaving all other channels unchanged. The tensor network for that is presented in figure 3.10. It consists of three tensors, of which two are spiders. The role of the two spiders is identical to that in the bilinear layer; their role is to match corresponding spatial positions from both branches. The remaining tensor K_γ is responsible for both the γ scaling and the pass through, and is defined as

$$K_\gamma[c_{\text{out}}, c_L, c_R] = \begin{cases} \gamma_m^{c_{\text{out}} - C_{\text{pre}}} & \text{if } c_L = c_{\text{out}} \text{ and } c_R = 0 \text{ and } c_{\text{out}} \in \text{data channels} \\ 1 & \text{if } c_L = c_{\text{out}} \text{ and } c_R = 0 \text{ and } c_{\text{out}} \notin \text{data channels} \\ 0 & \text{otherwise} \end{cases}$$

with $\gamma_m^{c_{\text{out}} - C_{\text{pre}}}$ referring to entry $C_{\text{out}} - C_{\text{pre}}$ of vector γ_m and with C_{pre} the number of prefix channels, i.e., the number of channels preceeding $x_{m-1}^{P,N}$. For the data channels this gives $x_c \cdot \gamma_c \cdot 1 = x_c \cdot \gamma_c$, implementing the channel-wise scaling with γ . For all other channels, the scale is 1. This implements the same pass through trick as in the bilinear layer conversion.

The complex case can be decomposed in two steps. The first step computes $\lambda_P^N = \text{RMS}(x_{m-1}^{P,N})$ and inserts it into a new channel. The second step applies the γ scaling using the tensor network from the simpler case. This order is crucial; we must compute λ_P^N from the unscaled $x_{m-1}^{P,N}$ first. If we scale before that, we would then be computing $\text{RMS}(\gamma_m \cdot x_{m-1}^{P,N})$. Since the γ scaling step is already covered by the base case, we only need to explain how the RMS is computed and inserted into a new channel. The reason why it must be inserted into a new channel and not yet accumulated with the other RMS scalar will become clear in subsection 3.2.6 when we cover the residual block conversion. The tensor network for this is illustrated in figure 3.11. It consists of a spider and three fourth-order tensors R_C, R_H , and R_W . The fourth index of these tensors has two dimensions is shared across all three tensors and is connected through a spider, meaning it is summed over during contraction. Hence, we can encode two different operations in that fourth index because the tensor contraction sums these two behaviours together. One is used for the local pass through and the other one is used for the RMS computation. The reason we cannot merge these two operations into a single index is because they are fundamentally different from each other. One is local while the other is global. The first value of the fourth index encodes the local pass through. For R_H and R_w this is a spider just like in the base case. The tensor R_C is responsible for letting every channel pass through, shifted by one position to make room for the

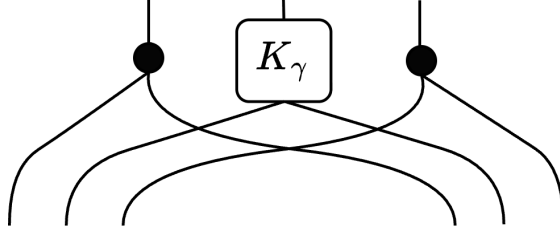


Figure 3.10: Tensor network equivalent of simple RMS normalisation case whose role is to scale the data channels with γ_m for layer m . The tensor K_γ scales the data channels by γ_m while passing all other channels through unchanged. The two spiders enforce the spatial pass through along the spatial dimensions.

new RMS slot. It must let all channels pass through with a scale of 1 for all channels since the γ scaling is done later explicitly. this tensor R_C is defined as

$$R_C[c_{\text{out}}, c_L, c_R, 0] = \begin{cases} 1 & \text{if } c_{\text{out}} = c_L + 1 \text{ and } c_R = 0 \\ 0 & \text{otherwise} \end{cases}$$

The second value of the fourth index encodes the global RMS computation. For R_H and R_W , it enforces that both legs come from the same spatial position, which results in squared values at the same spatial position. It also averages over all positions by defining R_H as

$$R_H[h_{\text{out}}, h_L, h_R, 1] = \begin{cases} \frac{1}{H} & \text{if } h_L = h_R \\ 0 & \text{otherwise} \end{cases}$$

and identically for R_W with factor $\frac{1}{W}$. For R_C , the second value of the fourth index accumulates all data channels into the new RMS slot weighted by $\frac{1}{C}$ with C the number of channels of $x_{m-1}^{P,N}$. This gives the definition

$$R_C[c_{\text{out}}, c_L, c_R, 1] = \begin{cases} \frac{1}{C} & \text{if } c_{\text{out}} = 1 \text{ and } c_L = c_R \text{ and } c_L \in \text{data channels} \\ 0 & \text{otherwise} \end{cases}$$

When contracted, the second value of the fourth index contributes $\frac{1}{C \cdot H \cdot W} \sum_{c,h,w} x_{c,h,w}^2$ while the first value contributes the shifted pass through for all channels. Summing these two together yields the required operation. To implement this complex case completely, one needs to chain the tensor network of this complex case with the tensor network of the base case. But due to the two branches within the tensor network components, it would result in duplicating the first tensor network component. This is sometimes unavoidable, but in this case we can avoid it. In the first dimension of the fourth index, we implemented a scaling of 1 for the local pass through, but we could have just applied the γ scaling instead. We can replace the plain pass through by a scaled γ pass through and elegantly merge these two operations into one single tensor network. This removes the need to chain two separate tensor networks. The second value of the fourth index remains unchanged for all tensors, while the first value now uses the same construction as K_γ from the base case, shifted by one position to make place for the RMS scalar. The final tensor network structure remains the same.

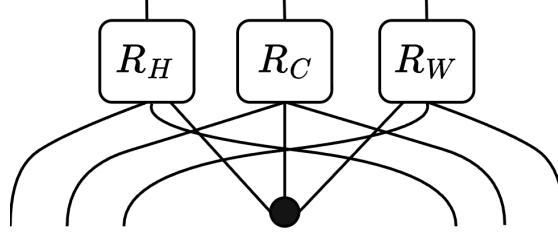


Figure 3.11: Tensor network equivalent of the complex RMS normalisation case whose role is to compute the squared RMS of the data channels, add it into a new slot of the input, and let all other channels pass through. The tensors R_C , R_H , and R_W are fourth-order tensors and encode two different behaviours in their fourth index. The first behaviour is the local pass through of channels, and the second behaviour is the computation of the squared RMS value. These two behaviours are summed over at contraction time by the spider that connects these three tensors with each other.

3.2.6 Residual block

With all individual components covered, we can now explain how a complete residual block is converted into an equivalent tensor network. The P^{th} residual block of stage N implements the mapping

$$\left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N, u_{P-1}^N, u_{P-1}^N\right) \rightarrow \left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^P \lambda_p^N, u_P^N, u_P^N\right)$$

This mapping consists of many different operations. We need to compute λ_P^N and multiply it with the currently accumulated RMS scalar $\prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N$, and we also need to compute u_P^N from u_{P-1}^N with

$$u_P^N = \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N x_M^{P,N} + \lambda_P^N \text{skip}(u_{P-1}^N)$$

assuming that the residual block consists of M pairs of RMS normalisations and bilinear layers, and with $x_0^{P,N} = u_{P-1}^N$. To implement this complex mapping, we decompose it into individual steps. Each step is converted to an equivalent tensor network using the techniques from the previous subsections, and all resulting components are then chained together to yield the tensor network representing the residual block. We walk through the first steps in detail to build up the intuition, and then illustrate the chained tensor network it produces. The first step applies the skip connection and implements the mapping

$$\left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N, u_{P-1}^N, u_{P-1}^N\right) \rightarrow \left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N, u_{P-1}^N, \text{skip}(u_{P-1}^N)\right)$$

Depending on whether the residual block is the first within the stage, the skip function is either a convolutional layer or the identity. In the case of the identity, this mapping does not change anything and no equivalent tensor network is needed. If the skip function is a convolutional layer, the mapping can be implemented using the technique from subsection 3.2.3. To make the example complete, we assume that the skip connection is a convolutional layer. The next step applies the first RMS normalisation layer. Assuming it is not the last RMS layer within the

residual block, the equivalent tensor network implements the mapping

$$\left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N, x_0^{P,N}, \text{skip}(u_{P-1}^N)\right) \rightarrow \left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N, \gamma_1^{P,N} \cdot x_0^{P,N}, \text{skip}(u_{P-1}^N)\right)$$

with $x_0^{P,N} = u_{P-1}^N$. This is implemented using the technique from subsection 3.2.5. The next step applies the first bilinear layer. Its equivalent tensor network implements the mapping

$$\left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N, \gamma_1^{P,N} \cdot x_0^{P,N}, \text{skip}(u_{P-1}^N)\right) \mapsto \left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N, x_1^{P,N}, \text{skip}(u_{P-1}^N)\right)$$

with $x_1^{P,N} = \text{bilinear}_1(\gamma_1 \cdot x_0^{P,N})$, using the technique from subsection 3.2.4. The resulting tensor network of these three operations is illustrated in figure 3.12. Each row within that tensor network represents one of the three operations mentioned before, starting with the skip connection, followed by the first RMS layer, and finally the first bilinear layer. Notice that there are four identical copies of the skip connection tensor network. The reason why these copies are needed is that each subsequent tensor network component requires the same input on its left and right branches. In a tensor network, the only way to feed the same data to two branches is to duplicate the tensor network that generates it. Since the RMS normalisation layer requires a duplication, and the bilinear layer requires a duplication too, the identical skip connection tensor network ends up appearing four times. This exponential growth in the number of copies makes it impractical to draw the full residual block conversion for residual blocks with multiple alternations. We therefore introduce a compact diagrammatic notation illustrated in figure 3.13 inspired by [Dooms et al., 2025]. This notation introduces a triangle primitive whose role is to represent the duplication of its input. It is purely a diagrammatic construction and has no tensor network equivalent. The underlying tensor network is still exponential in the number of tensors, but this notation allows us to draw it compactly. With this notation, we can continue to roll out all operations within a residual block. It consists of repeating the same pattern of RMS normalisation and bilinear layer conversions, except for the last RMS layer, which is treated differently. The full tensor network implementing the chain of alternations, or equivalently, the mapping

$$\left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N, x_0^{P,N}, u_{P-1}^N\right) \rightarrow \left(1, \lambda_N^P, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N, x_M^{P,N}, \text{skip}(u_{P-1}^N)\right)$$

is presented in figure 3.14. It consists of a chain of tensor network components needed to implement this mapping.

The chain of a residual block yields an output that contains all ingredients to compute u_P^N . The remaining steps are to multiply the two RMS scalars λ_N^P and $\prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N$ together, compute the weighted sum $\prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N x_M^{P,N} + \lambda_N^P \text{skip}(u_{P-1}^N)$, and duplicate the result in the last two channels for the next residual block. This is handled by a single tensor network component illustrated in figure 3.15 and implements the mapping

$$\left(1, \lambda_N^P, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N, x_M^{P,N}, \text{skip}(u_{P-1}^N)\right) \rightarrow \left(1, \prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^P \lambda_p^N, u_P^N, u_P^N\right)$$

The two spiders are responsible for combining the corresponding spatial positions with each other, like in the bilinear and RMS conversion case. The core logic resides in tensor P_C . This time,

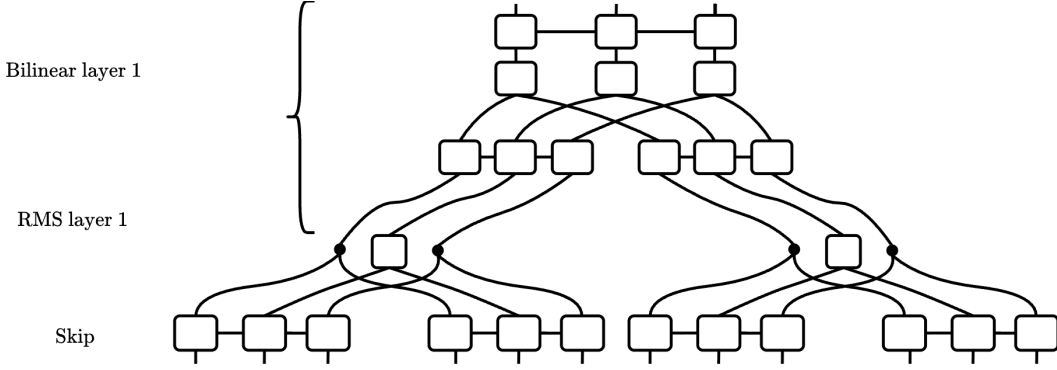


Figure 3.12: Tensor network equivalent of the first three operations of a residual block. The first operation is the skip connection, which is assumed to be a convolutional layer here. It is represented by a tensor network that is copied four times due to the tensor network construction of subsequent operations. The second operation is the RMS normalisation layer, whose role is to scale the input here. Its tensor network component appears twice. Finally, the last layer is bilinear and only appears once since there is no subsequent operation.

we define P_C in fragments rather than providing a single definition since it needs to implement different operations. The constant channel is passed through by

$$P_C[0, 0, 0] = 1$$

which takes the constant channel from both left and right and multiplies them together. This yields the same constant channel in the output. The new accumulated RMS is computed by

$$P_C[1, 1, 2] = 1$$

which takes the previously accumulated scalar $\prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N$ from the left leg at channel index 1, and the newly computed λ_P^N from the right leg at channel index 2, and multiplies them together. The result is placed at the output channel index 1. The weighted data term $\prod_{n=1}^{N-1} \lambda^n \prod_{p=1}^{P-1} \lambda_p^N \cdot x_M^{P,N}$ is computed by

$$P_C[C_{out}, 2, C_x] = 1$$

which takes λ_P^N at channel index 2 from the left leg and the corresponding data $x_M^{P,N}$ from the right leg, and multiplies them together. The weighted skip term $\lambda_P^N \cdot \text{skip}(u_{P-1}^N)$ is computed by

$$P_C[C_{out}, 1, C_{skip}] = 1$$

which takes the previously accumulated scalar from the left leg at channel index 1 and the corresponding skip channel from the right leg, and multiplies them together. Since both entries write to the same output channel C_{out} , their contributions are summed during the tensor contraction, yielding u_P^N . This tensor network needs to be further chained on top of the tensor network from figure 3.15 to yield the complete residual block tensor network conversion.

3.2.7 Pooling layers

The average pooling layer and the global average pooling layer can both be represented using the same tensor network conversion technique as for a convolutional layer. We can do this because

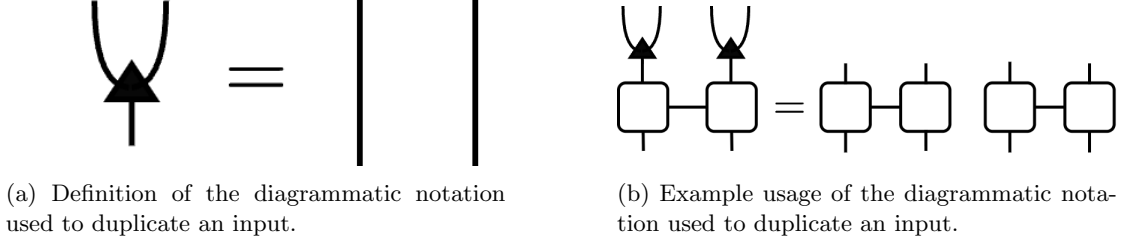


Figure 3.13: Diagrammatic notation used to represent the duplication of an input, inspired by [Dooms et al., 2025].

both pooling layers can be seen as a special convolutional layer. One where the parameters of the kernel are not learned, but rather fixed to uniform values to compute the average pooling. The architecture of the resulting tensor network is therefore the same, but the data stored in the tensors differ.

The average pooling after stage N implements the mapping

$$\left(1, \prod_{n=1}^N \lambda^n, u^N, u^N\right) \rightarrow \left(1, \prod_{n=1}^N \lambda^n, \text{AVG}(u^N), u^N\right)$$

with the average pooling operation denoted by AVG . Its equivalent tensor network is presented in figure 3.16. It has the same tensor network structure as the convolutional layer conversion, and tensors $\tilde{C}^H$ and $\tilde{C}^W$ are constructed identically to the convolutional case, using the pooling hyperparameters. However, tensor $\tilde{K}_{avg}$ is a different tensor now. Its role is to simulate the pooling layer as a convolutional operation. We can do this by setting its entries to $\frac{1}{K_h \cdot K_w}$ as this represents the weight for each element in the pooling window, which corresponds to the average of the entire kernel. There is, however, one important difference between average pooling and a standard convolutional layer, which is that pooling layers do not mix channels. Each channel is processed independently by the same kernel, which is a matrix. This is implemented by constructing $\tilde{K}_{avg}$ as follows by setting all

$$\tilde{K}_{avg}[c_{out}, c_{in}, k_h, k_w] = \begin{cases} \frac{1}{K_h \cdot K_w} & \text{if } c_{out} = c_{in} \\ 0 & \text{otherwise} \end{cases}$$

This construction forces channel C_{in} of the input to only contribute to channel C_{out} of the output, which matches the behaviour of a pooling layer. This time, we did not pay attention to a pass through mechanism. The reason for that is that when you apply an average pooling layer to a channel consisting of a scalar only, you get the same result because averaging over a constant returns that same constant. We implemented it this way because the pooling layer needs to downscale the spatial dimensions of the third-order input tensor. But that means that every single channel needs to be downscaled, not the channels containing u^N only, because otherwise they would have a different size.

The global average pooling after stage N implements the mapping

$$\left(1, \prod_{n=1}^N \lambda^n, u^N, u^N\right) \rightarrow \left(1, \prod_{n=1}^N \lambda^n, \text{GAVG}(u^N), u^N\right)$$

with the global average pooling operation denoted by GAVG . The global average pooling tensor network uses the same construction as the average pooling tensor network presented in figure

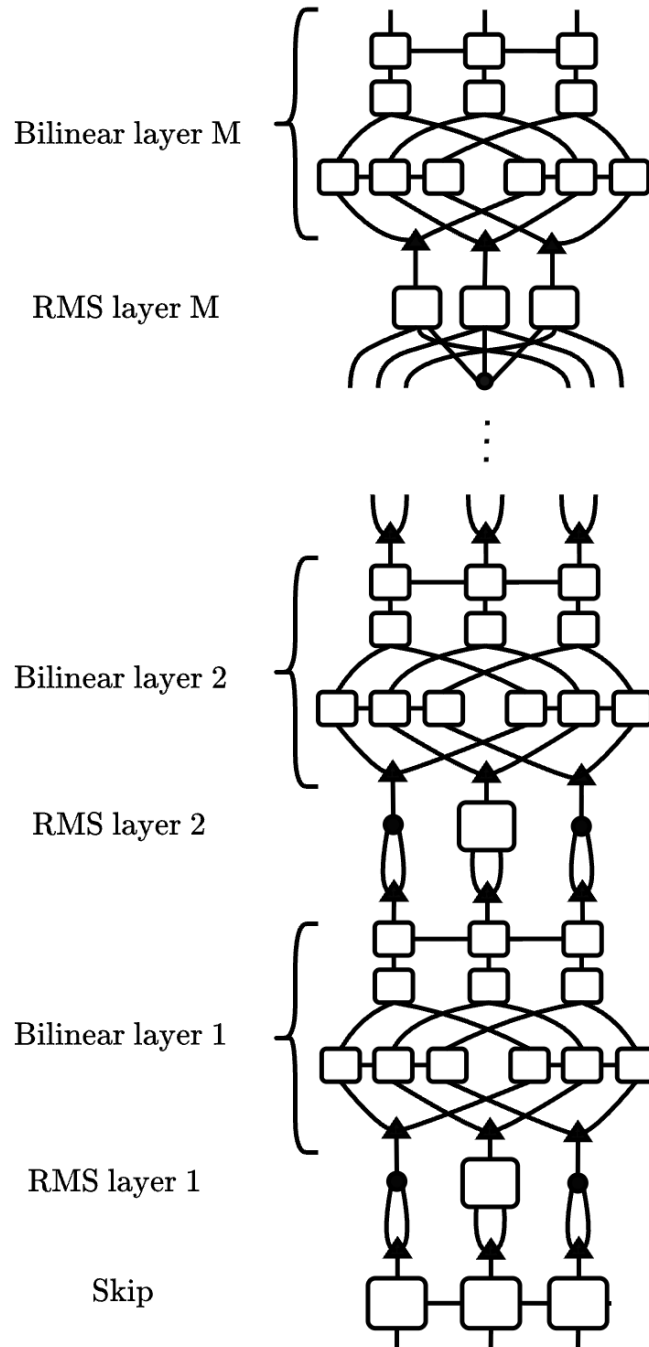


Figure 3.14: Tensor network representation of a residual block with M layers, unrolled to show the alternating RMS normalisation and bilinear layers tensor networks explicitly with a skip connection at the start. Each layer was converted to its equivalent tensor network and chained to its preceding tensor network layer equivalent.

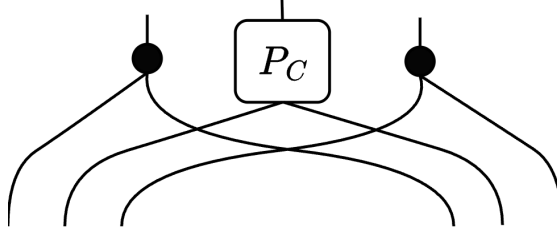


Figure 3.15: Tensor network equivalent of the residual connection mechanism. Its role is to represent a weighted sum consisting of the skip connection and the chain of a residual block. The weights are the accumulated RMS scalars and the RMS scalar from the last RMS normalisation layer stored in the input. The spiders ensure that spatial positions are matched with each other, while the third-order tensor P_C encodes the weighted sum logic.

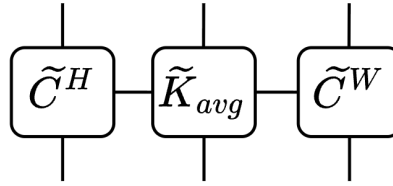


Figure 3.16: Tensor network equivalent of the average pooling layer. The tensors $\tilde{C}^H$ and $\tilde{C}^W$ are responsible for simulating a convolution operation adapted to represent an average pooling operation, and the tensor $\tilde{K}_{avg}$ contains the parameters for that.

figure 3.17. The only difference is that the kernel size is set to the full spatial dimensions of the input. This means that $K_h = H$ and $K_w = W$. This reduces each channel to a single value representing the average over all positions.

3.2.8 Flatten

The flatten operation implements the mapping

$$\left(1, \prod_{n=1}^N \lambda^n, x', x\right) \rightarrow \left[\prod_{n=1}^N \lambda^n, \text{flatten}(x')\right]$$

with x' and x arbitrary data. This operation maps a third-order tensor to a vector, hence the square brackets notation to denote a flat vector. Three things happen simultaneously in this

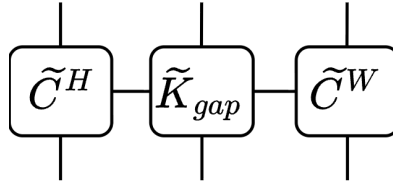


Figure 3.17: Tensor network equivalent of the global average pooling layer. The tensors $\tilde{C}^H$ and $\tilde{C}^W$ are responsible for simulating a convolution operation adapted to represent a global average pooling operation, and the tensor $\tilde{K}_{gap}$ contains the parameters for that.

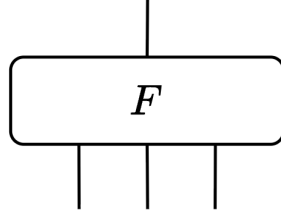


Figure 3.18: Tensor network equivalent of the flatten operation. The single tensor simultaneously extracts the accumulated RMS scalar, flattens the data channels, and discards the constant channel and the second data part.

mapping. First, the constant channel is dropped entirely. Second, the accumulated RMS scalar $\prod_{n=1}^N \lambda^n$ that was stored as a spatially uniform channel (a matrix) is extracted as a single scalar value. It is placed in the first position of the output vector. Third, the data x' is flattened and placed after the RMS scalar in the output vector. The second data x is discarded since it was reserved for residual connections, but we do not have any anymore.

The tensor network equivalent of the flatten operation is presented in figure 3.18. It consists of a single fourth-order tensor F with shape $F_{out} \times C_{in} \times H_{in} \times W_{in}$, under the assumption that the input of the mapping has shape $C_{in} \times H_{in} \times W_{in}$ and that x' has shape $C_{x'} \times H_{x'} \times W_{x'}$, where $F_{out} = 1 + C_{x'} \cdot H_{x'} \cdot W_{x'}$ is the length of the vector. The entries of this tensor are defined as

$$F[f, c, h, w] = \begin{cases} 1 & \text{if } f = 0, c = 1, h = 0, w = 0 \quad (c \in \text{RMS channel}) \\ 1 & \text{if } f = 1 + (c - 2) \cdot H_{in} \cdot W_{in} + h \cdot W_{in} + w \quad (c \in \text{data channels}) \\ 0 & \text{otherwise} \end{cases}$$

where $c \geq 2$ for data channels. The first case extracts the RMS scalar from channel 1 at position $(0, 0)$. The second case implements the flattening of data channels, mapping each (c, h, w) to its corresponding flat index. All other entries are zero and therefore do not contribute to the output.

3.2.9 Linear layer

The last linear layer implements the mapping

$$\left[\prod_{n=1}^N \lambda^n, x \right] \rightarrow \left[\prod_{n=1}^N \lambda^n, \text{linear}(x) \right]$$

where both the input and output are vectors. Its tensor network equivalent is responsible for letting the RMS scalar pass through, and applying the linear transformation on the data x . The tensor network is depicted in figure 3.19. It consists of a single tensor, which is a matrix L of shape $(1 + d_{out}) \times (1 + d_{in})$ with d_{in} the size of the input vector x and d_{out} the number of output classes in the classification task. This matrix F is constructed as

$$L = \begin{pmatrix} 1 & 0 \\ 0 & W \end{pmatrix}$$

with W the parameters of the linear layer organised as a matrix.

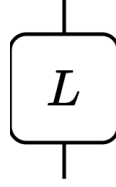


Figure 3.19: Tensor network equivalent of a linear layer. Second-order tensor L is responsible for simultaneously applying the linear layer to its input x , but also letting the RMS scalar Λ pass through in the input $[\Lambda, x]$ which represents a vector whose first entry is the scalar Λ and the remaining entries correspond to vector x .

3.2.10 Complete tensor network architecture

We have covered the conversion of all individual components of the proposed architecture. The last step is to present the final tensor network, where all tensor network components are combined with each other. This tensor network is illustrated in figure 3.20. A residual block tensor network is depicted as a box in the figure to keep the result compact and is equal to the tensor network presented in figure 3.14 with the additional tensor network component chained at the end from figure 3.15. The final tensor network chains those residual blocks with each other in stages, and every stage is interleaved with an average pooling tensor network equivalent. The final stage ends with an average pooling tensor network equivalent, followed by a global average pooling tensor network equivalent. The output is then flattened and passed through the last linear tensor network equivalent. This exactly matches the proposed architecture from figure 3.20, but with equivalent tensor network components.

3.2.11 Implementation

The conversion is implemented using PyTorch [Paszke et al., 2019] and the Quimb [Gray, 2018] library. This library is a Python-based library for quantum information many-body calculations, focusing primarily on tensor networks. Depending on how the result of the conversion is used, two different approaches exist for the conversion. For weight-based analysis, every layer is mapped onto an equivalent component that is chained to the existing network, possibly requiring duplication of existing tensors. If the goal is instead to use the result as a substitute for the model to evaluate its output on a given input, constructing the full tensor network is neither necessary nor efficient. Components with two branches require duplication, and repeating this across multiple layers produces a tree whose width grows exponentially, making a full contraction intractable. The solution is to apply the contraction stepwise. Rather than constructing the full tensor network, we proceed layer by layer. We convert one layer at a time, contract it with the current input, and feed the result to the next component. One practical consideration when evaluating the network stepwise is numerical instability. The bilinear layer is quadratic in its input, as illustrated by equation (3.2). This means that input values above 1 grow, and values below 1 shrink. For models with multiple bilinear layers, intermediate outputs can therefore vanish or explode before reaching the final division by the accumulated RMS scalar. We handle this by performing the division after every residual block contraction rather than delaying it until the end. After each division, the accumulated RMS scalar is reset to 1 since it has already been used at that step.

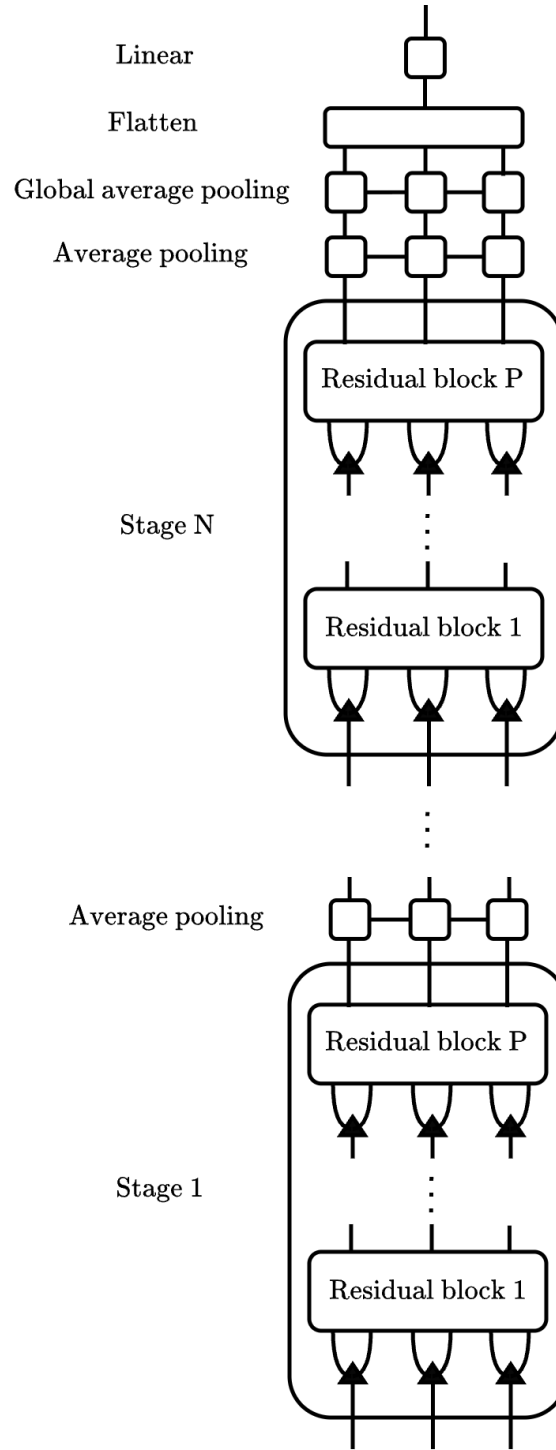


Figure 3.20: Tensor network equivalent of the proposed architecture. Each stage consists of P tensor network residual blocks chained sequentially, with average pooling tensor networks between stages. After the final stage, the input is processed by an average pooling tensor network, a global average pooling tensor network, a flatten tensor network, and finally a linear tensor network. The residual block tensor networks are shown as boxes for visual clarity.

3.3 Tensor network analysis

We now have a technique for generating a tensor network equivalent to the previously proposed convolutional neural network architecture. In this section, we turn our attention to how to analyse it and what insights this analysis provides.

3.3.1 Spectral decomposition

To analyse the tensor network, we generate and analyse its spectral decomposition. Depending on whether the tensor network can be contracted to a single tensor tractably, there exist two different methods to generate this decomposition. If the tensor network can be contracted to a single tensor, the work of [Pearce et al., 2025] can be used. It describes how a tensor can be analysed using its spectral decomposition. If the tensor network cannot be contracted because it is intractable to do so, the work of [Dooms et al., 2025] proposes the ODT algorithm. This algorithm analyses the spectrum of the entire tensor network. In our work, we consider the first case since the convolutional model based on our proposed architecture we analyse is small and shallow; hence, it is affordable to contract its equivalent tensor network into a single tensor.

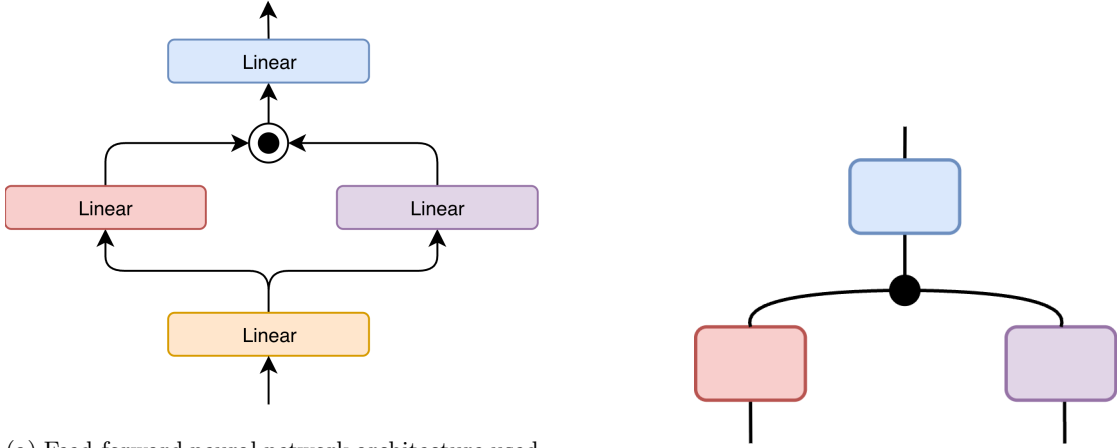
The work of [Pearce et al., 2025] analyses a bilinear feed-forward neural network presented in figure 3.21a. The model consists of an embedding linear layer, followed by a bilinear linear layer, followed by a linear classification that produces one output per class. This model is analysed by first converting it to an equivalent tensor network illustrated in figure 3.21b, which is then contracted to a single tensor. Since all layers within the model are linear, each layer is represented using a matrix, and the corresponding tensor network is simply a composition of these matrices. Note that the embedding layer is not included in the tensor network for reasons we explain in subsection 3.3.3. Once the tensor network is constructed, we can analyse it using the eigendecomposition techniques from linear algebra. To use these techniques, we need a matrix. To that end, the tensor network is contracted to a third-order tensor $\mathcal{B}$ of shape output $\times$ input $\times$ input. To analyse how a model makes a prediction for a particular output direction u , we generate its corresponding interaction matrix Q . This matrix is constructed by contracting $\mathcal{B}$ on its output model with a one-hot vector representing the output direction u . This matrix represents how the inputs interact with each other to produce the output for that class u . Since it is symmetric as described in [Pearce et al., 2025], the spectral theorem gives an eigendecomposition of Q as

$$Q = \sum_i^d \lambda_i v_i v_i^T$$

With this eigendecomposition, the output o_u for class u can be computed by

$$o_u = x^T Q x = \sum_i^d \lambda_i (v_i^T x)^2 \quad (3.10)$$

with d orthonormal eigenvectors v_i , eigenvalues λ_i , and input x [Pearce et al., 2025]. The eigenvectors and eigenvalues generated from the eigenspectrum analysis of the weights of the model for an output class u can be used to yield weight-based insights into the model. Their interpretation in the input space make the model interpretable for humans. We describe this in the next section.



(a) Feed-forward neural network architecture used in [Pearce et al., 2025]. This feed-forward neural network consists of an embedding layer, followed by a bilinear multi-layer perceptron. The output of that layer is then passed to a classification head, which produces one value for each output class.

(b) Bilinear multi-layer perceptron tensor network. This tensor network is equivalent to the bilinear multi-layer perceptron component of the feed-forward neural network.

Figure 3.21: Overview of the feed-forward neural network used in [Pearce et al., 2025] and the equivalent bilinear multi-layer perceptron tensor network. The colours indicate the corresponding mapped components from each layer.

3.3.2 Eigenspectrum interpretation

Equation (3.10) is the core of this weight-based interpretability tool. It allows us to interpret and link the eigenvectors and eigenvalues to our original image digit classification task. The output of a class u is computed as a weighted sum with eigenvalues as weights and square dot products of the corresponding eigenvector with some input x as terms, as demonstrated by equation (3.10). Since each term is squared, the sign and magnitude of the eigenvalue λ_i determine how the term $(v_i^T x)^2$ contributes to the final output of the class u . Terms with positive eigenvalues increase the output of the class, while terms with negative eigenvalues decrease that output. Since the output of the class determines the prediction of the model, the eigenvectors and the terms $(v_i^T x)^2$ can directly be used for interpretability. They allow us to understand how a model constructs its prediction. The components of the weighted sum with a large positive eigenvalue λ_i and large $(v_i^T x)^2$ are components that encourage the prediction of the class u , while components with a large negative eigenvalue λ_i and large $(v_i^T x)^2$ are components that discourage the prediction of class u by the model. Components with eigenvalues close to 0 have a small contribution. This insight allows us to link the eigenvalues and eigenvectors to the prediction of the model for class u . By looking at the sign and the magnitude of an eigenvalue, we know whether the component supports or suppresses the class u . Furthermore, the corresponding eigenvector v_i tells us how that component supports or suppresses that class. For the final component $\lambda_i(v_i^T x)^2$ to be large, both a large positive eigenvalue λ_i and $(v_i^T x)^2$ are needed. This squared dot product is high if and only if the eigenvector v_i aligns with the input x . If that is the case, the two vectors are aligned, and hence we can use that eigenvector as an interpretation to understand why it supports the prediction. The eigenvector can be thought of as a prototype for the class u in the input space, as it represents the pattern that the model associates strongly with that class u . We can do the same

analysis for components with a large negative eigenvalue. However, these are eigenvectors that should be used to interpret why it suppresses the prediction for class u , and act as anti-prototypes for that class. With these eigenvectors, we can now interpret what detected patterns in an input encourage and discourage a model to make its prediction. If we visualise these eigenvectors in the input space, which correspond to the detected patterns, we have an interpretation for class u not tied to a fixed input and solely based on the eigenspectrum analysis of the weights of the model. This is precisely what makes the technique weight-based; no additional data is required, and we analyse the weights of the model. The eigenvectors describe the pattern the model looks for, and this is independent of any input because equation (3.10) works for any input x . This analysis is not tied to a single input x , but for instance-based explainability, we can fix the input x to some input sample. This analysis reveals which eigenvectors contribute the most for that specific input, and hence explains how the model reached its conclusion for that specific input only.

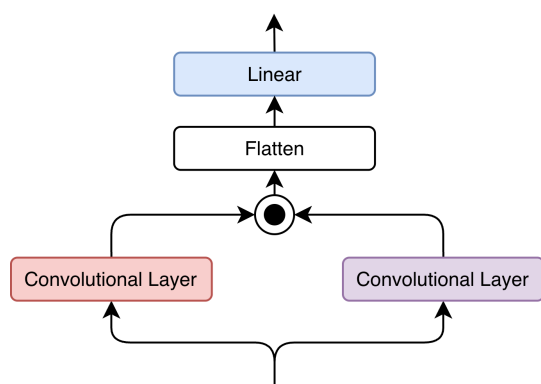
Since the eigenvectors reveal the detected patterns within an image, it would be ideal to visualise them. However, in the case of the presented feed-forward neural network, the eigenvectors are in the hidden space because of the embedding layer. Therefore, we cannot visualise them directly in the input space. We first have to convert them back into the input space by using the embedding weights. These resulting vectors can then be visualised as images and reveal which patterns support or suppress the prediction for a class u .

3.3.3 Convolutional neural network extension

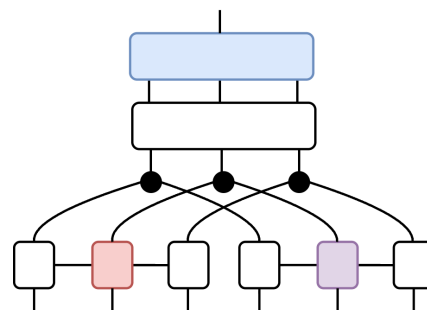
We extend the work of [Pearce et al., 2025] to convolutional neural networks by applying the spectral decomposition to a shallow CNN trained on the same digit classification task, and comparing our results with the spectral decomposition of a bilinear feed-forward neural network presented in the same work. Figure 3.22a presents the CNN we analyse. It consists of a bilinear convolutional layer, followed by a flatten operation, and a linear layer. Unlike the feed-forward model, no embedding layer is used here. The reason is that the feed-forward model needs the embedding layer to project the high-dimensional input, the flattened image of 784 dimensions for MNIST³, into a smaller hidden space before the bilinear layer operates on it. A bilinear layer operating directly on 784-dimensional inputs would require a large number of parameters, as motivated in subsection 2.3.4. Our convolutional neural network does not suffer from this problem because of its reusable kernels.

Figure 3.22b presents the tensor network equivalent of the bilinear CNN presented in figure 3.22a, which is constructed using the same techniques as described in the previous section. However, since this model uses no normalisation or residual connections, the tensor network does not need to track any additional factors. It works with input of the form x rather than $(1, \Lambda, x, y)$ as we presented before. Contracting this tensor network yields a seventh-order tensor of shape $\text{output} \times C_{in} \times H_{in} \times W_{in} \times C_{in} \times H_{in} \times W_{in}$. To apply the same presented spectral decomposition, we select one output direction u , which reduces the tensor to a sixth-order tensor of shape $C_{in} \times H_{in} \times W_{in} \times C_{in} \times H_{in} \times W_{in}$. Unlike the feed-forward case, this does not generate a matrix because the input of a CNN is not flattened. So we first have to reshape this tensor by grouping the two sets of input indices. This yields a matrix of shape $(C_{in} \cdot H_{in} \cdot W_{in}) \times (C_{in} \cdot H_{in} \cdot W_{in})$. This interaction matrix can be analysed using the same spectral decomposition technique presented in subsection 3.3.1. However, since we do not have any embedding layer, the eigenvectors can be directly visualised as they are present in the input space.

³MNIST consists of $1 \times 28 \times 28$ images



(a) Convolutional neural network illustration. This CNN consists of a bilinear convolutional layer, followed by a flatten operation and a linear layer to produce a value for each output class.



(b) Convolutional tensor network illustration. This tensor network is equivalent to the convolutional neural network presented in figure 3.22a. It is constructed using the principles introduced in section 3.2.

Figure 3.22: Overview of the convolutional neural network and its equivalent tensor network used in our analysis. The colours indicate the corresponding mapped components from each layer.

Chapter 4

Results

4.0.1 Overview

This chapter presents the results of our experiments with the goal of answering our research question. The interpretation of these results is discussed in chapter 5. First, we evaluate how the proposed architecture compares to state-of-the-art CNN models on standard benchmarking datasets. We then study the conversion in greater detail by analysing the conversion accuracy. We study how the output of the CNN model and the contraction of its equivalent tensor network with the same input compare to each other. Despite the conversion being mathematically exact, some differences may occur in both outputs due to finite floating-point operations, so we quantify this difference. Finally, we present the results of the interpretability analysis of the bilinear convolutional neural network introduced in subsection 3.3.3 and compare our results with the bilinear feed-forward neural network analysis of [Pearce et al., 2025].

4.1 Model performance

One of the requirements of the proposed CNN architecture was that its performance must be comparable to state-of-the-art CNN models so that it can be used as a credible interpretable substitute. However, we do not aim to achieve marginally better performance since our focus is on interpretability. Based on some literature review, we selected state-of-the-art pure CNN models for image classification on the standard benchmarking datasets introduced previously. These models are ResNet [He et al., 2016], DenseNet [Huang et al., 2017], Wide ResNet (WRN) [Zagoruyko and Komodakis, 2017], Deep pyramidal residual networks [Han et al., 2017], and an ensemble of CNNs [An et al., 2020] for MNIST only. To also situate the proposed architecture within the broader state-of-the-art, we briefly compare it against the transformer-based architecture ViT-H/14 [Dosovitskiy et al., 2020]. The results are summarised in table 4.1. This table shows the classification error rates for each model on the four benchmarking datasets, along with the number of parameters the model contains. The reported error rates for our model are computed on the official test set of each dataset. For hyperparameter selection, for each dataset, the official training set was split into a training subset (80%) and a validation subset (20%) using a fixed random seed. The final models reported in table 4.1 are retrained on the full training set using the selected hyperparameters. We tested four hyperparameter configurations, one per dataset, selected by best validation accuracy. Due to limited compute and a large hyperparameter space, our tuning was constrained; the reported results reflect the strongest configuration found, not necessarily the optimal one. We expect further gains to be achievable with compara-

ble effort to that invested in the benchmark models. For instance, we fixed the kernel size across all bilinear layers and assumed equal numbers of residual blocks per stage. The results reported for the benchmark models come from their respective papers.

On the digit classification task (MNIST), the proposed architecture achieves an error rate of 0.56% with 658K parameters. The ensemble of CNNs from [An et al., 2020] reports the best result with an error rate of 0.09%. However, comparing the two with each other is not fair because an ensemble works with multiple architectures, while we work with one. This result still allows us to situate our model within a broader state-of-the-art context. Unfortunately, other state-of-the-art models listed in the table do not report any results on MNIST.

On the object recognition task with 10 classes (CIFAR-10), the proposed architecture achieves an error rate of 4.73% with 25M parameters. This is better than all ResNet variants. Among models with a similar parameter count, it has a somewhat higher error rate than the strongest architectures. The transformer-based architecture is significantly better.

On the object recognition task with 100 classes (CIFAR-100), the proposed architecture achieves an error rate of 27.71% with 25M parameters, which is comparable to ResNet-110. However, on the other benchmark models, our model is behind.

On the house number recognition task (SVHN), the proposed architecture achieves a classification error rate of 1.58% with 25M parameters. It performs comparably to other state-of-the-art models. However, those results should be taken cautiously since not all models report results for SVHN.

4.2 Tensor network conversion accuracy

In section 3.2, we explained how to convert the proposed CNN architecture to an equivalent tensor network. We now assess how well this conversion scales by measuring the conversion accuracy, defined as the mean absolute error between the raw logit outputs of the model and the output of the stepwise tensor contraction given the same input. This allows us to verify whether the conversion remains accurate for deeper models. While the conversion is exact in theory, working with finite floating-point numbers introduces small numerical differences. This experiment characterises how these differences scale with model size, as they may grow with the number of operations involved. The scalability is characterised by three hyperparameters of the proposed architecture, which are the number of stages, the number of residual blocks per stage, and the repeat factor of a residual block. All other hyperparameters are fixed throughout the experiment and are listed in table 4.2.

The experiment is set up as follows. For three different repeat factors, we create models according to the configurations of table 4.2. We randomly initialise a model with those hyperparameters and convert it to an equivalent tensor network; no training is performed. For a given random input in the range $[0, 1]$, we compute the output of the model and the stepwise contraction of that input with the equivalent tensor network. Both outputs are compared with each other, and the absolute error is computed to measure the accuracy of the conversion. This is repeated for 10 random inputs per configuration, and the absolute error is averaged across these inputs. The results are presented in figure 4.1. It shows three heatmaps corresponding to each of the different repeat factor values we experimented with. The horizontal axis of each heatmap shows the number of blocks per stage, and its vertical axis shows the number of stages. The scale between all heatmaps is logarithmic and is shared across the three heatmaps to allow comparisons between heatmaps. A bright colour indicates a higher mean absolute error.

For each repeat factor, the error increases with model size since the colours become brighter as the number of stages and blocks per stage grows. This error growth is larger for larger repeat

Table 4.1: Classification error rates (%) on MNIST, CIFAR-10 (C10), CIFAR-100 (C100), and SVHN. Results in bold indicate the best performance for that dataset, and results reported with an asterisk * indicate results reported with no data augmentation. The symbol - indicates that the model reports no results for that dataset. All results were taken from their original papers, except for the results of our proposed architecture.

Model	#params	MNIST	C10	C100	SVHN
Ours using hyperparameter table B.1	658K	0.56*	-	-	-
Ours using hyperparameter table B.3	25M	-	4.73	-	-
Ours using hyperparameter table B.4	25M	-	-	27.71	-
Ours using hyperparameter table B.2	25M	-	-	-	1.58*
Ensemble of simple CNNs	N/A	0.09*	-	-	-
ResNet-20	270K	-	8.75	-	-
ResNet-56	850K	-	6.97	-	-
ResNet-110	1.7M	-	6.41	27.22	2.01*
ResNet-1202	19.4M	-	7.93	-	-
DenseNet (depth = 40, k = 12)	1M	-	5.24	24.42	1.79*
DenseNet (depth = 100, k = 12)	7M	-	4.10	20.20	1.67*
DenseNet (depth = 100, k = 24)	27.2M	-	3.74	19.25	1.59*
DenseNet-BC (depth = 100, k = 12)	800K	-	4.51	22.27	1.76*
DenseNet-BC (depth = 250, k = 24)	15.3M	-	3.62	17.60	1.47*
DenseNet-BC (depth = 190, k = 40)	25.6M	-	3.46	17.18	-
WRN (depth = 40, k = 4)	8.9M	-	4.53	21.18	-
WRN (depth = 16, k = 8)	11M	-	4.27	20.43	-
WRN (depth = 28, k = 10)	36.5M	-	4.00	19.25	-
PyramidNet ($\alpha = 48$, depth = 1114)	27.0M	-	3.48	17.01	-
PyramidNet ($\alpha = 200$, depth = 272)	26.0M	-	3.31	16.35	-
ViT-H/14	632M	-	0.05	5.05	-

factors. Furthermore, the scale of the error values differs significantly across the three heatmaps. As the repeat factor increases, the overall error level increases too. This suggests that among the studied hyperparameters, the repeat factor has the strongest influence on the conversion accuracy of the conversion. Finally, the errors are, on average, of the order of 10^{-7} ; this corresponds to the machine ϵ for float32 arithmetic [IEEE, 2019].

4.3 Interpretability analysis

In this section, we present the results of the interpretability analysis on the MNIST dataset on both the feed-forward bilinear network, referred to as the linear model in this section, from [Pearce et al., 2025], and our (bilinear) convolutional model proposed in figure 3.22a. We use the linear model to compare the two analyses with each other.

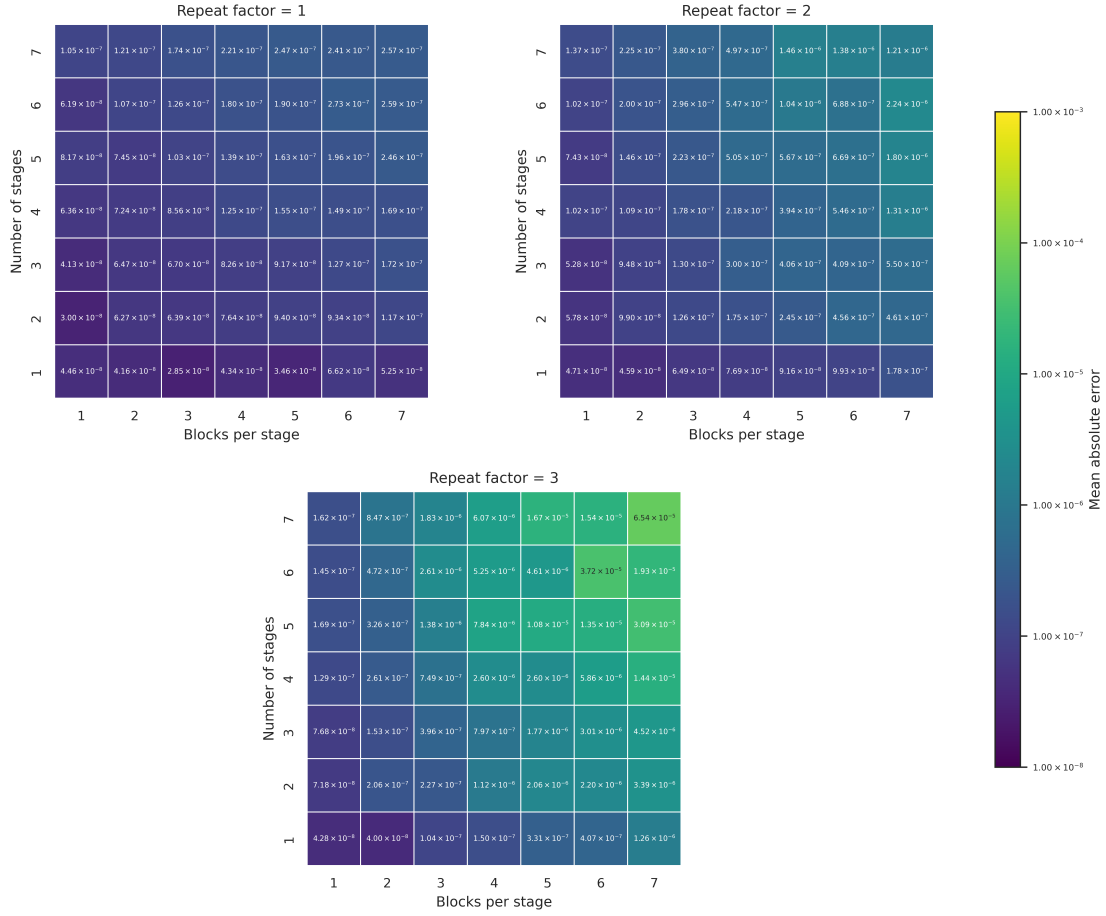


Figure 4.1: Heatmaps showing the accuracy for different hyperparameter configurations of the conversion of the proposed architecture. The horizontal axis of each heatmap shows the number of residual blocks per stage, and the vertical axis shows the number of stages of the architecture. The same logarithmic scale is used for all heatmaps.

Table 4.2: Hyperparameter configuration used to measure the scalability of the conversion, measured by the conversion accuracy.

Hyperparameter	Values
Input height	8
Input width	8
Input channels	3
Number of output classes	10
Repeat factor	1, 2, 3
Blocks per stage	1, 2, 3, 4, 5, 6, 7
Number of stages	1, 2, 3, 4, 5, 6, 7
Kernel size	3
Upscale factor	2
Channels factor	2

Table 4.3: Hyperparameter sweep configuration of the linear model taken from [Pearce et al., 2025].

Hyperparameter	Value
Epochs	40
Hidden dimension	512
Weight decay	1.0
Noise std. dev.	0.5
Optimiser	AdamW
Learning rate	0.001
Learning rate scheduler	Cosine scheduler

4.3.1 Hyperparameter configuration

Before performing the analysis, we need to choose hyperparameters for both models. We do not attempt to match the hyperparameters between the two models. Since the two models are fundamentally different in their architecture, and a convolutional neural network is more parameter-efficient than a linear model, matching the parameters would be misleading. Instead, we tuned each model separately.

For the linear model, the hyperparameters were taken directly from [Pearce et al., 2025] and are listed in table 4.3. We observed that changing the hyperparameters did not significantly affect the performance of the linear model, except that increasing the hidden dimension or lowering the regularisation resulted in overfitting. We obtained these observations by reproducing the results of the paper with its publicly available code¹.

For the convolutional model, no prior configuration exists since this analysis is entirely new. Therefore, we ran a small hyperparameter sweep listed in table 4.4. With this sweep, we noticed that the classification accuracy of these models is not very sensitive to configurations that do not overfit. If the number of epochs is set too high, with low or no weight decay and low noise standard deviation, the model overfits. Based on this insight and the sweep, we selected 5 epochs, a hidden dimension of 256, a weight decay of 1.0, and a noise standard deviation of 0.25. We chose the highest number of hidden dimensions because it results in the highest number of parameters for the convolution model. This allows us to verify and show that the analysis remains interpretable even with a large number of parameters. The results of the performance of each configuration are listed in table 4.5.

4.3.2 Eigenspectrum analysis

With the hyperparameters set, we run the analysis on both the linear model and the convolutional model for different kernel sizes. We take one output direction u corresponding to digit 3 and analyse the eigenspectrum using the technique discussed in section 3.3. This digit is interesting because a badly written 3 can be confused with other digits such as 2, 8, or 9. With the analysis, we can see what the model looks at to make its prediction and not confuse the digit with another one. We discuss the results for digit 3, but the results for all other digits remain consistent and are illustrated in appendix C.

Figure 4.2 shows the results of the analysis for each model for the output class corresponding with digit 3. For each model, the figure contains two rows. The first row shows the four eigenvectors associated with the four highest positive eigenvalues, ordered from left to right by

¹<https://github.com/tdooms/bilinear-decomposition>

Table 4.4: Hyperparameter sweep configuration for the convolutional model. Bold values indicate the selected configuration.

Hyperparameter	Values
Padding	0
Stride	1
Epochs	5 , 10, 20
Hidden dimension	64, 128, 256
Weight decay	0, 0.1, 0.5, 1.0
Noise std. dev.	0, 0.25 , 0.5
Optimiser	AdamW
Learning rate	0.001
Learning rate scheduler	Cosine scheduler

Model	Training Accuracy (%)	Validation Accuracy (%)
Linear	96.8	98
Convolutional (K = 3)	97.6	97.6
Convolutional (K = 5)	98.5	98.2
Convolutional (K = 7)	98.6	98.4
Convolutional (K = 14)	98.4	98.4
Convolutional (K = 28)	93.1	94.1

Table 4.5: Training and validation accuracy for the different models.

decreasing eigenvalue. The second row shows the four eigenvectors associated with the four most negative eigenvalues, ordered the same way. Since each eigenvector resides in the input space, we can visualise them as pixels. Each eigenvector is visualised as an image where blue pixels indicate positive values and red pixels indicate negative values. As discussed in subsection 3.3.2, the eigenvectors in the first row support the prediction for digit 3, while the eigenvectors in the second row suppress it. Each row also contains a line plot on the left showing the eigenspectrum of the top 20 eigenvalues, ordered by magnitude. For rows showing eigenvectors associated with positive eigenvalues, these are the 20 highest eigenvalues. For the rows showing eigenvectors with negative eigenvalues, these are the 20 most negative eigenvalues.

The results are shown for the linear model first, followed by the convolutional model with increasing kernel size K . There are several observations we can make from these results. The first observation concerns the kernel size. For a small kernel size, the eigenvectors are localised. As the kernel size increases, the highlighted regions grow until being maximal for kernel size 28.

The second observation concerns the eigenvalues. As the kernel size increases, the eigenvalue distribution becomes steeper. This indicates that fewer eigenvectors carry most of the information and patterns. For small kernel sizes, the information and patterns are distributed across more eigenvectors. There are more eigenvectors that contribute meaningfully to the output in those settings.

The third observation concerns the interpretability of the eigenvectors themselves. For the convolutional model with kernel size 3, the positive eigenvectors correspond with horizontal strokes of the digit 3, while the negative eigenvectors correspond to local patterns inconsistent with that digit. For larger kernel sizes, the positive eigenvectors correspond with the digit 3 itself, while the negative eigenvectors correspond with general patterns inconsistent with that

digit. Compared to the linear model, the eigenvectors of the convolutional model are concentrated in compact regions that are coherent with the digit 3. The eigenvectors of the linear model are less concentrated and more spread out across the image. This is illustrated by the eigenvectors with positive eigenvalues of that linear model; there are some blue and red pixels outside the center of the image where the digit should normally be written.

4.3.3 Instance-based explainability

So far, the analysis has provided a general interpretation of what the model looks for when predicting a particular class u , and this is independent of any input x . Now we turn to instance-based explanations: we fix the input x to a specific test sample to understand how the model reaches a prediction for that particular input. The results for the linear model are shown in figure 4.3a and for the convolutional model with kernel size $K = 14$ in figure 4.3b. The hyperparameters of these models are the same as in the previous analysis. The test input is shown in the top right corner. In this case, the test input is the digit 5. Below that image, the raw logits are shown for each of the 10 classes. These correspond to the output of the model given the fixed input x . The three classes with the highest logits are highlighted, and for each of these, the figure shows the most important positive eigenvectors in the top row and the most important negative eigenvectors in the bottom row, ordered numerically. The line plots on the left show the ten largest eigenvalue magnitudes for each of the top three classes.

Both models correctly predict the output class and produce the same top three. The differences are in the eigenvectors and eigenvalues. The eigenvalues of the convolutional model are larger in magnitude, and their evolution is steeper than that of the linear model. This indicates that the eigenvectors with the largest positive and negative eigenvalues contribute more strongly to the output than the ones from the linear model, meaning that the convolutional model has fewer eigenvectors carrying most of the information. These positive eigenvectors are also more localised and compact. They show patterns clearly consistent with digits 3, 5, and 9. The positive eigenvectors of the linear model also show patterns consistent with those digits, but with activations spread across the image and less localised. The negative eigenvectors of both models do not coincide with each other at all. This suggests that both models use other patterns to suppress their predictions for a particular class.

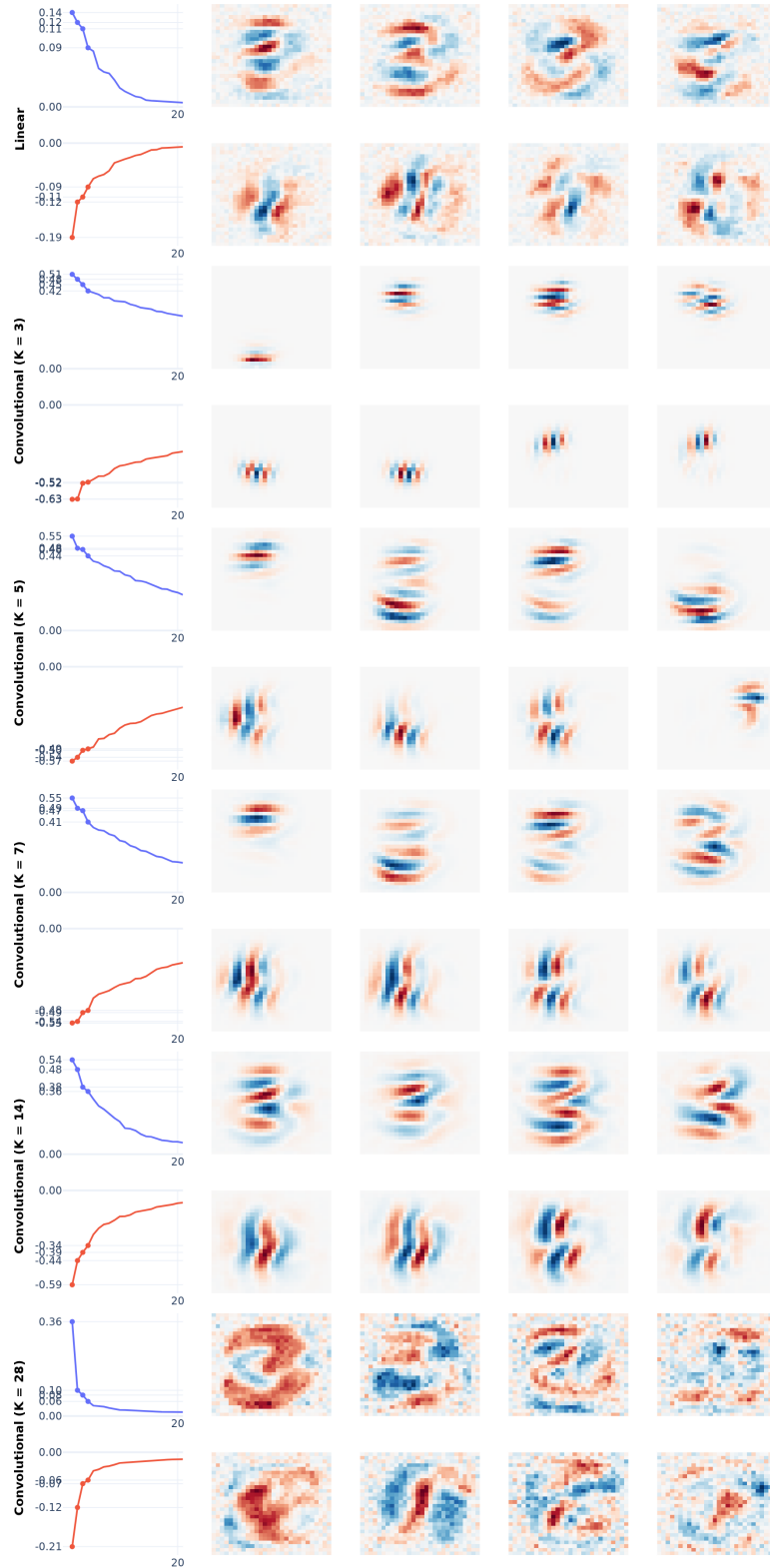
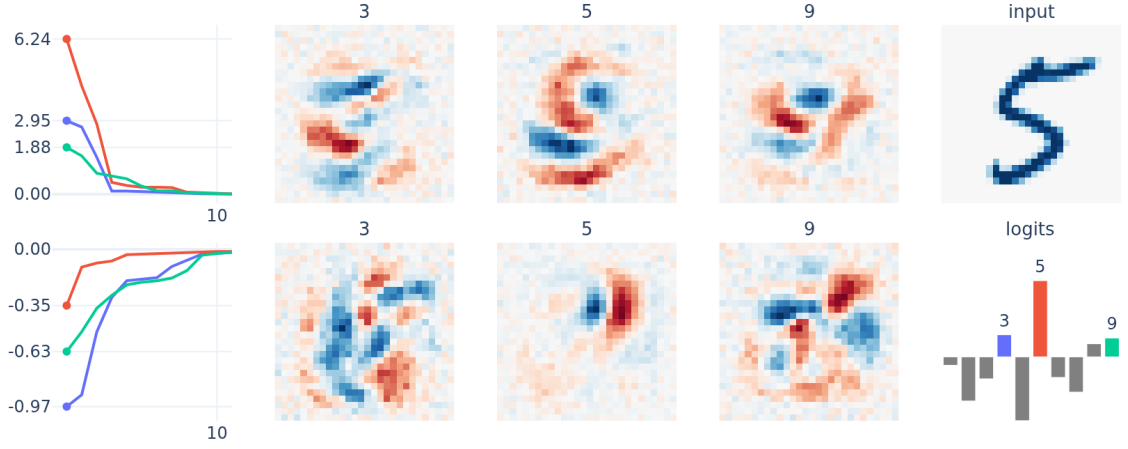
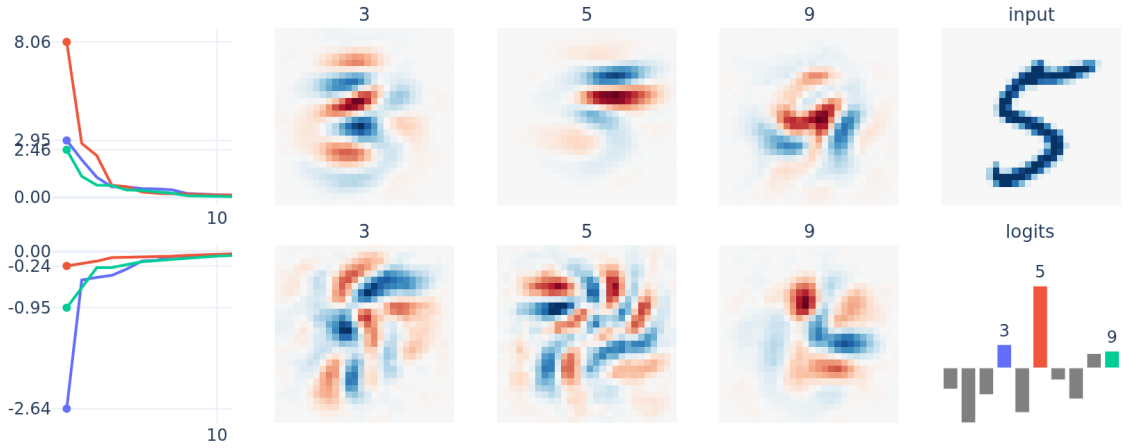


Figure 4.2: Analysis for output class digit 3 on MNIST for a linear model and convolutional models with increasing kernel size K . For each model, the top row shows the four most important positive eigenvectors, and the bottom row shows the top four most important negative eigenvectors, both ordered by decreasing eigenvalue magnitude. Blue pixels indicate positive values and red pixels indicate negative values. The line plot of each row shows the sorted eigenvalue magnitudes, with only the twenty largest displayed.



(a) Instance-based explainability for the linear model on a test sample of digit 5. The top right shows the input image and the logits for each class, with the three top classes highlighted. For each of these three classes, the top row shows the eigenvector with the largest positive eigenvalue, and the bottom row shows the eigenvector with the largest negative eigenvalue. The plots on the left show the ten largest eigenvalue magnitudes for each class. Blue pixels indicate positive values and red pixels indicate negative values in the eigenvector.



(b) Instance-based explainability for the convolutional model with kernel size $K = 14$ on a test sample of digit 5. The top right shows the input image and the logits for each class, with the three top classes highlighted. For each of these three classes, the top row shows the eigenvector with the largest positive eigenvalue, and the bottom row shows the eigenvector with the largest negative eigenvalue. The plots on the left show the ten largest eigenvalue magnitudes for each class. Blue pixels indicate positive values and red pixels indicate negative values in the eigenvector.

Figure 4.3: Instance-based explainability on a test sample of digit 5 for the linear model (top) and the convolutional model with kernel size $K = 14$ (bottom).

Chapter 5

Discussion

5.0.1 Overview

This chapter interprets the results from chapter 4 with the goal of answering our research question. Recall that our research question consists of three main components. The first is that the proposed convolutional neural network must be convertible to an equivalent tensor network. The second is that we want to extract interpretable weight-based insights from the tensor network representation of a shallow and an adapted instantiation of the proposed architecture. Lastly, we want the architecture to achieve competitive performance, measured by state-of-the-art benchmarking datasets on image classification tasks. If that is the case, we can use the proposed architecture as a credible interpretable substitute for existing CNN models on image classification tasks.

5.1 Model performance

In this section, we interpret the model performance results from section 4.1 with the aim of addressing the last part of the research question. Based on the results and observations of table 4.1, we conclude that the proposed architecture performs comparably to state-of-the-art CNN models on MNIST, CIFAR-10, and SVHN. These are datasets with a relatively small number of classes. The transformer-based architecture performs significantly better than our proposed model, but that comparison is not entirely meaningful given the fundamental architectural differences between the two models. Nevertheless, it allows us to situate our results in a broader context.

On CIFAR-100, which has ten times more output classes than the three other datasets, the proposed model’s performance is comparable to ResNet-110, but behind the other reported architectures. This suggests that the model struggles when the number of classes is large, and the amount of training data per class is smaller. We can think of two possible explanations for why that is the case. First, the hyperparameter space was not exhaustively explored. For example, the reported results from our architecture use a fixed kernel size across all convolutional layers of the bilinear layers. State-of-the-art models like ResNet use a different kernel size for the first layer and then a fixed kernel size for subsequent layers, which gives more flexibility to the model to extract features early. Another simplification in our configurations is that the number of residual blocks is fixed across all stages. A more careful tuning of the hyperparameters of the model might close the gap. Second, the global RMS normalisation might not be sufficient to stabilise the activations during training. The global RMS is computed as the average of

all values in the output of a bilinear layer. This means that if all outputs are large or small across all channels, computing this global average can cause numerical instability, resulting in unstable training. A channel-wise RMS normalisation could help here, and one of our additional experiments confirms this. Switching the model to use channel-wise normalisation yields better performance, as it reduces the gap to 19.58%. The hyperparameters of that configuration are listed in table B.5. The configuration is the same as the global normalisation configuration in table B.4 except that the normalisation is done channel-wise. However, we cannot use channel-wise normalisation as it would break the convertibility requirement. Lastly, the results should also be situated in the context of what the proposed architecture is meant to be used for: weight-based interpretability. The proposed architecture is constrained due to the convertibility requirement. The other state-of-the-art CNNs did not have this restriction and were designed with the goal of beating benchmarks.

Despite all constraints, the proposed architecture still achieves competitive performance on three out of four benchmarking datasets. No other model listed in the results table can be used as an interpretable substitute, while our model can because it is convertible to an equivalent tensor network. This is a fundamental trade-off; we trade some performance for interpretability. We believe that for the datasets with a small number of classes, our model can be used as an interpretable substitute because the model performs comparably. For CIFAR-100, the current implementation falls too far behind to be a competitive substitute, though it remains interpretable, and the gap likely reflects our limited tuning.

5.2 Tensor network conversion accuracy

The results of section 4.2 show that the conversion accuracy is high across all tested configurations. The largest mean absolute error observed is 6.54×10^{-5} , which is reported for the largest model configuration. This error is almost negligible and is unlikely to affect the predicted class prediction in practice. The results also show that error increases as the model grows in size, which is expected since larger maps to tensor networks with more components and hence more operations. Since each tensor network component contributes to the accumulated error, it accumulates and results in a larger error. The strongest factor influencing the conversion accuracy is the repeat factor. This was expected because a larger repeat factor means more alternations of RMS normalisation and bilinear layers before the normalisation is done at the end of the residual block in the stepwise contraction. With a high repeat factor, the intermediate outputs within the chain of alternations have more opportunity to vanish before being normalised. This leads to greater numerical instability in the intermediate computations.

This experiment was conducted purely to evaluate the accuracy of the conversion. This small error appears only when the tensor network is contracted with some input; otherwise the network on its own is mathematically equivalent to the proposed CNN architecture. If such a small error is not acceptable, the original model can simply be used directly for inference.

5.3 Weight-based interpretability

This section interprets the results from the weight-based analysis presented in section 4.3.

5.3.1 Eigenspectrum analysis

The eigenspectrum analysis yields meaningful insights into the internal workings of both models. For each model, the eigenvectors with the largest positive eigenvalues reveal the pixel patterns

the model uses to support its prediction for an output class, while the eigenvectors with the largest negative eigenvalues reveal the patterns that suppress it. The output class used in our results is the digit 3, so we interpret those results specifically for the digit 3, but they can be generalised to other digits.

For the linear model, the eigenvectors with positive eigenvalues show the patterns that effectively constitute a prototypical digit 3, along with some patterns that capture its borders. However, some of the activated pixels are spread across the image outside the central area where the digit is typically written. This is a consequence of how a dense model, like the linear one, works; it looks at all input pixels simultaneously when making a decision. The eigenvectors with negative eigenvalues show patterns that are inconsistent with a digit 3, as it shows vertically oriented strokes. The eigenvalues decay steeply, meaning that only a few dominant patterns capture most of the information the model relies on to identify digit 3. According to the work of [Martin and Mahoney, 2021], who studies the eigenvalue spectrum of weights as an indicator of generalisation, heavy-tailed distributions are associated with good generalisation while rapidly decaying distributions are associated with overfitting. The steep decay in eigenvalues observed here suggests that the model is overfitting according to this work. This is consistent with the fact that the model looks at pixels spread across the image. It has memorised a small number of specific patterns, and therefore overfits. This overfitting behaviour is not directly observable in table 4.5 as the validation accuracy is larger than the training accuracy.

The conventional model yields different insights, but that is expected since convolutional layers operate completely differently compared to a linear model. A convolutional layer does not look at the entire input at once, but it applies a kernel of fixed size that slides over the image. This kernel looks at local patches within the image. This is clearly reflected in the eigenspectrum analysis of the convolutional model. The eigenvectors are localised rather than being spread over the image. The effect of the kernel size K makes this particularly clear. For a small kernel size $K = 3$, the activated regions in each eigenvector are small and compact. The eigenvectors with positive eigenvalues show localised horizontal strokes that precisely correspond to the top stroke of a digit 3. The eigenvectors with negative eigenvalues show vertical strokes in locations where a digit 3 has none, which are the patterns that suppress that prediction. For larger kernel sizes $K = 5$, $K = 7$, $K = 14$, and $K = 28$, the kernel is large enough to capture nearly the entire digit. That is why the eigenvectors begin to resemble the digit 3, and do not represent its individual strokes anymore. The negative eigenvectors show larger vertical patterns covering a broader area. The eigenvalue spectrum itself also matches one property of convolutional layers. As the kernel size K increases, the eigenvalue evolution becomes steeper. This indicates that fewer eigenvectors are needed to capture the relevant information as the kernel size increases. This exactly matches the role of the kernel size. A convolutional model with a large kernel size captures the global shape in a single operation and can therefore rely on fewer patterns. While a convolutional layer with a small kernel size detects smaller patterns and hence needs to combine multiple patterns to identify a digit.

Some differences between the two models can be observed by comparing their eigenvectors. The linear model bases its prediction on pixels from the digit, but also on pixels spread across the rest of the image. This is a consequence of how a linear model works; it looks at all pixels simultaneously. Some of these activations outside the digit region may reflect spurious correlations in the training data rather than features of the digit. The convolutional model concentrates its patterns in compact regions coherent with the digit, which stems directly from how a convolutional model works. They are designed to look at local regions. When the convolutional model has kernel size $K = 28$, it shows a similar behaviour to the linear model. Its activated regions are spread across the image. That is expected since the kernel looks at every individual pixel of the image, and is effectively a reformulation of a linear layer. The eigenvalue spectra further

support this interpretation. The convolutional variants with small kernel sizes have heavy-tailed distributions. According to [Martin and Mahoney, 2021], this suggests that these variants generalise better since they learn to compose multiple low-level local patterns rather than relying on a small number of memorised prototypes. The linear model and the convolutional variant with $K = 14$ and $K = 28$ have a steeper spectrum, suggesting that they rely on fewer but more specific patterns that correspond to prototypes of digits.

5.3.2 Instance-based explainability

The eigenspectrum analysis for both models on a fixed specific test sample of a digit 5 yields conclusions consistent with the eigenspectrum analysis presented in the previous subsection. The convolutional model produces compact and locally focused eigenvectors, while the linear model has activated pixels more broadly across the image. Both models correctly predict the digit 5 and produce the same top three classes with similar logit values. This suggests that despite being architecturally different, both models reach the same conclusions. However, both models reach the same conclusion differently. The eigenvalues of the convolutional model are larger and decay more steeply. This is consistent with the earlier observation that larger kernel sizes concentrate information into fewer patterns. There is one interesting difference between the two models in their eigenvectors with negative eigenvalues. The patterns that suppress a given output class differ between the two models. This suggests that the two models use different pixels to suppress this class. This is likely because convolutional models are localised while linear models are global.

5.4 Limitations

The results and conclusions presented in this work come with several limitations that should be kept in mind. The first limitation concerns the model performance. We discussed that our proposed architecture performs comparably to state-of-the-art CNNs on MNIST, CIFAR-10, and SVHN. However, we observed that it underperforms on CIFAR-100 in comparison with the same state-of-the-art CNNs. In cases where interpretability is important and no loss in performance is allowed, this model falls short. Due to compute limitations, we did not conduct a full analysis of the hyperparameter space. This could yield important insights into the model’s behaviour and eventually increase its performance. The weight-based analysis was conducted on a small and shallow bilinear convolutional network. We did not run the analysis on the tensor network produced by our pipeline, so our analysis on a shallow model is not proof that a deep model is interpretable, although it shows promising results. Lastly, when the tensor network generated by our conversion pipeline is used as a direct substitute for the proposed architecture, we observed that the largest tensor network makes a mean absolute error of 6.54×10^{-5} . In cases where such a small error is not acceptable, the tensor network cannot be used as a substitute for the model. However, for the weight-based analysis we conduct, this is not a problem since we do not replace the model with its tensor network, but we analyse that tensor network without contracting it.

5.5 Future work

The work presented in this thesis and its limitations open opportunities to extend this work in several directions. We extended the weight-based analysis technique from [Pearce et al., 2025] that works for simple models containing one layer, but the models we have trained are far more complex than that. The ODT algorithm presented in [Dooms et al., 2025] could be used to analyse the eigenspectrum of the equivalent deep tensor network of our proposed architecture. Besides

being applied to deeper and more complex CNNs, it could also be applied to more complex datasets, such as CIFAR-100, to understand why the current model with the best performance is underperforming in comparison with the other models. Another interesting direction of research would be to push the proposed architecture to its limits by further exhaustive search of the hyperparameter space to try to close the performance gap on the CIFAR-100 dataset with state-of-the-art CNN models. If the architecture itself proves to be the limiting factor, the concepts and ideas developed in this thesis can be carried over to models built from other components that are convertible to tensor networks, opening up a broader design space to explore.

Chapter 6

Conclusion

This work investigated how a convolutional neural network can be designed to achieve competitive performance compared to state-of-the-art CNNs while remaining convertible to an equivalent tensor network, and what interpretable weight-based insights can be extracted from that tensor network representation for a shallow CNN model. This provides the foundations for deeper models.

To answer the first part of the research question, we proposed a multi-stage deep residual CNN architecture built entirely from components that are convertible to an equivalent tensor network. This requirement restricted the design space of the model drastically, preventing the use of several state-of-the-art CNN components. Therefore, the proposed architecture makes use of bilinear layers and global RMS normalisation inspired by [Pearce et al., 2025] and [Dooms et al., 2025]. Despite the architectural constraints, the proposed architecture achieves competitive performance on MNIST, CIFAR-10, and SVHN. This demonstrates that for relatively simple datasets with a small number of classes, the tensor network convertibility requirement is not a major obstacle for the performance of the model. However, on CIFAR-100, the observed performance gap is larger, which may indicate that a more extensive hyperparameter exploration is required, or that the design space is too limited to achieve state-of-the-art performance for more complex datasets.

The weight-based interpretability techniques of [Dooms et al., 2025; Pearce et al., 2025] rely on tensor networks, requiring a complete conversion pipeline for the proposed architecture. We therefore developed a complete tensor network conversion pipeline covering every component of the proposed multi-stage deep residual architecture. This includes convolutional, bilinear, RMS normalisation, pooling, flatten, and linear layers. Even if the conversion is exact in theory, in practice, due to finite floating-point arithmetic, some numerical differences exist between the model’s output and the contraction of the resulting tensor network with the same input. The conversion accuracy experiment demonstrated that the maximum mean absolute error is 6.54×10^{-5} across all tested configurations. This error is small in practice and unlikely to affect the predicted class.

Finally, we extended the weight-based analysis of [Pearce et al., 2025] to a shallow convolutional setting based on our architecture and compared the resulting patterns with those of a feed-forward model. The resulting patterns are visualised in the input space of images and linked to their digit classes. The analysis reveals that the convolutional architecture learns more structured and spatially localised patterns than the feed-forward neural network. The patterns are coherent with the digit classification and correspond to meaningful strokes or regions of digits. Convolutional models with a smaller kernel size learn patterns corresponding to strokes of the digits, while models with larger kernel sizes learn patterns corresponding to larger regions

of the digit. The linear model analysis reveals less localised and structured patterns. These differences suggest that the convolutional architecture is less influenced by spurious correlations in the training data. The analysis was also used for instance-based explainability and further demonstrated that weight-based analysis can explain individual predictions by revealing which patterns supported and suppressed each classification decision.

The results in this thesis demonstrated that it is possible to design a CNN that is convertible to a tensor network and that achieves competitive performance with state-of-the-art CNN models on MNIST, CIFAR-10, and SVHN. This work provides the foundation to analyse deeper and more complex architectures using the full tree tensor network technique from [Dooms et al., 2025], with the shallow case already revealing interpretable insights.

Bibliography

- Yaser S Abu-Mostafa, Malik Magdon-Ismail, and Hsuan-Tien Lin. *Learning from data*. AMLBook New York, January 2012. ISBN 978-1-60049-006-4.
- Sanghyeon An, Minjun Lee, Sanglee Park, Heerin Yang, and Jungmin So. An ensemble of simple convolutional neural network models for mnist digit recognition, August 2020. doi: 10.48550/arXiv.2008.10400.
- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization, July 2016. doi: 10.48550/arXiv.1607.06450.
- Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, March 1994. doi: 10.1109/72.279181.
- Leonard Bereska and Efstratios Gavves. Mechanistic interpretability for ai safety—a review. *Transactions on Machine Learning Research*, September 2024. url: <https://openreview.net/forum?id=ePUVetPKu6>.
- Saket Bhardwaj and Ajay Mittal. A survey on various edge detector techniques, February 2012. doi: 10.1016/j.protcy.2012.05.033.
- Jacob Biamonte and Ville Bergholm. Tensor Networks in a Nutshell, July 2017. doi: 10.48550/arXiv.1708.00006.
- Jacob C. Bridgeman and Christopher T. Chubb. Hand-waving and Interpretive Dance: An Introductory Course on Tensor Networks. *Journal of Physics A: Mathematical and Theoretical*, 50(22), May 2017. doi: 10.1088/1751-8121/aa6dc3.
- Andrzej Cichocki. Era of big data processing: A new approach via tensor networks and tensor decompositions. August 2014. doi: 10.48550/arXiv.1403.2048.
- Andrzej Cichocki, Danilo Mandic, Lieven De Lathauwer, Guoxu Zhou, Qibin Zhao, Cesar Caiafa, and Huy Anh Phan. Tensor decompositions for signal processing applications: From two-way to multiway component analysis. *IEEE Signal Processing Magazine*, 32(2):145–163, February 2015. doi: 10.1109/MSP.2013.2297439.
- Bob Coecke and Aleks Kissinger. *Picturing Quantum Processes: A First Course in Quantum Theory and Diagrammatic Reasoning*. First edition, March 2017. ISBN 978-1-316-21931-7 doi: 10.1017/9781316219317.
- Ekin D. Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V. Le. Randaugment: Practical automated data augmentation with a reduced search space. In *2020 IEEE/CVF Conference*

- on *Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 3008–3017, June 2020.
- R Dhiman, G Joshi, and C Rama Krishna. A deep learning approach for indian sign language gestures classification with different backgrounds. In *Journal of physics: conference series*, volume 1950, August 2021. doi: 10.1088/1742-6596/1950/1/012020.
- Thomas Doods, Ward Gauderis, Geraint A Wiggins, and José Oramas. Compositionality unlocks deep interpretable models. April 2025. doi: 10.48550/arXiv.2504.02667.
- Thomas Doods, Ward Gauderis, Geraint Wiggins, and Jose Oramas. Bilinear autoencoders find interpretable manifolds. May 2026. doi: 10.48550/arXiv.2605.08891.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. June 2020. doi: 10.48550/arXiv.2010.11929.
- Glen Evenbly. Gauge fixing, canonical forms, and optimal truncations in tensor networks with closed loops. *Physical Review B*, 98(8):085155, August 2018. doi: 10.1103/PhysRevB.98.085155.
- Glen Evenbly. A practical guide to the numerical implementation of tensor networks i: Contractions, decompositions, and gauge freedom. *Frontiers in Applied Mathematics and Statistics*, 8, June 2022. doi: 10.3389/fams.2022.806549.
- Ward Gauderis, Thomas Doods, Steven T Holmer, Kola Ayonrinde, and Geraint A Wiggins. From mechanistic to compositional interpretability, May 2026. doi: 10.48550/arXiv.2605.08934.
- ML Gonzalez, Melwina Albuquerque, Laurence Wroe, Jacob Meyer Cohen, Logan Riggs Smith, and Thomas Doods. When are two networks the same? tensor similarity for mechanistic interpretability, May 2026. doi: 10.48550/arXiv.2605.15183.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. url: <http://www.deeplearningbook.org>.
- Johnnie Gray. quimb: A python package for quantum information and many-body calculations. *Journal of Open Source Software*, June 2018. doi: 10.21105/joss.00819.
- Safa Hamreras, Sukhbinder Singh, and Román Orús. Tensorization is a powerful but underexplored tool for compression and interpretability of neural networks, May 2025. doi: 10.48550/arXiv.2505.20132.
- Dongyoon Han, Jiwhan Kim, and Junmo Kim. Deep pyramidal residual networks. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6307–6315, July 2017. doi: 10.1109/CVPR.2017.668.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the 2015 IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034. IEEE, December 2015. doi: 10.1109/ICCV.2015.123.

- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, June 2016. doi: 10.1109/CVPR.2016.90.
- Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366, March 1989. doi: 10.1016/0893-6080(89)90020-8.
- Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2261–2269, July 2017. doi: 10.1109/CVPR.2017.243.
- IEEE. IEEE standard for floating-point arithmetic. *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pages 1–84, July 2019. doi: 10.1109/IEEESTD.2019.8766229.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37, pages 448–456, July 2015. url: <https://proceedings.mlr.press/v37/ioffe15.html>.
- Jayesh Jain. What are tensors?, January 2023. https://medium.com/@jayeshjain_246/what-are-tensors-495cf37c18e6 accessed 12 May 2026.
- Alex Krizhevsky. Learning multiple layers of features from tiny images. April 2009. url: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012. url: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf.
- David C Lay, Steven R Lay, and Judi J McDonald. *Linear Algebra and Its Applications*. Pearson Education, 5 edition, 2016. ISBN 978-0-321-98238-4.
- Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, November 1998. doi: 10.1109/5.726791.
- Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, May 2015. doi: 10.1038/nature14539.
- Yoav Levine, David Yakira, Nadav Cohen, and Amnon Shashua. Deep Learning and Quantum Entanglement: Fundamental Connections with Implications to Network Design, April 2017. doi: 10.48550/arXiv.1704.01552.
- Charles H Martin and Michael W Mahoney. Implicit self-regularization in deep neural networks: Evidence from random matrix theory and implications for learning. *Journal of Machine Learning Research*, 22(1):7479–7551, January 2021. doi: 10.5555/3546258.3546423.
- Tom M. Mitchell. *Machine Learning*. McGraw-Hill, March 1997. ISBN 0-07-042807-7.
- Naoki Nakatani and Garnet Kin Chan. Efficient tree tensor network states (ttns) for quantum chemistry: Generalizations of the density matrix renormalization group algorithm. *The Journal of chemical physics*, 138(13), April 2013. doi: 10.1063/1.4798639.

- Yuval Netzer, Tao Wang, Adam Coates, A. Bissacco, Bo Wu, and A. Ng. Reading digits in natural images with unsupervised feature learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, 2011. url: https://ai.stanford.edu/~twangcat/papers/nips2011_housenumbers.pdf.
- Chang Nie, Junfang Chen, and Yajie Chen. Deep tree tensor networks for image recognition. February 2025. doi: 10.48550/arXiv.2502.09928.
- Alexander Novikov, Dmitrii Podoprikin, Anton Osokin, and Dmitry Vetrov. Tensorizing neural networks. In *Advances in Neural Information Processing Systems*, volume 28, December 2015. url: https://proceedings.neurips.cc/paper_files/paper/2015/file/6855456e2fe46a9d49d3d3af4f57443d-Paper.pdf.
- Keiron O'Shea and Ryan Nash. An introduction to convolutional neural networks. December 2015. doi: 10.48550/arXiv.1511.08458.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems 33*, pages 8026–8037, December 2019. url: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf.
- Michael T Pearce, Thomas Doods, Alice Rigg, Jose M Oramas, and Lee Sharkey. Bilinear MLPs enable weight-based mechanistic interpretability. June 2025. doi: 10.48550/arXiv.2410.08417.
- Vitali Petsiuk, Abir Das, and Kate Saenko. Rise: Randomized input sampling for explanation of black-box models. September 2018. doi: 10.48550/arXiv.1806.07421.
- Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "Why should i trust you?" explaining the predictions of any classifier. August 2016. doi: 10.48550/arXiv.1602.04938.
- David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, October 1986. doi: 10.1038/323533a0.
- Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, October 2017. doi: 10.1109/ICCV.2017.74.
- Edwin Stoudenmire and David J Schwab. Supervised learning with tensor networks. *Advances in neural information processing systems*, 29, May 2016. url: https://proceedings.neurips.cc/paper_files/paper/2016/file/5314b9674c86e3f9d1ba25ef9bb32895-Paper.pdf.
- Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*. MIT press Cambridge, 2 edition, 2018. 978-0-262-03924-6.
- Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2818–2826, June 2016. doi: 10.1109/CVPR.2016.308.
- Maolin Wang, Yu Pan, Zenglin Xu, Guangxi Li, Xiangli Yang, Danilo Mandic, and Andrzej Cichocki. Tensor Networks Meet Neural Networks: A Survey and Future Perspectives, May 2025. doi: 10.48550/arXiv.2302.09019.

- Ross Wightman, Hugo Touvron, and Hervé Jégou. Resnet strikes back: An improved training procedure in timm. October 2021. doi: 10.48550/arXiv.2110.00476.
- Sangdoo Yun, Dongyoon Han, Sanghyuk Chun, Seong Joon Oh, Youngjoon Yoo, and Junsuk Choe. Cutmix: Regularization strategy to train strong classifiers with localizable features. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, August 2019. doi: 10.1109/ICCV.2019.00612.
- Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. June 2017. doi: 10.48550/arXiv.1605.07146.
- Matthew D Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *European conference on computer vision*, pages 818–833, 2014.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in neural information processing systems* 33, pages 12381–12392, December 2019. https://proceedings.neurips.cc/paper_files/paper/2019/file/1e8a19426224ca89e83cef47f1e7f53b-Paper.pdf.
- Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. Mixup: Beyond empirical risk minimization. April 2018. doi: 10.48550/arXiv.1710.09412.
- Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, and Yi Yang. Random erasing data augmentation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 13001–13008, April 2020. doi: 10.1609/aaai.v34i07.7000.

Appendix A

Auxiliary proofs

A.1 Center of orthogonality decomposition

Let $\mathcal{A}$ be a center of orthogonality of a tree tensor network $\mathcal{T}$, and let $\mathcal{A}'$ be the local approximation of $\mathcal{A}$ resulting in tree tensor network $\mathcal{T}'$. Then the difference between the original tensor and its approximation, measured by the Frobenius norm $\|\mathcal{A} - \mathcal{A}'\|$, is equal to the global difference of the two tree tensor networks. This means that $\|\mathcal{A} - \mathcal{A}'\| = \|\mathcal{T} - \mathcal{T}'\|$. [Evenbly, 2022]

Proof. The proof relies on the definition of a center of orthogonality. When a tensor $\mathcal{A}$ is a center of orthogonality, all branches connected to it are isometric. Contracting the entire network $\mathcal{T}$ with its conjugate makes all branches become the identity, leaving only the contraction of $\mathcal{A}$ with $\mathcal{A}^\dagger$. This implies that

$$\text{Ttr}(\mathcal{A}\mathcal{A}^\dagger) = \text{Ttr}(\mathcal{T}\mathcal{T}^\dagger)$$

and by definition of the Frobenius norm follows

$$\|\mathcal{A}\| = \|\mathcal{T}\|$$

If $\mathcal{A}$ is replaced by a local approximation $\mathcal{A}'$, with the corresponding tensor network denoted $\mathcal{T}'$, the same argument shows that

$$\|\mathcal{A}'\| = \|\mathcal{T}'\|$$

Combining these two results yields the conclusion

$$\|\mathcal{A} - \mathcal{A}'\| = \|\mathcal{T} - \mathcal{T}'\|$$

The first part of this proof is illustrated in figure A.1 in which a network $\mathcal{T}$ is contracted with its conjugate, making all branches equal to the identity. The equalities are true under the assumption that $\mathcal{A}$ is a center of orthogonality. This result justifies the iterative use of the SVD in multi-stage decompositions; at each step, one may apply a truncated SVD to the current center of orthogonality, knowing that the resulting truncation error is globally minimised, under the assumption that the orthogonality condition is maintained throughout this procedure. $\square$

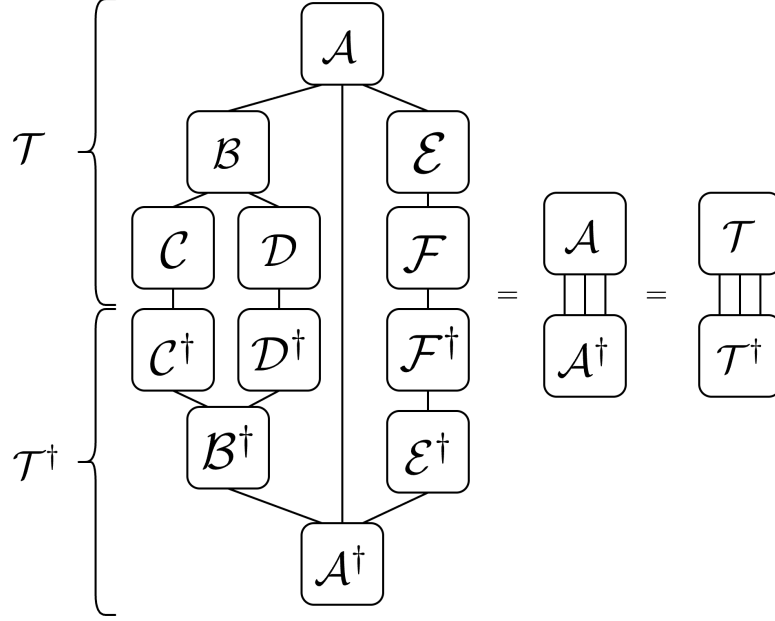


Figure A.1: Visual proof that $\text{Ttr}(\mathcal{A}\mathcal{A}^\dagger) = \text{Ttr}(\mathcal{T}\mathcal{T}^\dagger)$ in an arbitrary tree tensor network with $\mathcal{A}$ a center of orthogonality.

A.2 RMS normalisation scaling property

For any scalar $c > 0$ holds $\text{RMS}(\frac{x}{c}) = \frac{\text{RMS}(x)}{c}$.

Proof. This follows directly from the definition of the root mean square. We compute:

$$\text{RMS}\left(\frac{x}{c}\right) = \sqrt{\frac{1}{n} \sum_{i=1}^n \left(\frac{x_i}{c}\right)^2} = \sqrt{\frac{1}{n} \sum_{i=1}^n \frac{x_i^2}{c^2}} = \sqrt{\frac{1}{c^2} \cdot \frac{1}{n} \sum_{i=1}^n x_i^2}.$$

Since $c > 0$, we can take the constant out of the square root:

$$\text{RMS}\left(\frac{x}{c}\right) = \frac{1}{c} \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2} = \frac{\text{RMS}(x)}{c}.$$

□

Appendix B

Hyperparameters

In this chapter, we discuss the hyperparameters of the model in greater detail. We explain the role of the hyperparameters we did not mention in subsection 3.1.6, and for each dataset we present the used hyperparameter configuration.

B.1 Hyperparameter definitions

Subsection 3.1.6 already covers the hyperparameters related to the proposed architecture. The hyperparameters related to training are

- Optimiser: optimisation algorithm used to update the model parameters during training.
- Learning rate: starting learning rate used by the optimiser.
- Momentum: hyperparameter specific to the Stochastic Gradient Descent optimiser.
- Weight decay: regularisation term controlling the amount of L2 regularisation applied to the model parameters.
- Batch size: number of samples used to compute the gradients at each update step.
- Epochs: total number of complete iterations over the entire training dataset.
- Warmup epochs: number of initial epochs in which the learning rate is gradually increased from zero to the starting learning rate.
- Learning rate scheduler: scheduler used to decrease the learning rate during training.
- Warmup learning rate scheduler: scheduler used to gradually increase the learning rate during the warmup epochs.
- Label smoothing: label smoothing term controlling how the class labels are softened during training [Szegedy et al., 2016].

The hyperparameters related to the used data augmentation techniques are

- Normalisation mean: per-channel mean used to normalise the input images.
- Normalisation standard deviation: per-channel standard deviation used to normalise the input images.

- Random crop size¹: size of the crop randomly extracted from the input image.
- Random crop padding²: size of the padding around the image added before applying the random crop.
- Random horizontal flip probability³: probability determining whether the input image is horizontally flipped before training.
- Random erasing probability: probability determining whether a random rectangular region of the image is erased before training [Zhong et al., 2020].
- RandAugment number of operations: number of augmentation operations randomly applied to each image [Cubuk et al., 2020].
- RandAugment magnitude: the strength of each augmentation operation applied by RandAugment [Cubuk et al., 2020].
- Cutmix α : parameter controlling the Beta distribution used to sample the mixing ratio. This ratio determines the proportion of the image that is cut and replaced with a patch from another image [Yun et al., 2019].
- MixUp α : parameter controlling the Beta distribution used to sample the mixing coefficient. This coefficient determines how strongly two images and their corresponding labels are blended [Zhang et al., 2018].

B.2 Hyperparameter values

In this section, we present the hyperparameter values used for each dataset. Each configuration is a table consisting of three parts; the first part covers the hyperparameter values of the architecture, the second part covers the hyperparameter values related to training, and the last part covers the hyperparameter values related to the data augmentation techniques used. If a hyperparameter is not listed within the table, it means that it was not used.

¹<https://docs.pytorch.org/vision/main/generated/torchvision.transforms.RandomCrop.html>

²<https://docs.pytorch.org/vision/main/generated/torchvision.transforms.RandomCrop.html>

³<https://docs.pytorch.org/vision/main/generated/torchvision.transforms.RandomHorizontalFlip.html>

Table B.1: Hyperparameters used for training the proposed architecture on MNIST.

Hyperparameter	Value
Number of stages	2
Blocks per stage	2
Upscale factor	3
Channels factor	3
Starting output channels	16
Kernel size	3
RMS normalisation mode	Global
Repeat factor	2
Bias	None
Optimiser	SGD
Learning rate	10^{-2}
Momentum	0.9
Batch size	128
Epochs	7
Warmup epochs	3
Learning rate scheduler	Cosine
Warmup learning rate scheduler	Linear

Table B.2: Hyperparameters used for training the proposed architecture on SVHN.

Hyperparameter	Value
Number of stages	4
Blocks per stage	3
Upscale factor	2
Channels factor	2
Starting output channels	32
Kernel size	3
RMS normalisation mode	Global
Repeat factor	2
Bias	None
Optimiser	AdamW
Learning rate	10^{-4}
Weight decay	5×10^{-4}
Batch size	128
Epochs	50
Warmup epochs	10
Learning rate scheduler	Cosine
Warmup learning rate scheduler	Linear
Label smoothing	0.05
Normalisation mean	(0.4377, 0.4438, 0.4728)
Normalisation standard deviation	(0.1980, 0.2010, 0.1970)

Table B.3: Hyperparameters used for training the proposed architecture on CIFAR-10.

Hyperparameter	Value
Number of stages	4
Blocks per stage	3
Upscale factor	2
Channels factor	2
Starting output channels	32
Kernel size	3
RMS normalisation mode	Global
Repeat factor	2
Bias	None
Optimiser	AdamW
Learning rate	10^{-3}
Weight decay	5×10^{-4}
Batch size	128
Epochs	300
Warmup epochs	5
Learning rate scheduler	Cosine
Warmup learning rate scheduler	Linear
Label smoothing	0.05
Normalisation mean	(0.4914, 0.4822, 0.4465)
Normalisation std	(0.2023, 0.1994, 0.2010)
Random crop size	32×32
Random crop padding	4
Random horizontal flip probability	0.5
Random erasing probability	0.25

Table B.4: Hyperparameters used for training the proposed architecture on CIFAR-100.

Hyperparameter	Value
Number of stages	4
Blocks per stage	3
Upscale factor	2
Channels factor	2
Starting output channels	32
Kernel size	3
RMS normalisation mode	Global
Repeat factor	2
Bias	None
Optimiser	SGD
Learning rate	8×10^{-3}
Momentum	0.9
Weight decay	1.5×10^{-3}
Batch size	128
Epochs	300
Warmup epochs	10
Learning rate scheduler	Cosine
Warmup learning rate scheduler	Linear
Label smoothing	0.15
Normalisation mean	(0.5071, 0.4867, 0.4408)
Normalisation std	(0.2675, 0.2565, 0.2761)
Random crop size	32×32
Random crop padding	4
RandAugment number of operations	2
RandAugment magnitude	5
Random erasing probability	0.1

Table B.5: Hyperparameters used for testing the proposed architecture on CIFAR-100 with channel-wise RMS normalisation.

Hyperparameter	Value
Number of stages	4
Blocks per stage	3
Upscale factor	2
Channels factor	2
Starting output channels	32
Kernel size	3
RMS normalisation mode	Channel-wise
Repeat factor	2
Bias	None
Optimiser	SGD
Learning rate	8×10^{-3}
Momentum	0.9
Weight decay	1.5×10^{-3}
Batch size	128
Epochs	300
Warmup epochs	10
Learning rate scheduler	Cosine
Warmup learning rate scheduler	Linear
Label smoothing	0.15
Normalisation mean	(0.5071, 0.4867, 0.4408)
Normalisation std	(0.2675, 0.2565, 0.2761)
Random crop size	32×32
Random crop padding	4
RandAugment number of operations	2
RandAugment magnitude	5
Random erasing probability	0.1

Appendix C

Weight-based interpretability examples

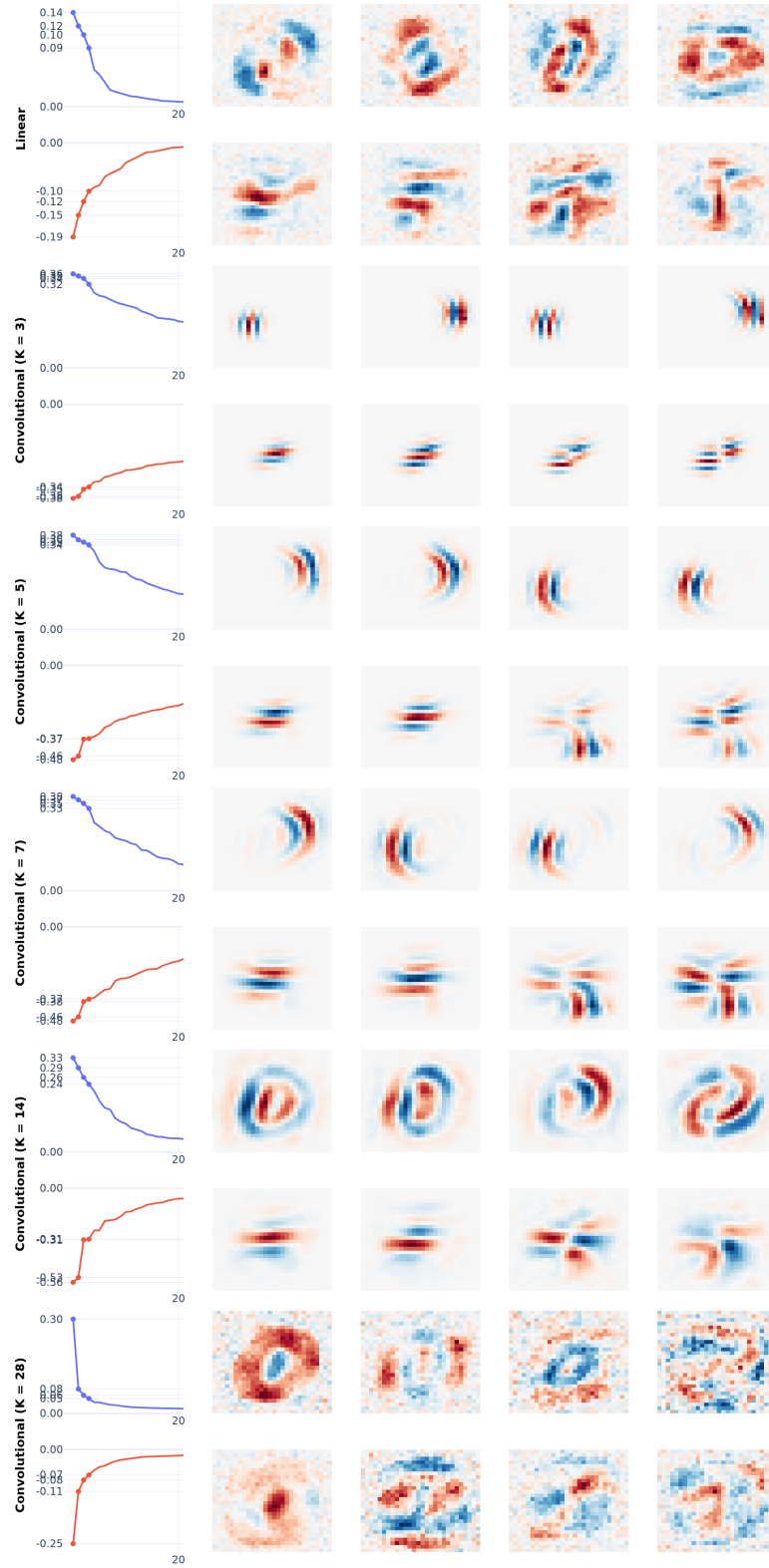


Figure C.1: Eigenspectrum analysis digit 0.

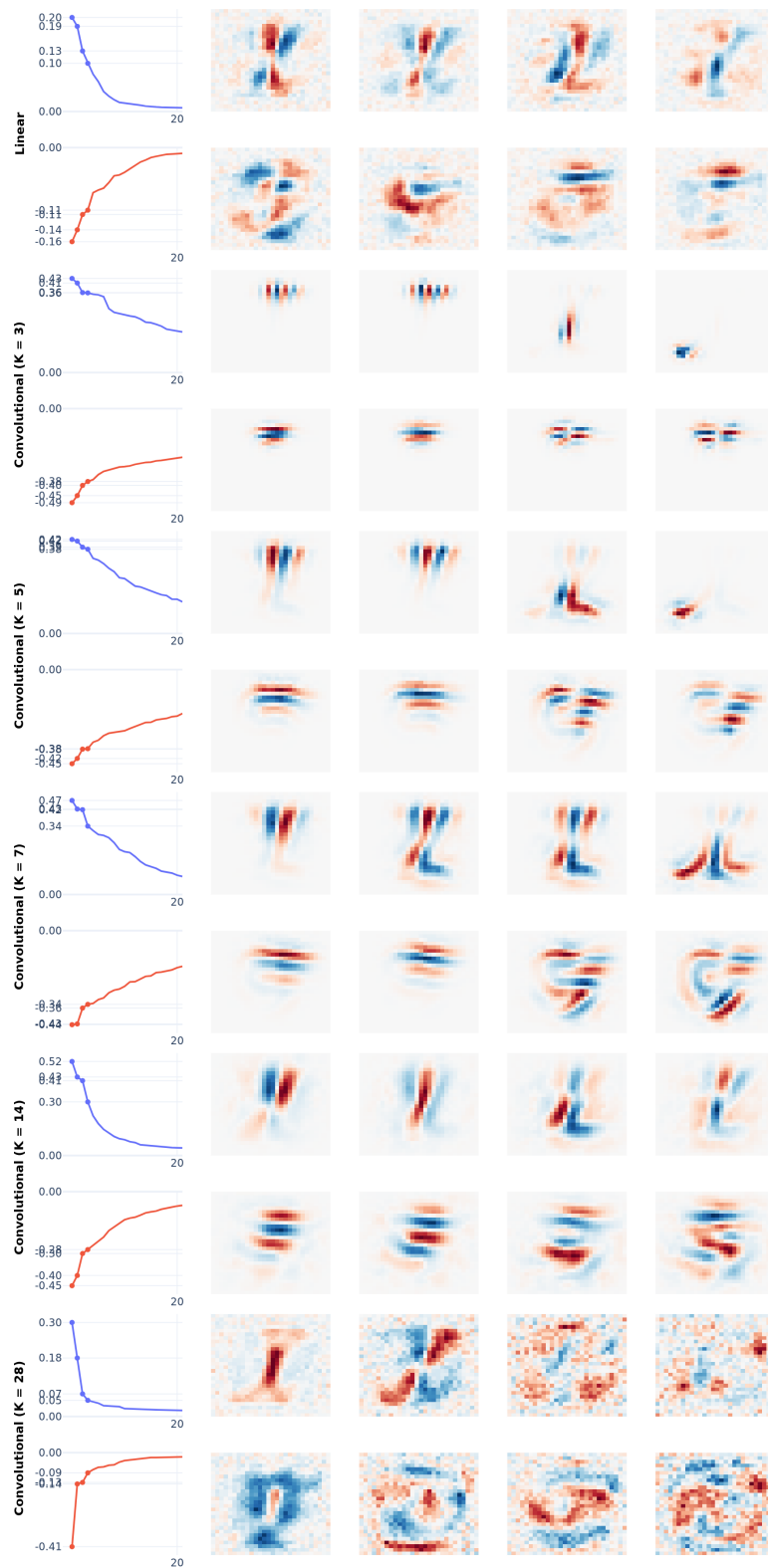


Figure C.2: Eigenspectrum analysis digit 1.

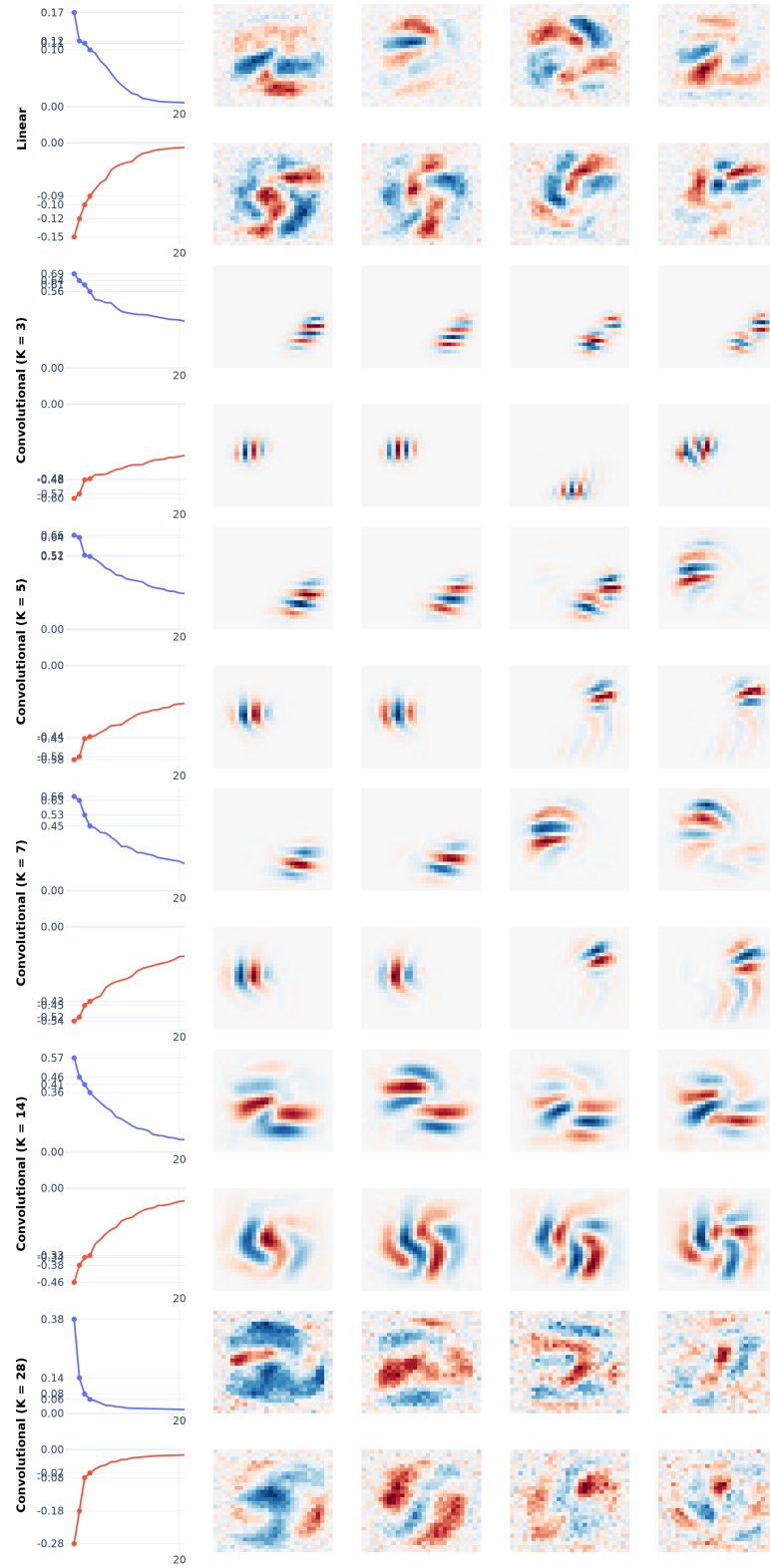


Figure C.3: Eigenspectrum analysis digit 2.

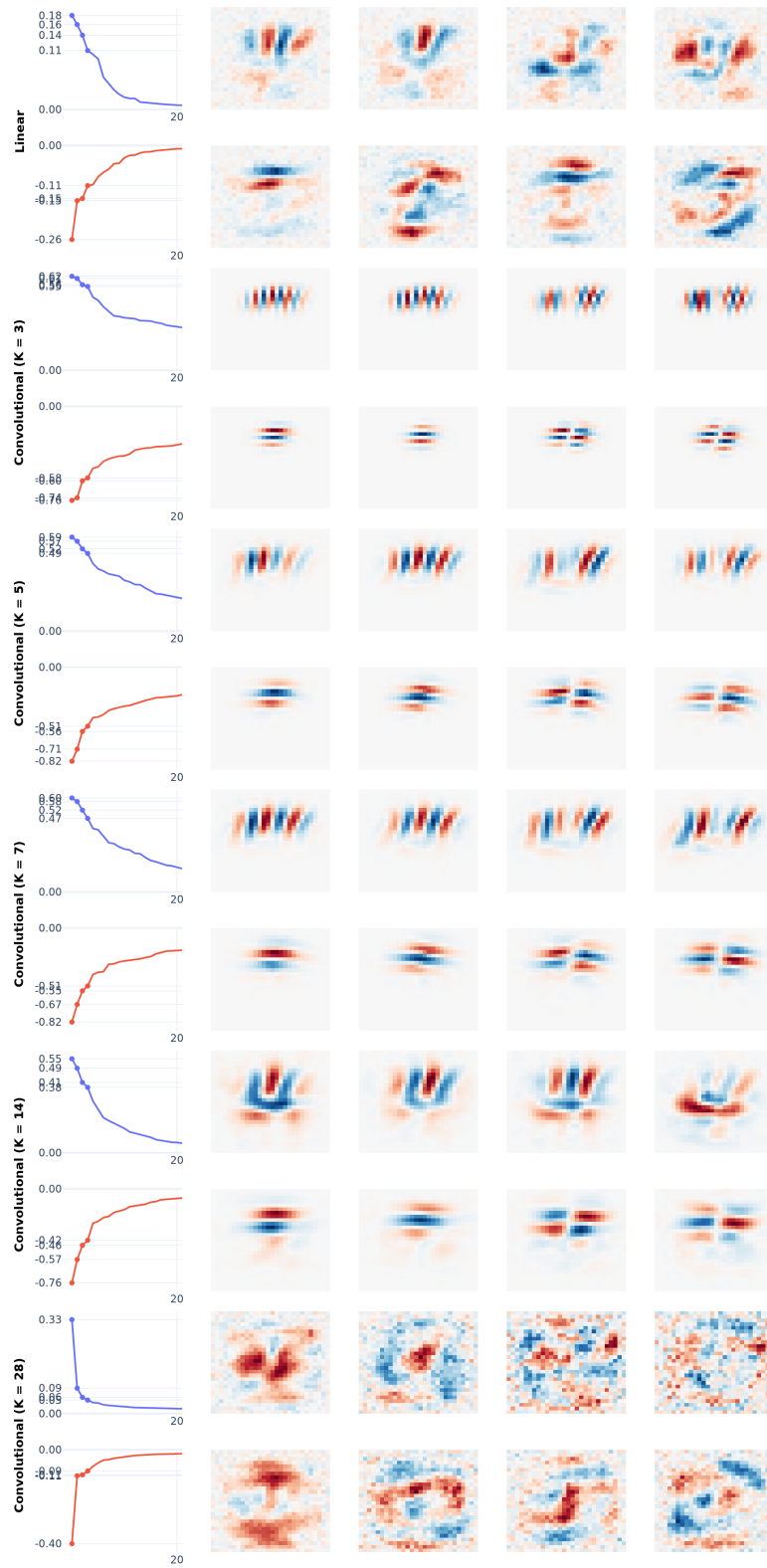


Figure C.4: Eigenspectrum analysis digit 4.

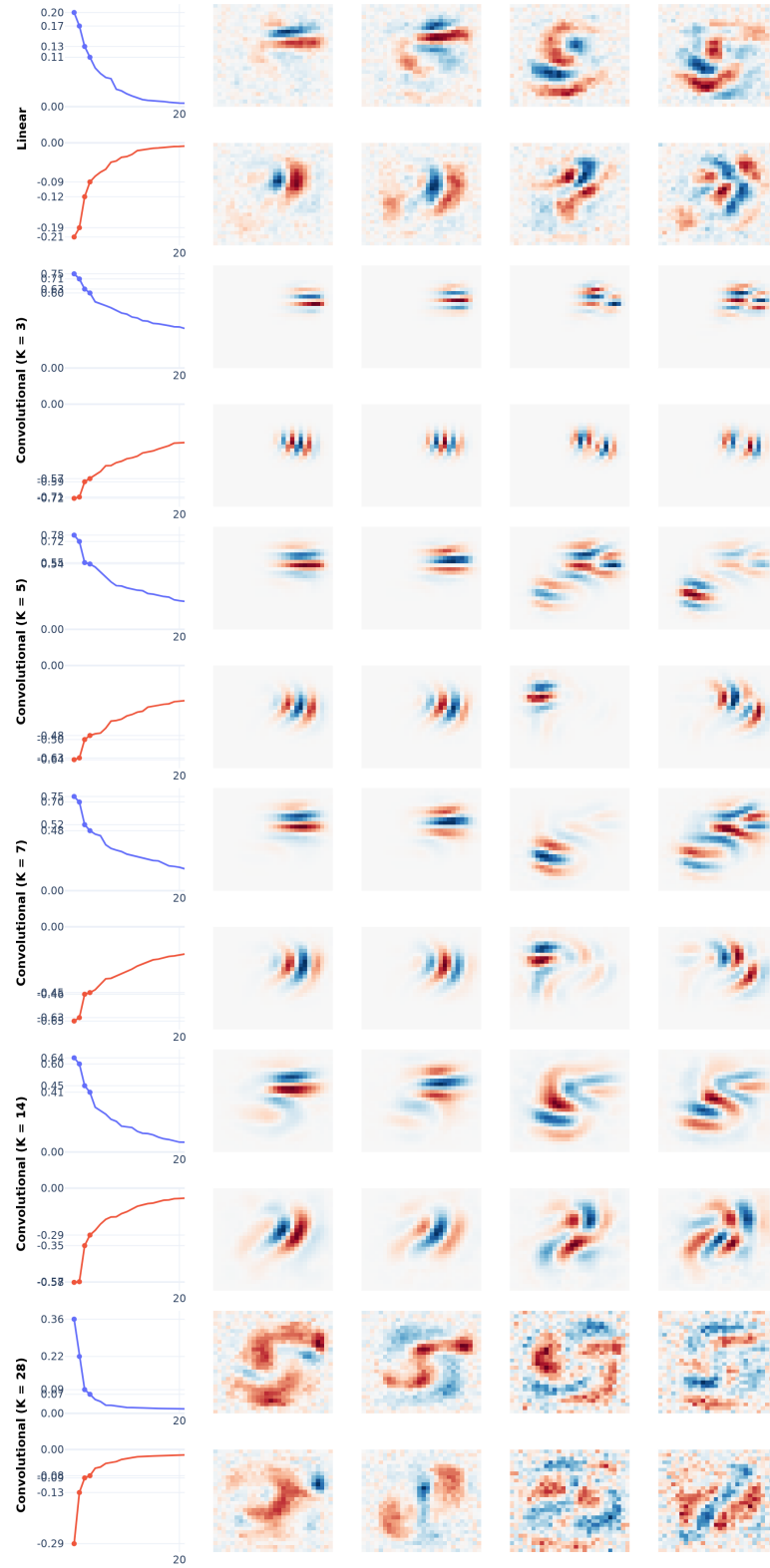


Figure C.5: Eigenspectrum analysis digit 5.

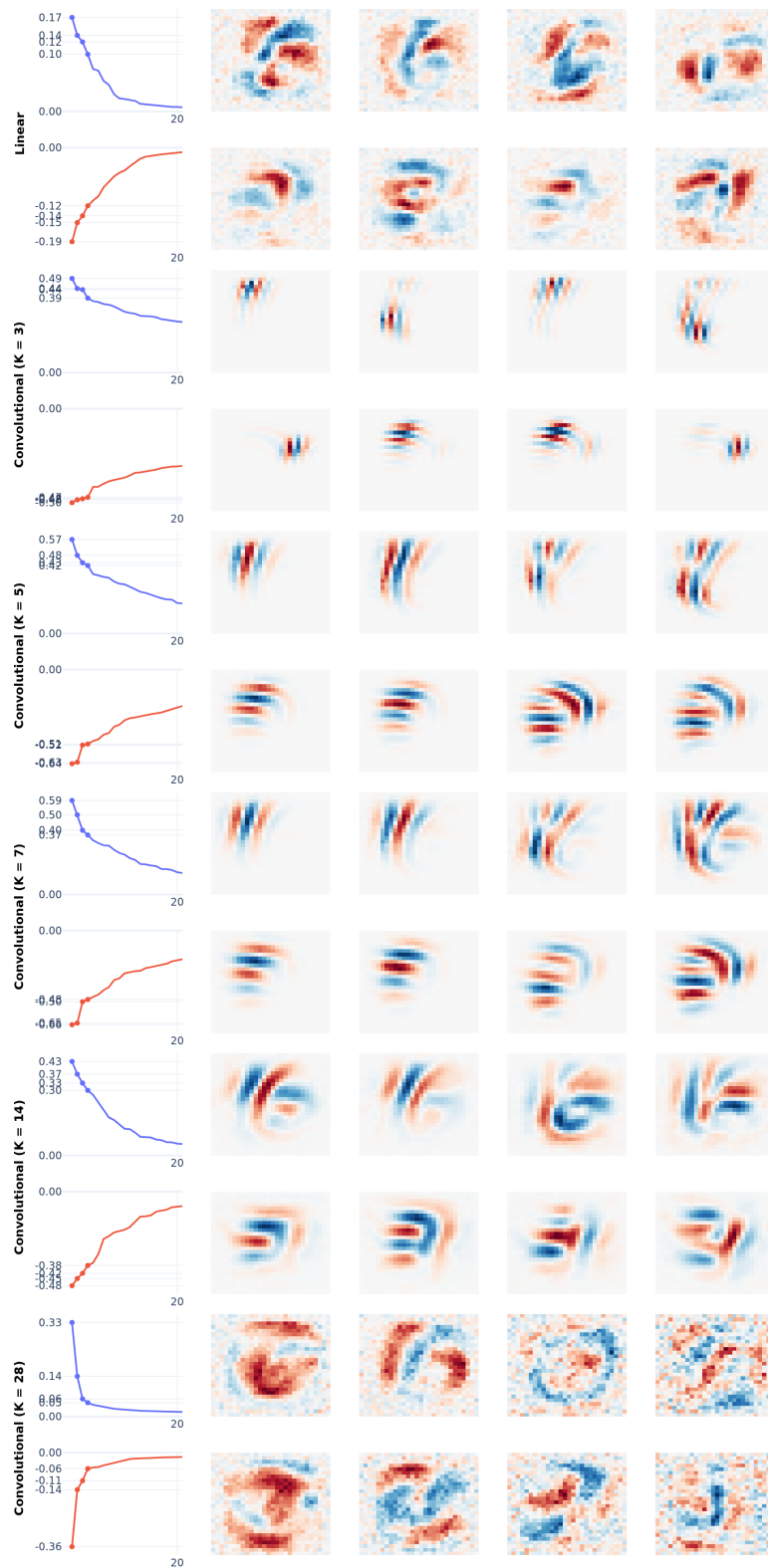


Figure C.6: Eigenspectrum analysis digit 6.

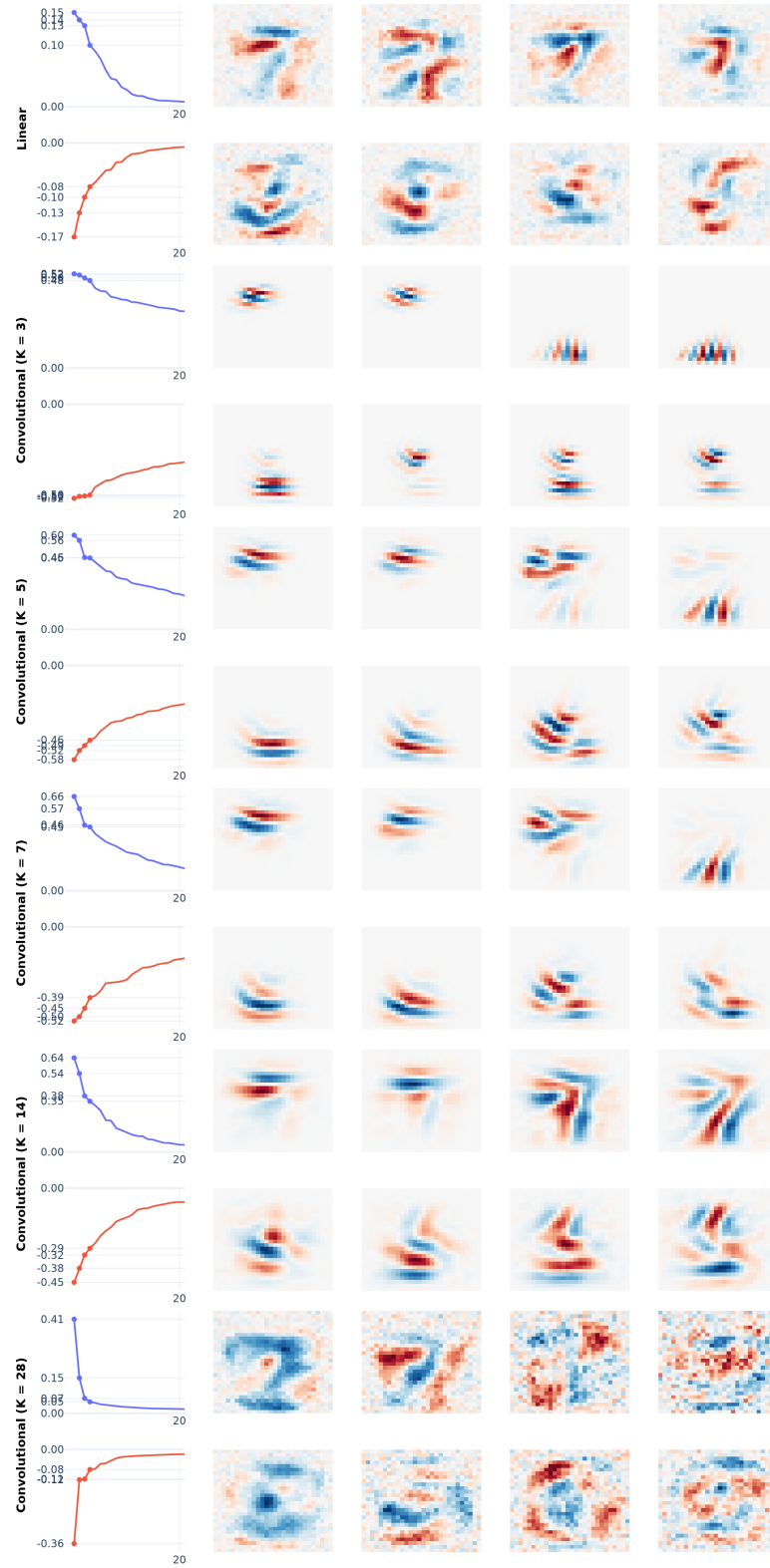


Figure C.7: Eigenspectrum analysis digit 7.

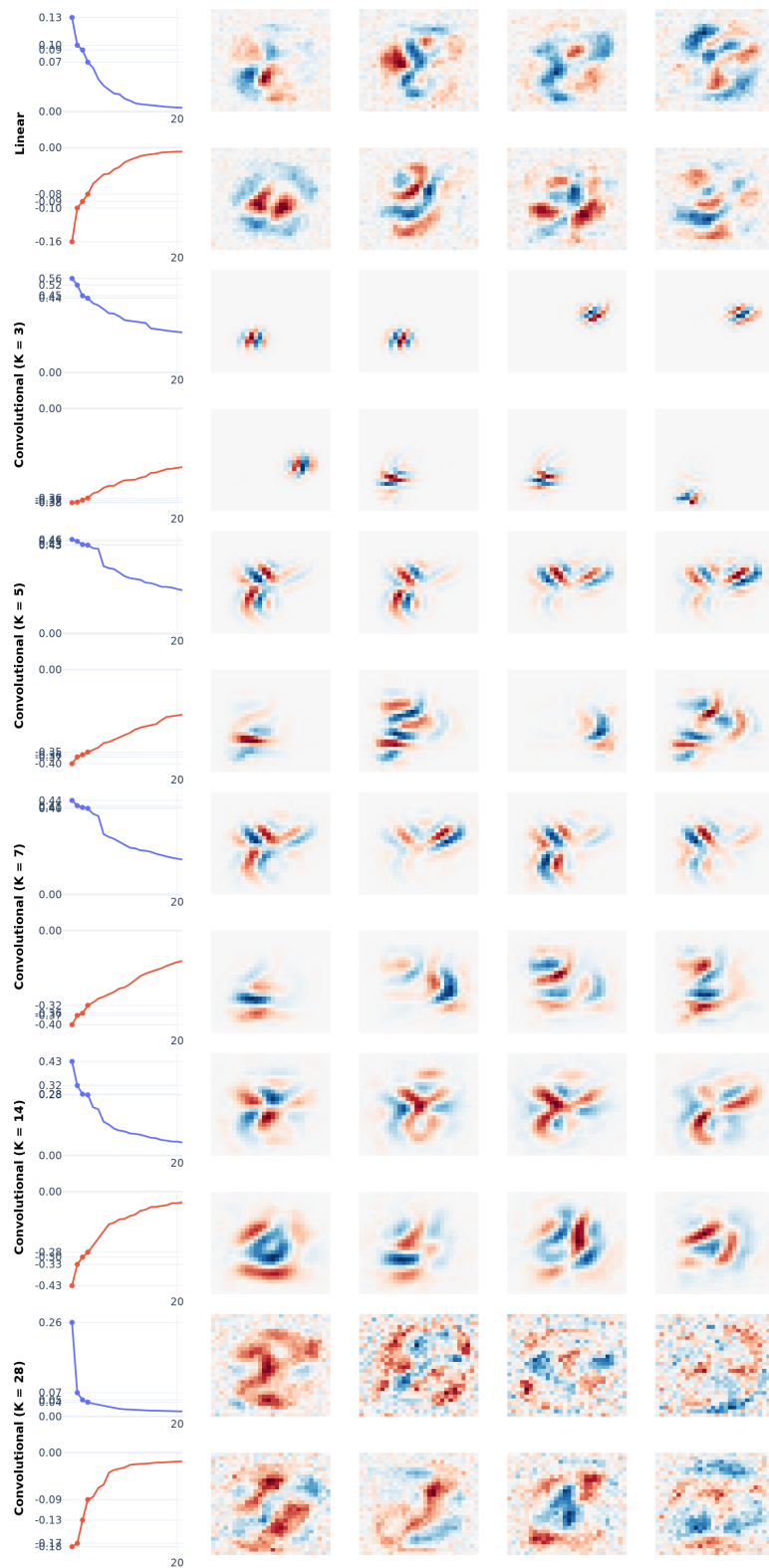


Figure C.8: Eigenspectrum analysis digit 8.

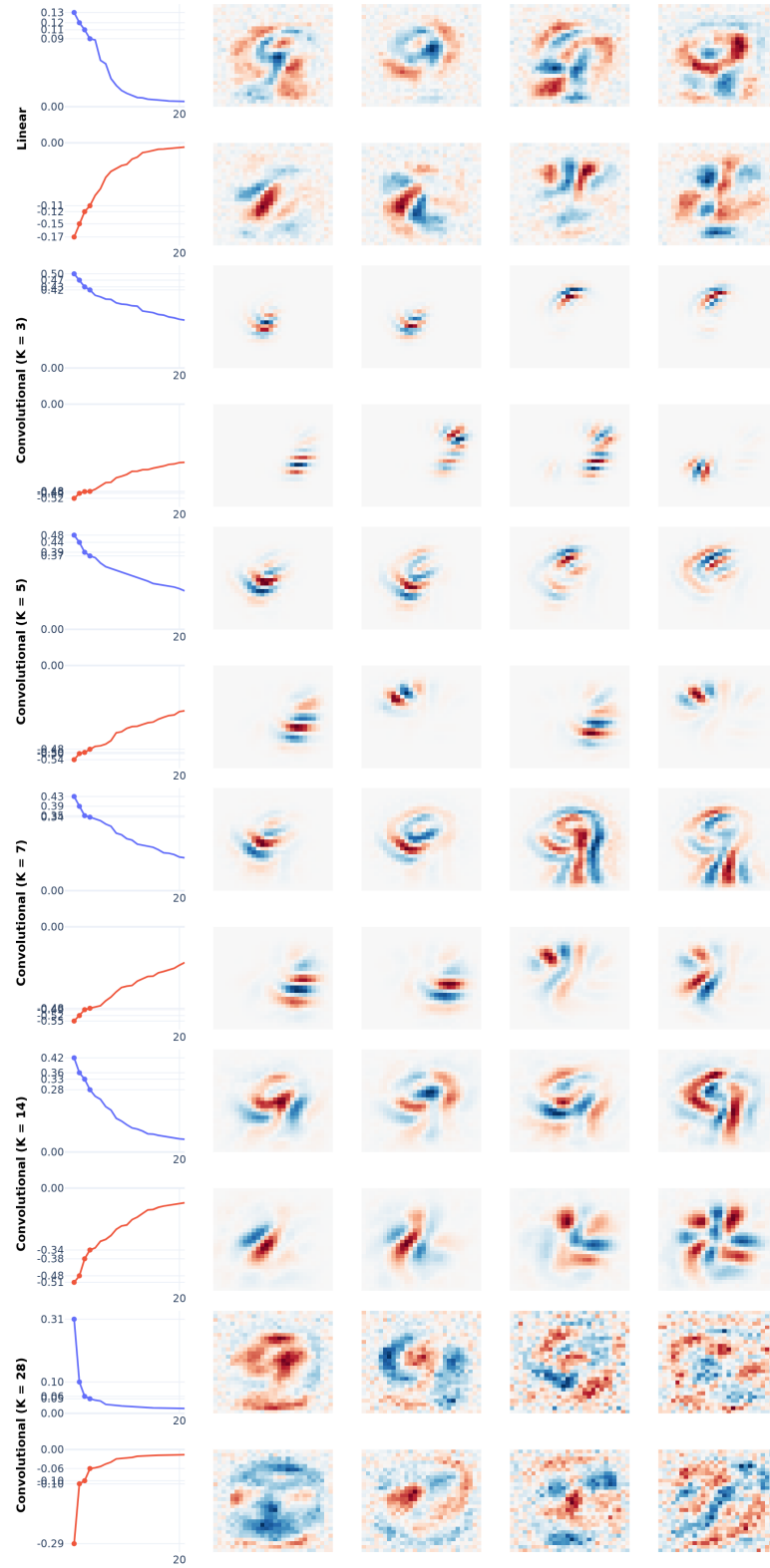


Figure C.9: Eigenspectrum analysis digit 9.